

KAMP 제조AI데이터셋 활용 권한

No. GB20260921-PU1789564111-364764



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 전자부품(배터리팩) 예지보전 AI 데이터셋, 스마트제조혁신추진단((주)인터엑스, 네스트필드(주)), 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Electronic components(battery pack) predictive maintenance AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 전자부품(배터리팩) 예지보전 AI 데이터셋

업종 : 전자부품 | **목적** : 예지보전 | **유형** : csv

알고리즘 : N-HITS, 통계분포 Z-score 알고리즘

사용조건 : 콘텐츠 변경허용

제공기관 : 스마트제조혁신추진단 (수행기관 : (주)인터엑스/네스트필드(주))

등록일 : 2022-12-23

이용자 정보

회원 ID : MSS_zas357

다운로드일 : 2026-09-21 14:45:27

활용목적 : 제조AI 솔루션 개발 및 고도화

인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

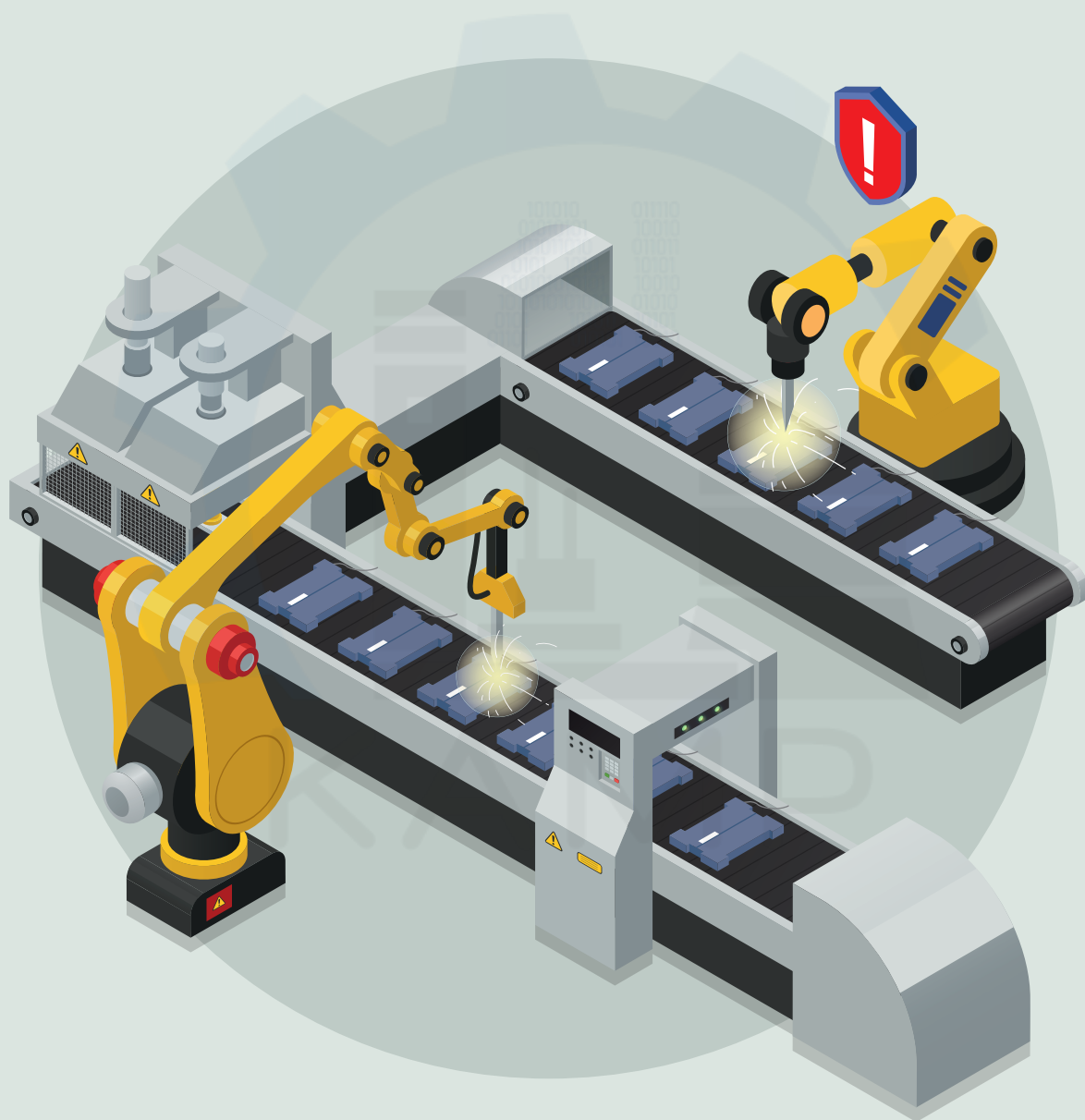
운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

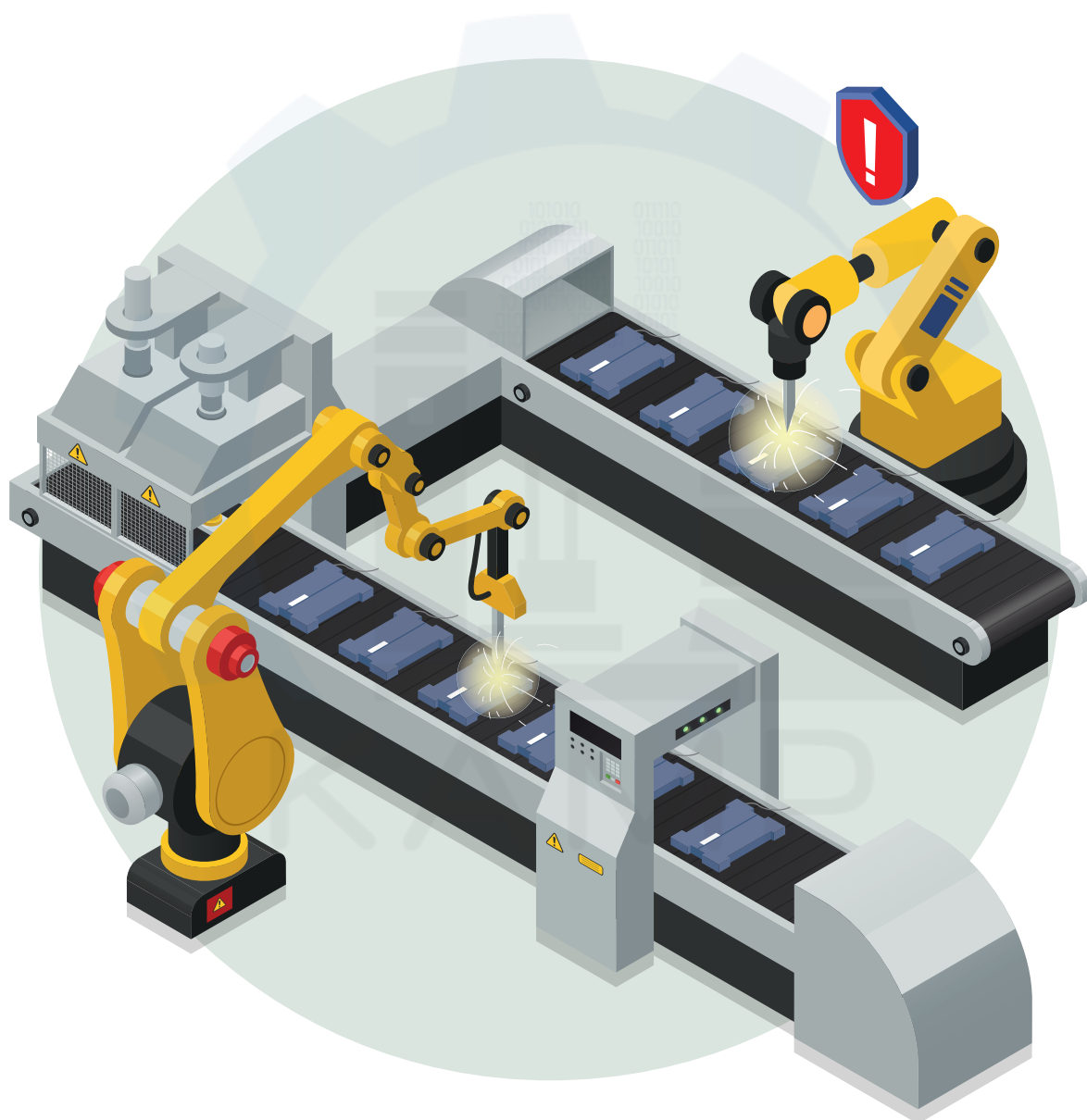
「전자부품(배터리팩) 예지보전 AI 데이터셋」

분석실습 가이드북



「전자부품(배터리팩) 예지보전 AI 데이터셋」

분석실습 가이드북





Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	08
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	20
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	23
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 데이터 준비	
[단계 ②] 데이터 구조 및 특징 탐색	
[단계 ③] 학습, 테스트 데이터 선택	
[단계 ④] AI 모델 알고리즘 선택	
[단계 ⑤] AI 모델 학습	
[단계 ⑥] 테스트셋에 대한 계산 수행	
[단계 ⑦] 테스트 데이터 결과 분석	
[단계 ⑧] 알고리즘 평가	
3. 유사 타 현장의 「전자부품(배터리팩) 예지보전 AI 데이터셋」 분석 적용	87
부 록	
분석환경 구축을 위한 설치 가이드	89
데이터 품질 전처리 [실습 코드]	98
부록_AAS S/W 설치 가이드	104



「전자부품(배터리팩) 예지보전 AI 데이터셋」

분석실습 가이드북

- **필요 SW** : Python, Anaconda, Jupyter Notebook
- **필요 패키지** : matplotlib, pandas, numpy, plotly, mpl_toolkits, sklearn, random, datetime, math, os, logging, torch, cpflows, pytorch_lightning, pytorch_forecasting
- **분석 환경** : [CPU] AMD Ryzen 9 5950X, [Memory] 128GB, [GPU] NVIDIA GeForce RTX 3090
- **필요 데이터** : WeldingTest_0X_OK.csv, WeldingTest_0X_NG.csv, Training_Data.csv

1 분석요약

No	구분		내용
1	분석 목적 (현장 이슈, 목적)		본 분석에서는 배터리팩 품질에 영향을 미치는 용접설비를 대상으로 하며, AAS 표준기반 제조데이터 수집/저장 체계를 통해 공정 데이터를 수집하여 분석에 사용하는 방법과 AI 기법(Neural Hierarchical Interpolation for Time Series Forecasting, N-HiTS)을 통해 설비의 이상을 예측하여 불량률 검출할 수 있는 설비 이상 예측 모델을 만드는 과정을 학습할 수 있다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : RealPower, SetPower, GateOnTime 외 4개 2) 데이터 수집 방법 : AAS 기반 제조데이터 수집/저장 체계를 통해 설비공정 TSDB 추출 3) 데이터셋 파일 확장자 : csv
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 1,268,865개 - 데이터셋 총량 : 7.96 MB
4	분석적용 알고리즘	알고리즘	N-HiTS, 통계분포 Z-score 알고리즘
		알고리즘 간략소개	N-HiTS (Neural Hierarchical Interpolation for Time Series Forecasting)알고리즘은 다변량 시계열의 미래값을 높은 정확도로 예측하기 위하여 최근에 고안된 알고리즘이다. 이 알고리즘의 특징은 시계열 데이터에 대한 샘플링 주기를 다르게 구성하여 긴 시간의 거동과 짧은 시간의 거동을 따로 예측하고 이들을 결합하는 식으로 이루어져 있어 효율적인 메모리 사용과 빠른 계산 속도를 가능하게 한다. 또한, 데이터값의 통계분포로부터 Z-score를 구하여 이로부터 이상값을 탐지한다.
5	분석결과 및 시사점		- 현장 설비로부터 수집되는 공정 데이터를 활용하여, 설비의 이상 상태를 사전에 예측할 수 있는 공정 이상 예측 시스템을 개발하였다. - 실제 본 분석을 E공장에 적용함으로써 생산성 향상과 함께 품질향상에 기여했다는 점에서 시사점이 크다고 판단된다.

2 분석 실습

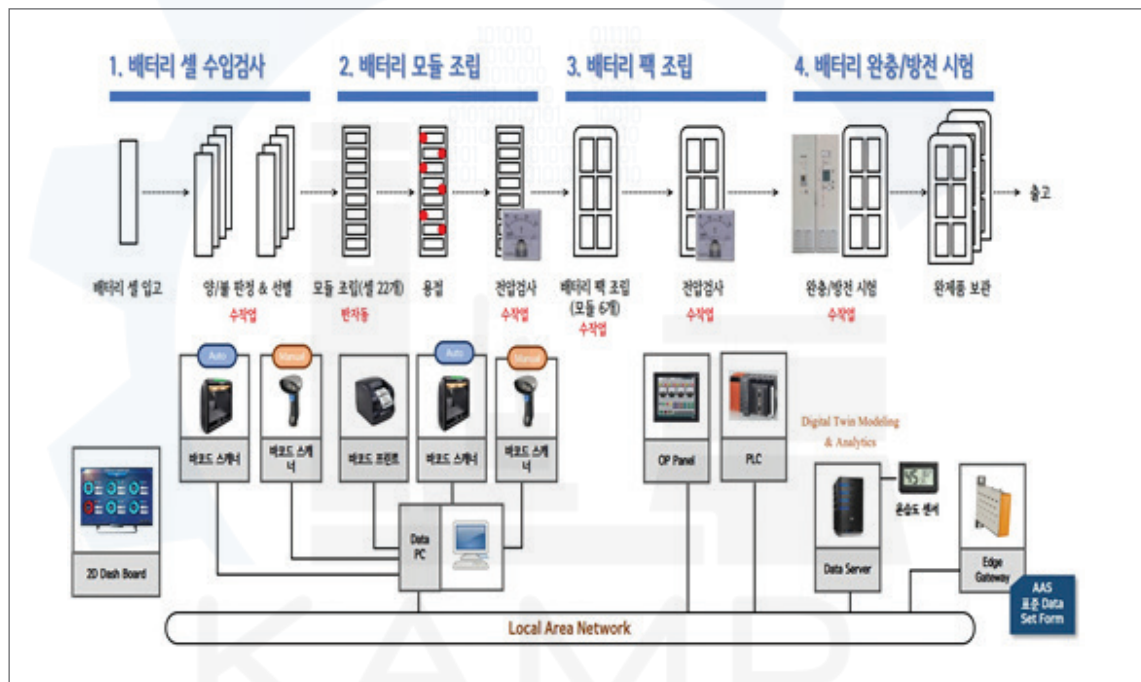
1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

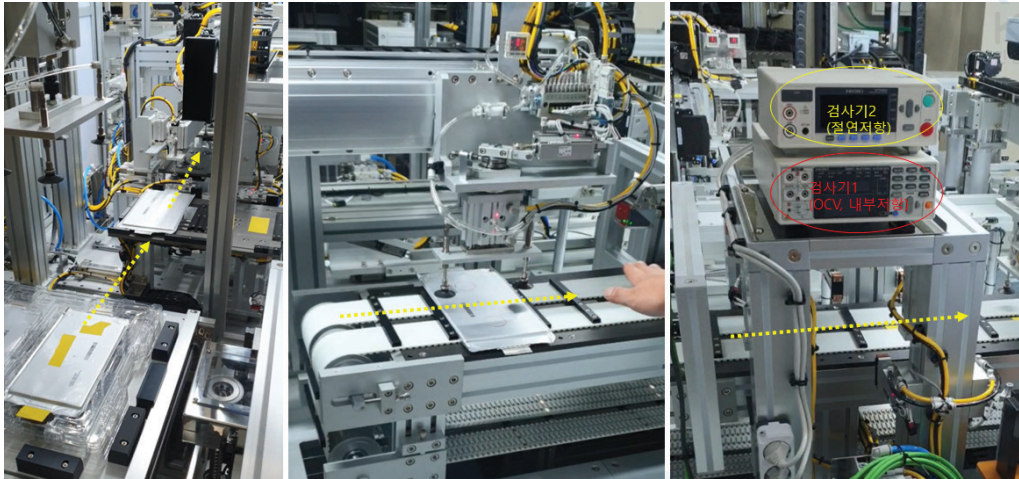
• 공정(설비) 정의 및 특징

- 전기버스 및 전기상용차에 사용되는 배터리팩을 조립/생산하는 시스템으로 최소 단위인 배터리셀을 검사/조립하는 공정, 그리고 양품인 배터리셀들을 모듈화하여 조립/용접/검사하는 공정, 마지막으로 완성된 배터리모듈을 모아 조립/검사/시험 공정을 거쳐 배터리팩이 완성된다.

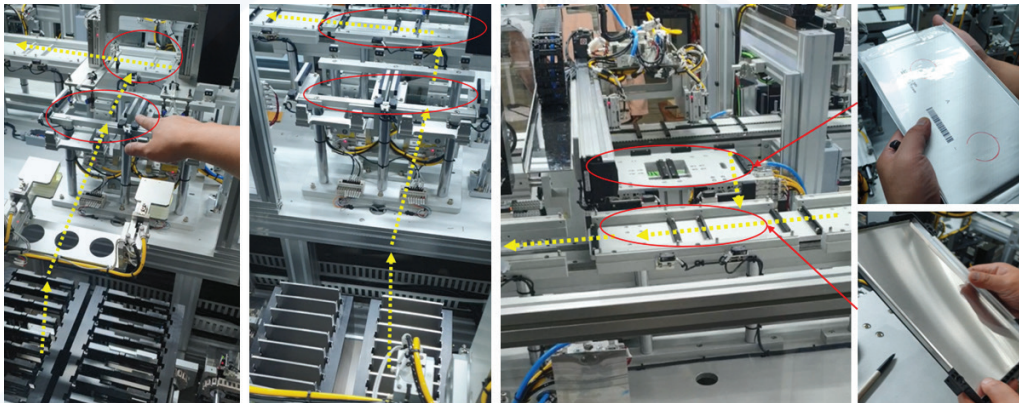


[그림 1] 배터리팩 제조공정 구성

- 배터리셀 검사/조립: 입고된 배터리셀을 검사기를 통해 OCV, 절연저항, 내부저항을 측정하여 양/불 여부를 판정하고, 양품 판정받은 배터리셀을 배터리모듈에 장착할 수 있도록 조립하는 공정이다.



[그림 2] 배터리셀 입고/정렬 및 양품검사



[그림 3] 배터리셀 카트리지/히트싱크 정렬 및 조립

- 배터리모듈 조립: 배터리셀 조립라인에서 출고된 배터리셀을 모아 하나의 모듈로 조립하는 공정이다. 배터리모듈 조립은 반자동으로 진행되며, 조립 자체는 작업자를 통해서 용접은 용접라인에서 자동으로 진행된다. 용접을 마친 배터리모듈은 다시 작업자를 통해 양/불 여부를 판정받고 이상 없을 시 BMS(Battery Management System) 모듈을 장착하여 배터리팩 조립라인으로 출고된다.



[그림 4] 배터리모듈 조립/용접 및 양품검사

- 배터리팩 조립: 양품 판정을 받아 출고된 배터리모듈을 배터리팩 케이스 안에 조립하는 공정이다. 배터리모듈 설치가 완료된 배터리팩은 완전히 조립하기 전 충방전 시험을 거치게 되고 양품인 경우 배터리팩 케이스(덮개)까지 완전히 조립하여 납품한다. 해당 공정에서는 조립/운송 모두 작업자에 의존한다.

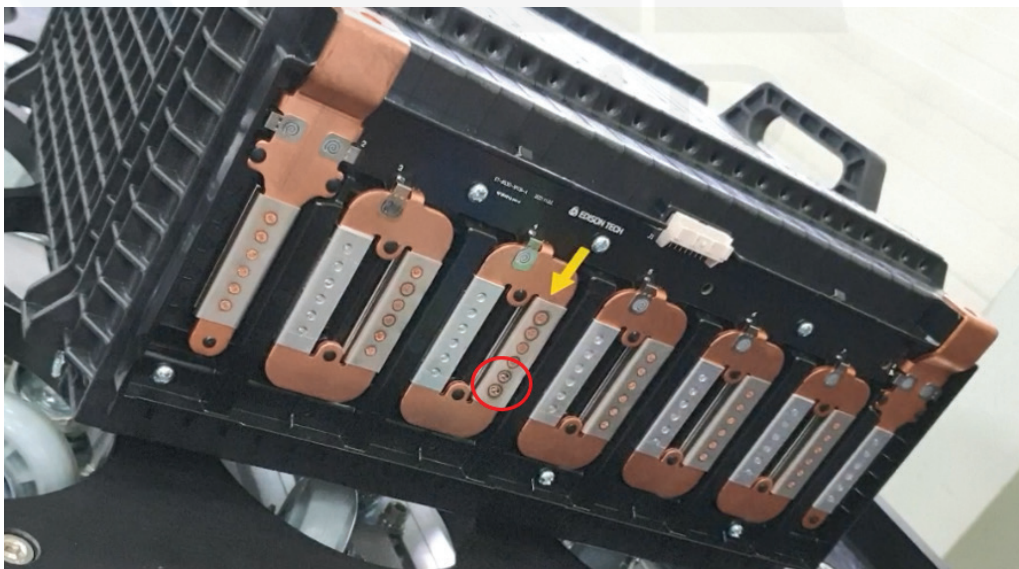


[그림 5] 배터리팩 케이스 조립 및 충방전 시험대기

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

- 설비 이상 발생 시 생산 지연이 발생하는 것을 피할 수 없으나 사전에 이상 조짐을 감지하여 미리 대비/점검 할 수 있다면 피해를 최소화 할 수 있다. 이를 위해 현장에서 수집되는 다양한 공정 데이터를 분석하여 이상 상태를 판단할 수 있는 기준이 필요하다. 본 분석에서는 배터리팩 조립에 들어가는 다양한 설비 공정 중 불량 또는 이상이 빈번하게 발생하고 있는 레이저 용접 설비를 대상으로 AI 모델을 적용하여 극복하고자 한다.



[그림 6] 배터리모듈 용접 불량 사례



• 문제해결 장애요인

- 현재 제조회장에서 사용되고 있는 레이저 용접설비는 배터리모듈을 구성하는 다수의 배터리셀 단자와 센싱라인을 용접하기 위해 사용되며, 용접 가능한 형태로 조립된 배터리모듈을 설비에 투입하여 설정된 명령에 따라 지정 포인트를 용접하는 방식으로 자동화되어 운용되고 있다. 그러나 용접 가능한 형태로 배터리모듈을 조립하여 설비에 투입하는 과정이 수작업으로 진행되기 때문에 설비 이상으로 인해 불량 발생 것인지 아니면 조립과정에서 불량 발생하여 용접에 영향을 미친 것인지 판단하는데 어려움이 있다.

• 극복 방안

- 용접공정에서 발생하는 주요 공정 데이터 수집 및 불량 데이터 패턴 분석을 위해 출고 검사 인원을 투입한 후 용접이 완료된 배터리모듈을 검사하여 불량 판정을 받은 날짜와 시간 그리고 용접 설비의 알람 발생 이력을 시스템에 입력하여 기록한다. 분석자는 실시간 공정 데이터와 용접 불량 발생 날짜와 시간 그리고 설비 알람 이력의 상관관계를 분석하고 이를 기반으로 정상 상태와 비정상 상태로 분류된 데이터셋을 활용하여 만든 AI 학습모델을 적용하여 불량 발생 원인을 찾아내고 용접 설비의 이상 상태를 예측할 수 있다.

1.2 분석 목표

1) 분석 목표

- 레이저 용접 설비부터 수집된 공정 데이터를 기반으로 배터리 품질과의 상관성에 대한 종합적인 분석을 통해 설비의 이상 상태를 사전에 감지하고 현장 작업자가 즉시 확인하고 조치할 수 있는 기반을 마련할 수 있다.

2) 데이터 정의 및 소개

- 본 분석에서 레이저 용접 설비의 공정 데이터 및 설비 상태정보를 AAS 표준기반 제조 데이터 수집/저장 체계로 수집하기 위해 별도의 데이터 수집 PC를 설치하여 OPC UA 통신 가능하도록 인터페이스를 개발하고, 이를 기반으로 설비의 이상 상태를 예측하기 위한 기계학습 데이터셋을 구성한다.



3) 데이터 분석 기대효과

- 본 분석을 통해 공정 데이터와 설비 상태, 그리고 불량 제품의 상관관계를 도출하여 불량 발생 원인을 추적하고, 설비 이상을 예측할 수 있는 분석 데이터셋을 구축한다. 공정 데이터를 기반으로 설비 이상 상태를 예측이 필요한 제조공정 분야에 본 분석을 적용할 수 있을 것으로 기대된다.
- 특히, 본 분석에서는 설비 이상 상태를 예측할 뿐만 아니라 제품의 품질관리에도 영향을 미치므로 품질검사/관리가 필요한 엔지니어들도 본 분석결과를 활용 할 수 있다는 점에서 확장성이 좋다고 판단된다.

4) 시사점(implication) 요약기술

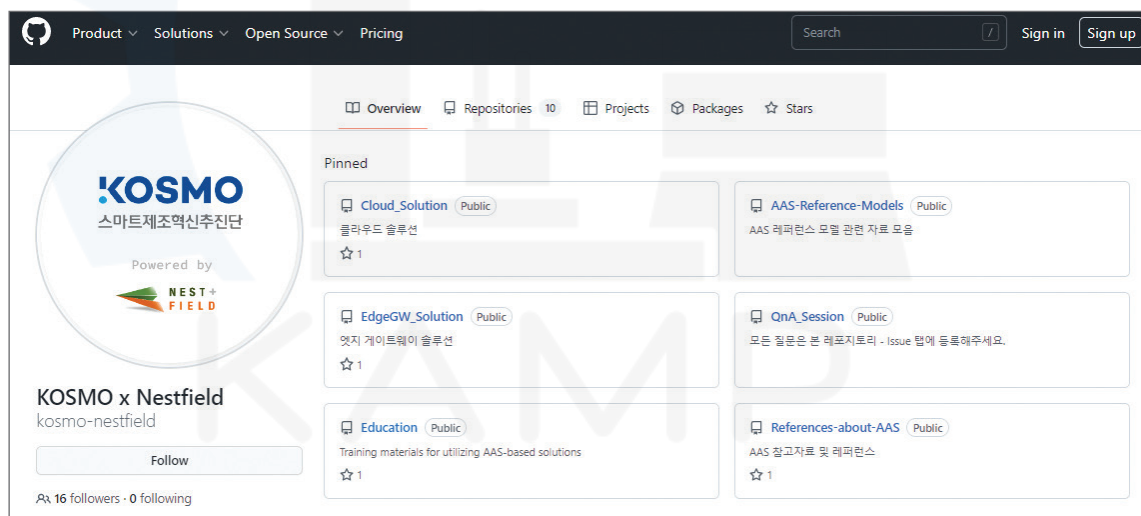
- 현재 E공장에서 사용중인 레이저 용접 설비는 배터리모듈을 용접하기 위해 사용되고 있으며, 설정된 파라미터에 따라 자동화되어 운용되고 있다. 그러나 설비가 정상적으로 동작하고 있음에도 불구하고 용접 불량 발생하는 경우가 종종 발생하여, 이를 해결하기 위해 용접 후 별도의 인력을 투입하여 불량률을 개선하고자 했으나 숙련된 작업자의 노하우에 의존해야 하는 상황이며, 불완전한 판단에 의해 불량 배터리모듈이 배터리팩에 조립되는 문제가 여전히 발생하고 있다. 이와 같은 문제를 해결하기 위해 작업자 노하우와 설비 전문가 그리고 공정 데이터를 간의 상관관계를 분석하고 정상/비정상 선별 레이블에 따른 데이터셋을 기반으로 AI 모델을 구축하여 발생 원인을 분석한다.
- 본 분석에서 용접공정의 정상/비정상 선별값 및 생산 조건 데이터를 수집하고 가공/전처리 후 학습을 통해 AI 모델을 개발하였다. 이렇게 현장에서 수집한 데이터를 활용하여 불량 발생 원인을 추적하고 이상 상태 발생 여부를 예측함으로써 효율적인 설비 관리뿐만 아니라 배터리팩 품질 향상에 기여했다는 점에서 시사점이 크다고 판단된다.

2. 분석실습

2.1 제조데이터 소개

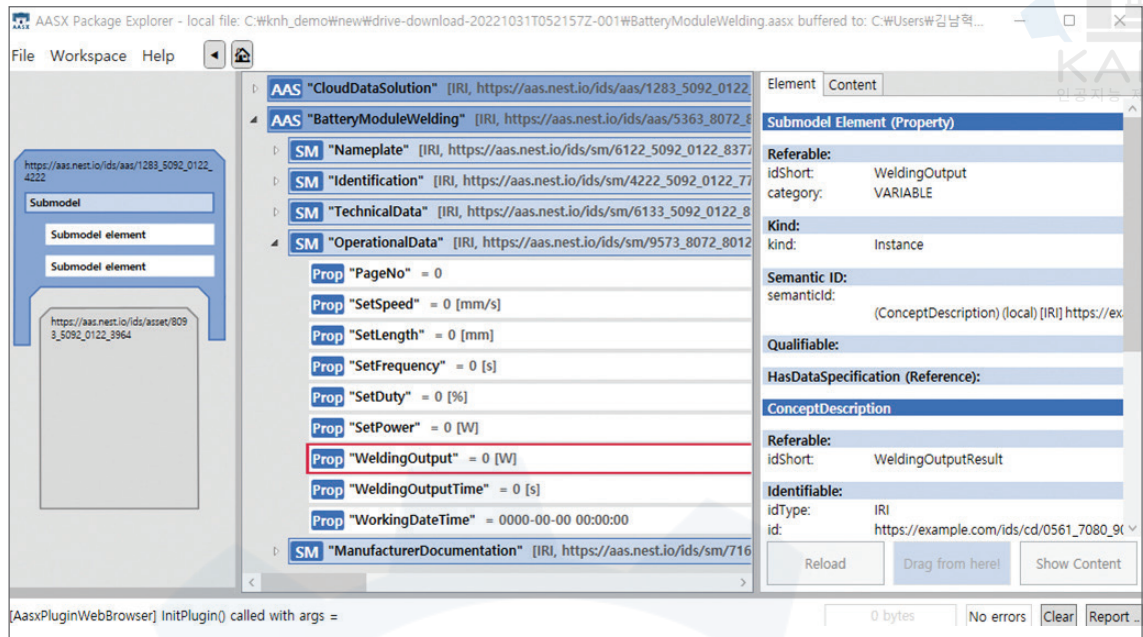
1) 데이터 수집 방법

- **제조(설비) 분야** : 전기차용 배터리팩 제조
- **제조 공정명** : 배터리팩 조립 공정
- **수집 장비** : AAS (Asset Administration Shell) 서버
 - AAS 표준기반 제조데이터 수집/저장 체계를 통해 AI 학습용 데이터 확보
- **수집 기간** : 2022.07.01 ~ 2022.09.30
- **수집 주기** : 약 3 sec
- **수집 방법** :
 - 본 분석에서는 데이터 수집을 위해 AAS 표준기반 제조데이터 수집/저장 체계가 준비된 상태를 가정한다. 단, 본 데이터 수집/저장 체계 구축을 위한 AAS 모델링 방법 및 소프트웨어 설치 가이드는 스마트제조혁신추진단과 네스트필드(주)에서 제공하는 오픈 소스 제공 사이트인 깃허브(<https://github.com/kosmo-nestfield>)에서 교육자료를 제공하고 있으므로 본 가이드북에서는 해당 내용을 다루지 않는다.



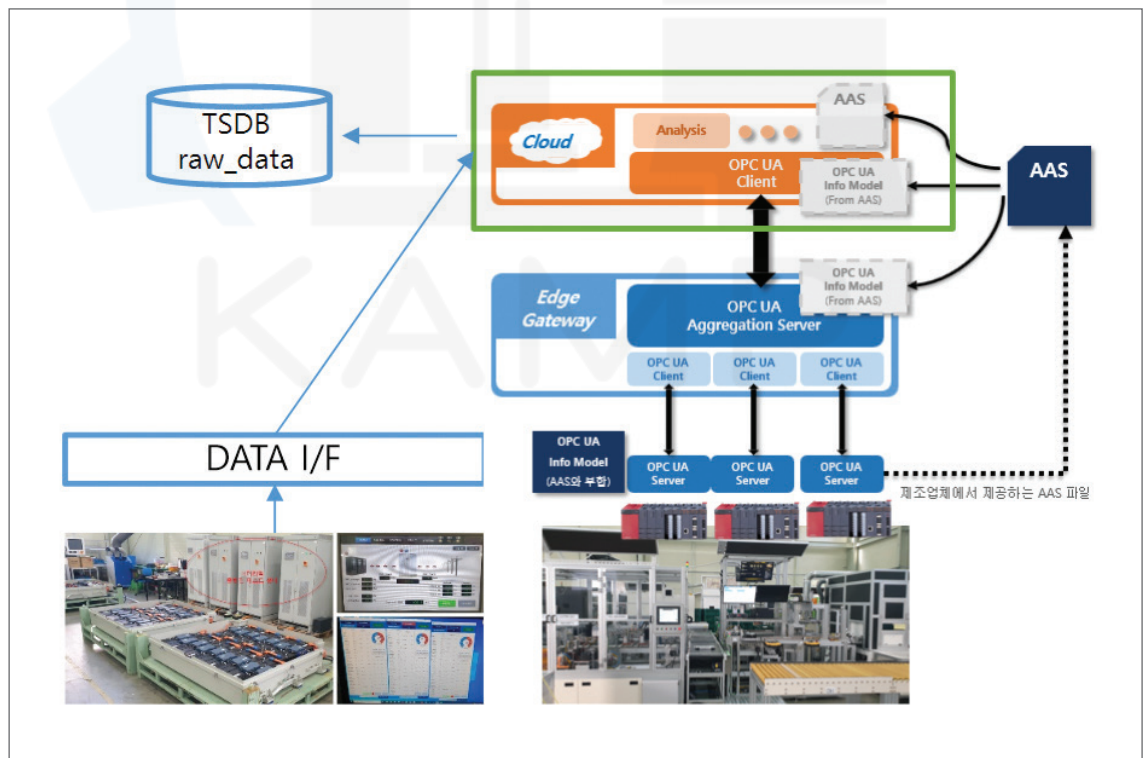
[그림 7] AAS 표준기반 제조데이터 수집/저장 체계 구축 가이드 배포 사이트

- 현장 설비공정에 대한 정보 모델은 IEC 63278-1 AAS (Asset Administration Shell) 기술을 통해 모델링하고 이를 기반으로 AAS 표준기반 제조데이터 수집/저장 체계에 사용될 수 있는 형태로 변환되어 사용된다. 본 분석에 사용된 배터리모듈 용접 설비공정 AAS 파일은 오픈 소스 소프트웨어인 'AASX Package Explorer'를 통해 뷰/편집이 가능하다. AASX Package Explorer 설치는 '부록(AAS S/W 설치 가이드)'를 참고한다.



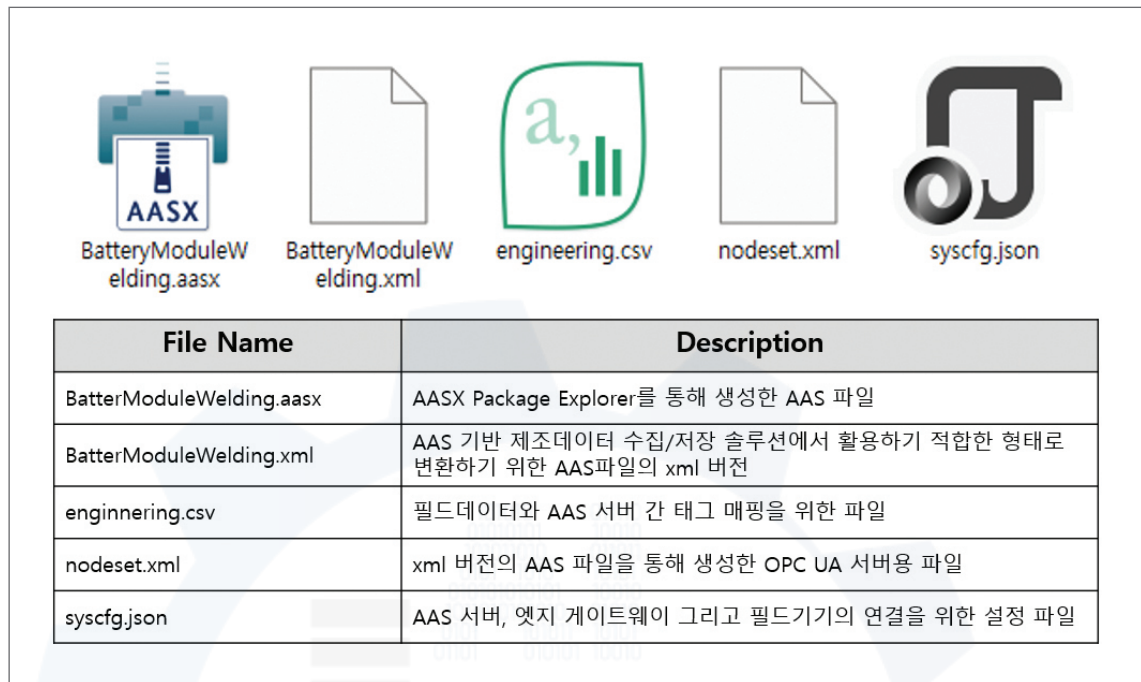
[그림 8] 배터리모듈 용접 설비공정 AAS 모델

- 현장 데이터는 설비공정에서 운용중인 PLC를 통해 엣지 게이트웨이로 수집하여 AAS 서버에서 TSDB에 저장하는 방법과 외부 서버 PC와 AAS 서버를 직접 연결하여 수집/저장하는 방법이 있다. 본 분석에서는 이러한 방식을 통해 RAW DATA를 수집하여 테스트/분석용 데이터를 확보하였다.



[그림 9] AAS 표준기반 제조데이터 수집/저장 체계 구성도

- 현장에서 운용중인 설비공정을 제어하기 위해 사용되는 PLC 또는 자체 운용서버 (OPC UA 통신 지원)를 통해 수집하는 데이터는 AAS 파일 변환을 통해 생성되는 파일 중 하나인 'engineering.csv'에 태그형태로 매핑되어야 한다.



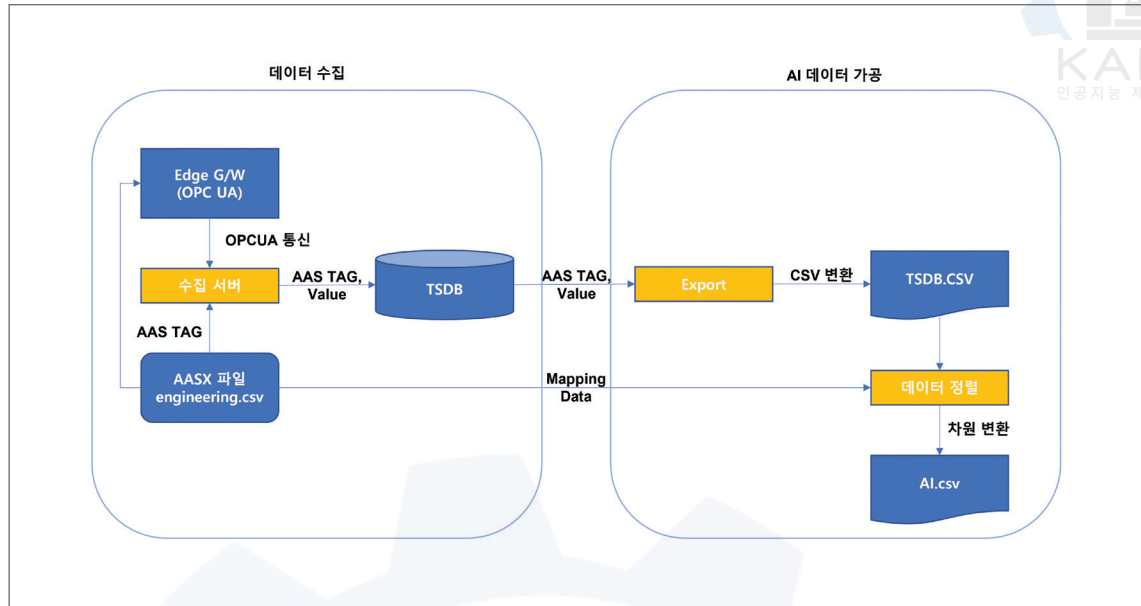
[그림 10] AAS 표준기반 제조데이터 수집/저장 체계 적용을 위한 AAS 파일 및 변환 파일

- engineering.csv 파일에서 A열은 AAS 모델을 통해 생성된 태그명, B열은 엣지 게이트웨이명, C열은 현장 데이터를 수집하는 필드장비명 D열은 현장에서 사용하는 OPC UA 태그명, E열은 ms 단위의 Sampling Interval, F열은 배열 정보 그리고 G열은 실제 필드에서 사용 중인 변수명이 들어간다. 단, 아래 그림과 같이 외부 공개하기 어려운 부분은 공백으로 표기하였다.

	A	B	C	D	E	F	G
37	ns=2;s=BatteryModuleWelding.Identification.ContactInfo01.Phone				100	-1	
38	ns=2;s=BatteryModuleWelding.Identification.ContactInfo01.Fax				100	-1	
39	ns=2;s=BatteryModuleWelding.Identification.URL				100	-1	
40	ns=2;s=BatteryModuleWelding.TechnicalData.GeneralInformation.ManufacturerName				100	-1	
41	ns=2;s=BatteryModuleWelding.TechnicalData.GeneralInformation.ManufacturerProductDesign				100	-1	
42	ns=2;s=BatteryModuleWelding.TechnicalData.GeneralInformation.ManufacturerPartNumber				100	-1	
43	ns=2;s=BatteryModuleWelding.TechnicalData.GeneralInformation.ManufacturerOrderCode				100	-1	
44	ns=2;s=BatteryModuleWelding.TechnicalData.ProductClassifications.ProductClassificationItem1				100	-1	
45	ns=2;s=BatteryModuleWelding.TechnicalData.ProductClassifications.ProductClassificationItem1				100	-1	
46	ns=2;s=BatteryModuleWelding.TechnicalData.ProductClassifications.ProductClassificationItem1				100	-1	
47	ns=2;s=BatteryModuleWelding.TechnicalData.TechnicalProperties.SemanticIdNotAvailable01				100	-1	
48	ns=2;s=BatteryModuleWelding.TechnicalData.TechnicalProperties.MainSection01.SemanticIdNotAvailable01				100	-1	
49	ns=2;s=BatteryModuleWelding.TechnicalData.TechnicalProperties.MainSection01.SubSection01				100	-1	
50	ns=2;s=BatteryModuleWelding.TechnicalData.FurtherInformation.TextStatement01				100	-1	
51	ns=2;s=BatteryModuleWelding.TechnicalData.FurtherInformation.ValidDate				100	-1	
52	ns=2;s=BatteryModuleWelding.OperationalData.PageNo				100	-1	PageNo
53	ns=2;s=BatteryModuleWelding.OperationalData.SetSpeed				100	-1	Speed
54	ns=2;s=BatteryModuleWelding.OperationalData.SetLength				100	-1	Length
55	ns=2;s=BatteryModuleWelding.OperationalData.SetFrequency				100	-1	SetFrequency
56	ns=2;s=BatteryModuleWelding.OperationalData.SetDuty				100	-1	SetDuty
57	ns=2;s=BatteryModuleWelding.OperationalData.SetPower				100	-1	SetPower
58	ns=2;s=BatteryModuleWelding.OperationalData.WeldingOutput				100	-1	RealPower
59	ns=2;s=BatteryModuleWelding.OperationalData.WeldingOutputTime				100	-1	GateOnTime
60	ns=2;s=BatteryModuleWelding.OperationalData.WorkingDateTime				100	-1	WorkingTime
61							

[그림 11] AAS 변환 파일 중 engineering.csv 파일 구성

- 실제 설비공정에서 발생하는 데이터들은 의미 없는 값이나 누락이 발생할 수 있어 데이터 품질이 떨어지며, 이는 AAS 표준기반 제조데이터 수집/저장 체계에서도 영향을 받기 때문에 수집된 데이터만으로는 AI 분석에 적합하지 않다. 따라서, 데이터 가공 및 품질 전처리하는 데이터 분석에서 필수적인 단계이다.
- 본 분석에서 AAS 표준기반 제조데이터 수집/저장 체계는 작성한 AASX 파일을 기반으로 Edge Gateway에서 OPC UA 통신을 통해 수집 서버(AAS 서버)로 데이터를 전송한다. 수집 서버는 수신한 데이터에 대하여 AAS TAG에 맞추어 값을 TSDB에 저장하게 된다.



[그림 12] 데이터 수집 및 가공 구성도

- 수집된 데이터는 아래와 같이 명령어를 사용하여 TSDB로부터 CSV 파일을 추출한다.

```
curl --request -G http://localhost:8086/query?orgID=INFLUX_ORG_ID&database=MyDB&retention_policy=MyRP \
  --header 'Authorization: Token INFLUX_TOKEN' \
  --header 'Accept: application/csv' \
  --header 'Content-type: application/json' \
  --data-urlencode "q=SELECT used_percent FROM example-db.example-rp.example-measurement WHERE host=host1"
```

[그림 13] 수집 데이터에 대한 CSV 파일 추출 예시

```
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-10-27T02:37:46.8472,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-10-27T02:37:51.1234982,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-02T06:06:29.8367752,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-02T18:37:07.2125452,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-03T07:28:00.5011492,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-03T09:56:06.569632,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-10T01:08:22.2183082,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-10T04:57:51.7041242,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-19T07:22:11.0771632,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
result,0,1970-01-01T00:00:00Z,2022-09-26T08:58:15.449872267Z,2021-11-19T07:35:03.4622442,0,ns=2,s=BT_Pack_Assembly_System.Component.CCH_Unloader.OperationalData.PLC_IO_19.Fall.mos.device
```

[그림 14] 추출한 데이터 예시

- 추출한 CSV 파일을 engineering.csv 파일 내 AAS 태그와 매핑된 변수명들을 매칭하여 데이터를 정렬한다. 이렇게 데이터가 모두 정렬되면 아래와 같이 데이터셋 생성을 위한 Raw 데이터를 획득할 수 있다.

PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime
1	250	241.1	1662	1000	100	82	1154	2022-02-10 09:02:48.515
2	250	241.2	1688	1000	100	83	1670	2022-02-10 09:02:51.894
3	250	241.1	1669	1000	100	82	1153	2022-02-10 09:02:54.740
4	250	241.2	1693	1000	100	83	1670	2022-02-10 09:02:57.948
5	250	241.1	1673	1000	100	82	1154	2022-02-10 09:03:00.734
6	250	241.2	1697	1000	100	83	1670	2022-02-10 09:03:03.988

[그림 15] 데이터셋 생성을 위한 Raw 데이터 획득



2) 데이터 유형/구조

A	B	C	D	E	F	G	H	I
PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime
1	250	241.1	1688	1000	100	82	1154	2022-01-08 10:11:45.181
2	250	241.2	1713	1000	100	83	1670	2022-01-08 10:11:48.487
3	250	241.1	1695	1000	100	82	1153	2022-01-08 10:11:51.245
4	250	241.2	1717	1000	100	83	1670	2022-01-08 10:11:54.493
5	250	241.1	1698	1000	100	82	1154	2022-01-08 10:11:57.269
6	250	241.2	1719	1000	100	83	1670	2022-01-08 10:12:00.659
7	250	241.1	1697	1000	100	82	1153	2022-01-08 10:12:06.660
8	250	241.2	1719	1000	100	83	1670	2022-01-08 10:12:09.910
9	250	241.1	1699	1000	100	82	1154	2022-01-08 10:12:12.674
10	250	241.2	1722	1000	100	83	1670	2022-01-08 10:12:15.900
11	250	241.1	1700	1000	100	82	1154	2022-01-08 10:12:18.684
12	250	241.2	1723	1000	100	83	1670	2022-01-08 10:12:22.008
13	30	19.4	692	1000	100	38	650	2022-01-08 10:12:26.774
14	30	19.4	692	1000	100	38	650	2022-01-08 10:12:28.506
15	30	19.4	692	1000	100	38	650	2022-01-08 10:12:31.410
16	30	19.4	692	1000	100	38	650	2022-01-08 10:12:34.702
17	30	19.4	692	1000	100	38	650	2022-01-08 10:12:36.378
18	30	19.4	691	1000	100	38	650	2022-01-08 10:12:38.084
19	250	241.1	1714	1000	100	83	1670	2022-01-08 10:12:59.328
20	250	241.2	1698	1000	100	82	1154	2022-01-08 10:13:02.038
21	250	241.1	1721	1000	100	83	1670	2022-01-08 10:13:05.500

[그림 16] Training_Data.csv 예시

• 데이터셋 구조

- 본 분석에서 배터리 모듈 용접 설비로부터 수집되는 총 9개의 변수들로 구성되어 있다. 이 중 분석 실습에서 사용하지 않는 변수를 제외하고, 7개 변수를 분석 실습에 사용한다.

• 데이터 개수 : 총 1,268,865개

- 학습용(Training) 데이터 : 1,221,831개
- 테스트용(Test) 데이터 : 47,034개



• 제조AI데이터셋 주요 변수 정의

변수 유형	변수명	설명
용접시퀀스	PageNo	용접 작업 시퀀스 정보 (Count)
용접속도설정(mm/s)	Speed	설정된 길이를 기준으로 모터 속도를 설정
용접길이설정(mm)	Length	용접할 부분의 길이를 설정
용접출력(W)	RealPower	용접 포인트 별로 측정된 실제 용접 출력
발광횟수설정(Hz)	SetFrequency	초당 발광 횟수 설정
최대용접출력설정(%)	SetDuty	최대 용접 출력 설정
용접출력설정(%)	SetPower	용접 재질에 따라 변하는 용접 출력 설정
용접시간(s)	GateOnTime	용접 게이트 오픈 시간
작업시간	WorkingTime	작업이 진행된 시간 (HH:MM:DD hh:mm:ss)

• 독립변수 / 종속변수

- 독립변수란, 다른 변수에 영향을 받지 않는 변수로, 입력값이나 원인을 나타낸다.
- 종속변수란, 독립변수의 변화에 따라 어떻게 변화하는지를 알고 싶은 변수로, 결과물이나 효과를 나타낸다.

구분	명칭
독립변수	PageNo
	Speed
	Length
	RealPower
	SetFrequency
	SetDuty
	SetPower
	GateOnTime
	WorkingTime
종속변수	없음



• 주요 변수 기술 통계

구분	개수	평균	표준편차	최소값	중앙값	최대값
PageNo	135759	20	11.25467	1	20	39
Speed	135759	165.0138	106.9779	30	250	250
Length	135759	155.5348	107.8064	19.4	241.1	241.2
RealPower	135759	1310.511	493.2247	0	1688	1733
SetFrequency	135759	1000	0	1000	1000	1000
SetDuty	135759	100	0	100	100	100
SetPower	135759	65.38462	21.65307	38	82	83
GateOnTime	135759	1117.99	421.9566	650	1154	1670

- 본 데이터셋에 포함된 연속형 수치 데이터에 대하여 기초적인 기술 통계를 확인한다.

3) 데이터 품질 전처리

• 데이터 품질 전처리 목적

- 원시 데이터 (raw data)의 수집은 다양한 센서를 사용하여 데이터를 수집하고 저장함으로써 이루어지며, 실제 현장의 데이터는 의미 없는 값을 포함하거나 결측치, 혹은 이상 데이터가 발생하여 데이터 품질이 떨어진다.
- 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없으므로 데이터의 전처리 (data pre-processing) 과정을 수행한다.
- 데이터의 품질을 측정하는 기준으로 다음 6가지 항목에 대하여 측정하여 데이터 품질 지수를 산출하고, 데이터의 전처리를 통하여 품질을 개선한다.



항목	항목 설명
완전성 (Completeness)	데이터의 필수 항목에 결측치가 없어야 함을 의미한다.
유일성 (Uniqueness)	데이터 엔티티 (data entity)가 관련 애플리케이션 내에서 고유하게 캡처, 표시 및 참조되어야 함을 나타낸다. 즉, 논리적으로 중복되는 엔티티가 없으며 데이터 세트 내의 각 엔티티를 고유하게 액세스하는 데 사용할 수 있는 고유 식별자가 있어야 함을 의미한다.
유효성 (Validity)	데이터 항목은 정해진 데이터의 유효범위 안에 있어야 한다.
일관성 (Consistency)	데이터의 구조, 유형 및 값은 형태가 일관되게 정의되고, 서로 일치해야 한다. 즉, 동일한 데이터 엔티티에 대하여 한 데이터셋의 데이터 값이 다른 데이터 세트의 값과 일관성이 있어야 함을 의미한다.
정확성 (Accuracy)	데이터가 모델링하려는 실제 개체를 정확하게 표현하는 정도를 나타낸다. 즉, 데이터 항목이 검증된 정확한 정보 (예: 참조 데이터)와 얼마나 일치하는지에 따라 측정된다. 정확성을 검증하기 위해서는 확증을 위한 참조 데이터 (다른 테이블의 유사한 검증 데이터 값)나 상관성이 존재하는 기존의 테이블의 값과 비교한다.
무결성 (Integrity)	데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 유효성, 일관성이 보호되어야 함을 의미한다.

[표 3] 데이터 품질지수

- 데이터 품질 지수에 관한 내용은 다음의 사이트를 참고하고, 구체적인 산출 방법은 4. 부록 [데이터 품질 전처리]를 참고한다.

◦ Data Quality Dimension 사이트:

<https://www.sciencedirect.com/topics/computer-science/data-quality-dimension>

• 데이터 품질 지수 세부 설명

(1) 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 결측 데이터의 값이 30% 이상인 레코드는 데이터의 완전성을 떨어뜨리기 때문에 각 열에 대한 결측 데이터값의 비율을 확인하여 삭제한다.
- ② 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다.
- ③ 위 식에 따라 완전성 품질지수를 계산한다.
- ④ 데이터의 완전성 (Completeness) 예시: 아래 테이블에서 RealPower 칼럼에 null 값이 존재하므로 완전성에 위배된다.

	A	B	C	D	E	F	G	H	I	J	K
1	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTi	WorkingTime	Disp1	Disp2
2	1	250	241.1	1688	1000	100	82	1154	2022-01-08 10:11:45.181	0	0
3	2	250	241.2	1713	1000	100	83	1670	2022-01-08 10:11:48.487	0	0
4	3	250	241.1		1000	100	82	1153	2022-01-08 10:11:51.245	0	0
5	4	250	241.2		1000	100	83	1670	2022-01-08 10:11:54.493	0	0
6	5	250	241.1		1000	100	82	1154	2022-01-08 10:11:57.269	0	0
7	6	250	241.2	1719	1000	100	83	1670	2022-01-08 10:12:00.659	0	0
8	7	250	241.1	1697	1000	100	82	1153	2022-01-08 10:12:06.660	0	0
9	8	250	241.2	1719	1000	100	83	1670	2022-01-08 10:12:09.910	0	0

(2) 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터에서 유일한 값을 갖는 열을 찾는다. 시계열 데이터는 주로 Timestamp이다.
- ② 유일한 값을 갖는 열에 대해 유일한 데이터 값을 확인하고, 그 개수를 구한다.
- ③ 위 식에 따라 유일성 품질지수를 계산한다.
- ④ 데이터의 유일성 (Uniqueness) 예시: 아래 테이블의 WorkingTime에서 각각 다른 시간대에 데이터가 기록되어야 하는데, 같은 시간대가 중복해서 기록되었으므로 유일성에 위배된다.

	A	B	C	D	E	F	G	H	I	J	K
1	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTi	WorkingTime	Disp1	Disp2
2	22	250	241.2	1700	1000	100	82	1154	2022-01-08 10:13:08.128	347.28	346.94
3	23	250	241.1	1722	1000	100	83	1670	2022-01-08 10:13:11.628	347.28	346.94
4	24	250	241.2	1702	1000	100	82	1154	2022-01-08 10:13:14.262	347.28	346.94
5	25	250	241.1	1721	1000	100	83	1670	2022-01-08 10:13:14.262	347.28	346.94
6	26	250	241.2	1702	1000	100	82	1154	2022-01-08 10:13:23.494	347.28	346.94
7	27	250	241.1	1724	1000	100	83	1670	2022-01-08 10:13:26.986	347.28	346.94
8	28	250	241.2	1703	1000	100	82	1154	2022-01-08 10:13:29.598	347.28	346.94
9	29	250	241.1	1726	1000	100	83	1670	2022-01-08 10:13:33.116	347.28	346.94

(3) 유효성 품질지수

$$\text{유효성 품질지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터가 주어진 유효범위를 벗어나는지를 확인한다.
- ② 전체 데이터의 개수와 유효범위 안에 있는 데이터의 개수를 구한다.
- ③ 위 식에 따라 유효성 품질지수를 계산한다.
- ④ 데이터의 유효성(Validity) 예시: 아래 테이블에서 SetPower에 유효하지 않은 범위의 값이 존재하므로 유효성에 위배된다.

	A	B	C	D	E	F	G	H	I	J	K
1	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTi	WorkingTime	Disp1	Disp2
2	31	30	19.4	692	1000	100	38	650	2022-01-08 10:13:40.146	347.28	346.94
3	32	30	19.4	691	1000	100	38	650	2022-01-08 10:13:41.676	347.28	346.94
4	33	30	19.4	692	1000	100	-50	650	2022-01-08 10:13:44.756	347.28	346.94
5	34	30	19.4	692	1000	100	-50	650	2022-01-08 10:13:46.496	347.28	346.94
6	35	30	19.4	692	1000	100	-50	650	2022-01-08 10:13:49.988	347.28	346.94
7	36	30	19.4	692	1000	100	38	650	2022-01-08 10:13:51.654	347.28	346.94
8	37	30	19.4	692	1000	100	38	650	2022-01-08 10:13:53.390	347.28	346.94
9	38	30	19.4	692	1000	100	38	650	2022-01-08 10:13:56.410	347.28	346.94

(4) 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 테이블 간 종속관계를 확인한다.
- ② 데이터의 유형이 정해진 형식에 맞는지 확인하고, 전체 데이터의 개수와 형식에 맞는 데이터의 개수를 구한다.
- ③ 위 식에 따라 일관성 품질지수를 계산한다.

(5) 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 항목에 대하여 검증된 정확한 정보 (참조 데이터)가 있는지를 확인한다.
- ② 참조 데이터가 있는 경우, 데이터 항목 값과 참조 데이터의 값을 비교한다.
- ③ 이 차이가 0이 아니면 정확성을 위배하는 데이터가 존재함을 의미한다.

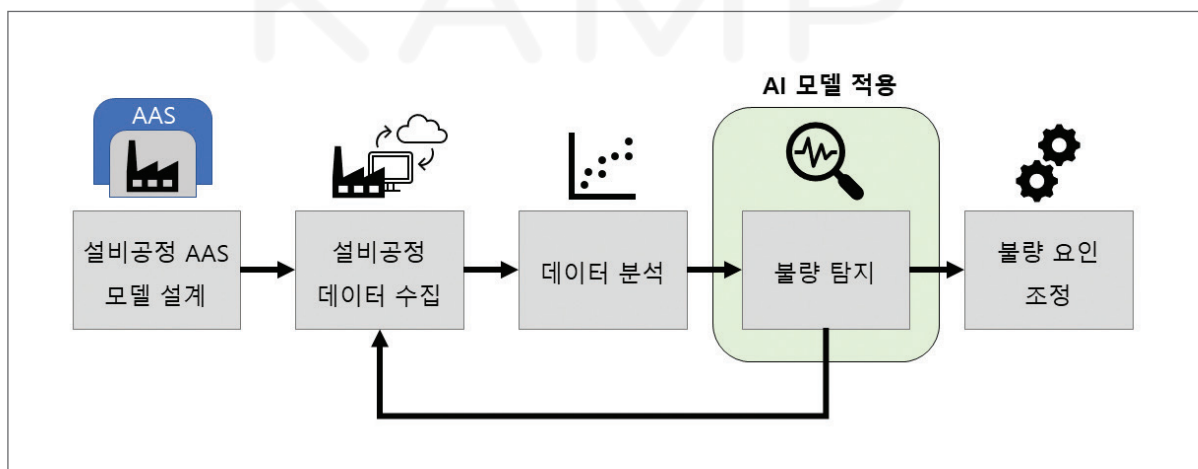
(6) 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100\%가 아닌 지수}}{3} \right) \right\} \times 100$$

무결성 품질지수는 앞서 구한 ‘유일성’, ‘유효성’, ‘일관성’ 지수를 기초로 위의 식으로 계산된다. 이 지수들이 100%이면 무결성 품질지수도 100%가 된다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 17] 데이터 흐름 및 인공지능 모델 적용 흐름도

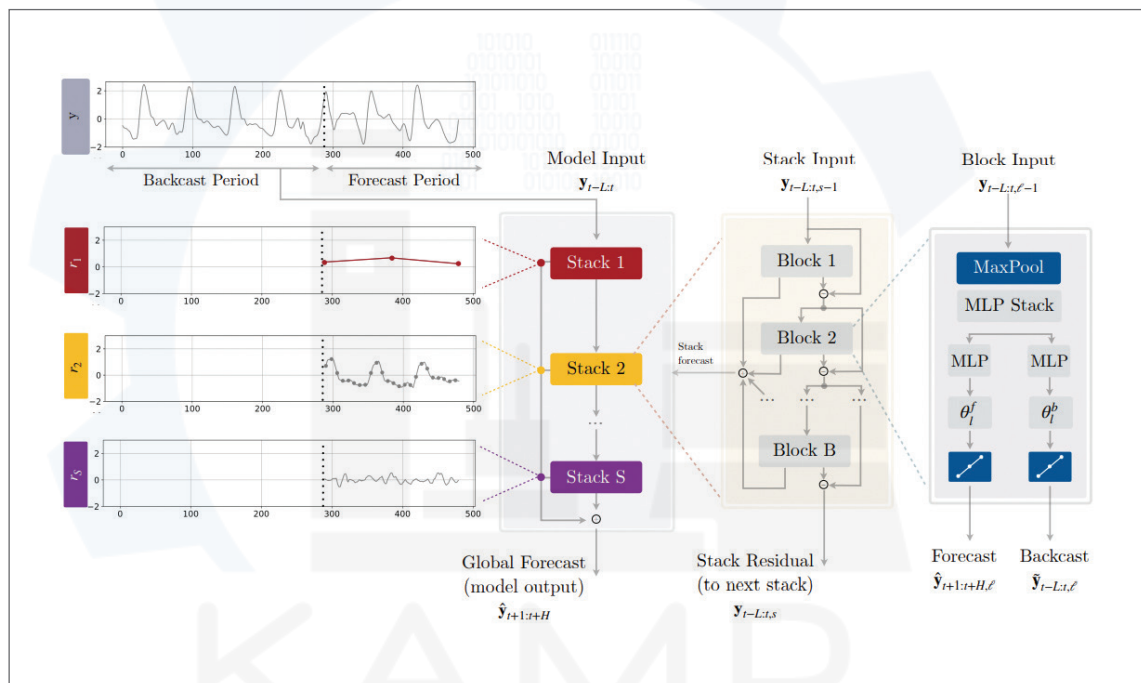
2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유

- 데이터분석을 위하여 최근에 나온 시계열 예측 알고리즘인 N-HiTS 알고리즘(C. Challu, et al, 2022)을 택한다. N-HiTS 알고리즘은 벤치마크 테스트에서 높은 장시간 예측 정확도를 보이며 메모리 사용이 효율적이며 계산속도도 다른 방법에 비해 빠르다는 점이 이 알고리즘을 택하게 된 이유이다. 또한 테스트 데이터의 성격에 따라서는 학습 데이터의 통계분포에서 구한 표준편차 값으로부터 테스트 데이터 값의 Z-score를 계산하여 이로부터 이상값을 탐지하는 방법도 사용한다.

• 적용하고자하는 AI 분석 방법론(알고리즘)의 구체적 소개

- N-HiTS 알고리즘은 아래 그림과 같이 구성되어 있다.

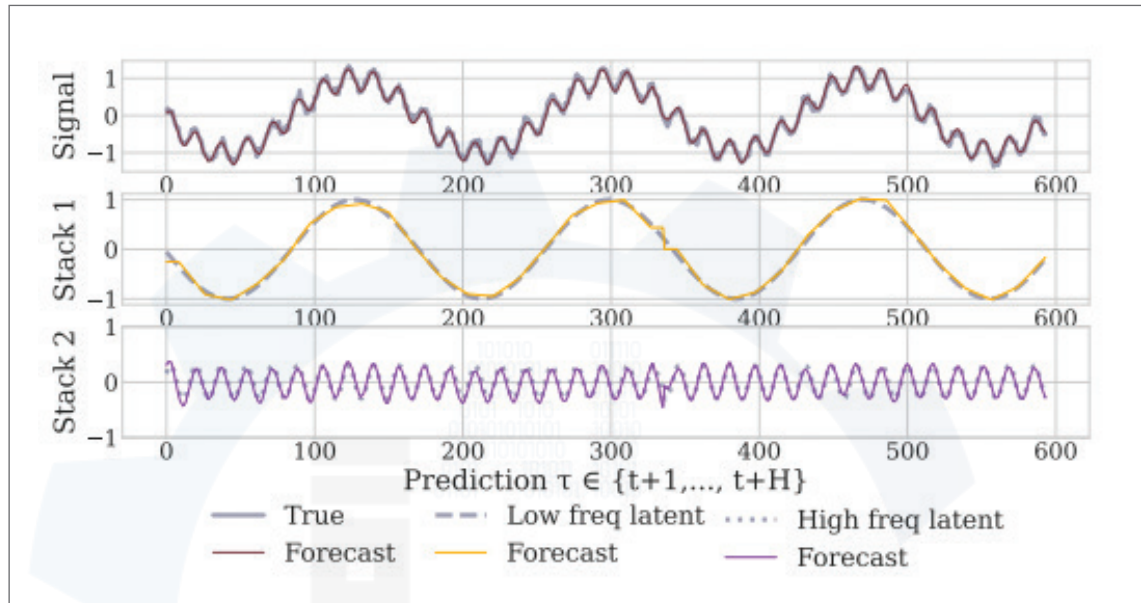


[그림 18] N-HiTS 구성도

※ 내부의 기계학습 모형은 ReLU를 포함하는 완전-연결 구조의 다층신경망(MLP)으로 구성된다. 보통 세 개의 스택으로 이루어져서 각 스택은 긴 시간 스케일과 중간 시간 스케일 그리고 짧은 시간 스케일의 변화를 담당하여 예측하고 이를 결합함으로써 전체적인 예측을 수행한다.

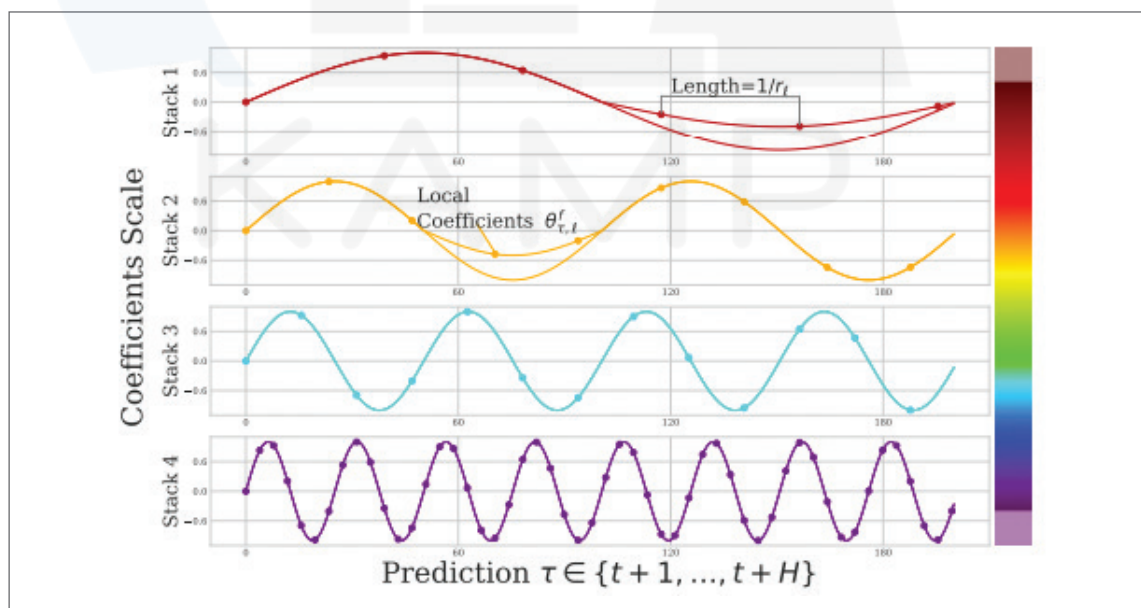
- N-HiTS 알고리즘은 위 구성도에서 보듯이 여러 스택으로 구성되어 있다. 각 스택은 완전-연결 다층 신경망블록에 앞서 시계열 데이터를 각각 다른 비율로 부분 샘플링하는 층을 두어 이를 다층신경망에 입력한다. 이를 다중 비율 데이터 샘플링(Multi-Rate Data Sampling)이라고 한다. 이 방법은 메모리 사용량과 계산량을 크게 줄이게 되어 장시간 데이터 의존성에 대한 모형 구성을 가능하게 한다.

- 부분 샘플링에 의한 다중-스텝(multi-step) 부분적 예측 결과로부터 전체 예측 구간에 맞추기 위해서는 비어 있는 데이터를 메꾸어 넣는 과정을 거치는데 이를 위해 다중 스케일 계층적-내삽 방법을 이용한다. 이를 multi-scale hierarchical interpolation라고 한다. 이와 같이 각 스택의 블록이 특정한 주파수 밴드의 시계열 데이터 거동을 예측하면서 동시에 이들 다른 블록들을 전체적으로 동기화함으로써 시계열 예측을 가능하게 한다. 이 원리는 아래 두 그림에 나타나 있다.



[그림 19] N-HITS 알고리즘 시계열 예측 원리1

- ※ 계층적 내삽과 다중-비율 데이터 입력을 결합하여 시계열 데이터의 특정한 주파수 대역에 집중하여 예측하고 이를 결합함으로써 효율적인 예측을 수행할 수 있다.



[그림 20] N-HITS 알고리즘 시계열 예측 원리2

- ※ N-HITS 알고리즘은 다른 주파수 대역의 데이터를 예측하는 블록들의 계층적인 구조로 구성되어 시계열 데이터를 예측한다.



- 통계분포 Z-score 알고리즘:

학습 데이터의 비정상 데이터 값이 연속적으로 긴 시점 구간에서 나타날 경우에는 N-HiTS 알고리즘이 적합하지 않을 수 있다. 이때는 먼저 학습 데이터에 대한 통계분석을 수행하여 학습 데이터의 분포를 몇 개의 가우스 분포의 중첩으로 가정하여 각각의 가우스 분포 모드에 대한 평균값과 표준편차를 구하고 테스트 데이터에 대하여는 각 시점의 테스트 데이터 값에 대하여 각각의 가우스 모드의 평균값으로부터의 거리를 해당 모드의 표준편차로 나눈 값 즉, Z-score를 계산할 수 있다. 다음으로 이들 Z-score 중의 최소값을 구하여 이 값이 정해진 threshold를 넘으면 이 데이터 값을 anomaly로 예측할 수 있다. 이 방법은 학습 데이터로부터 가우스 모드의 개수와 대략적인 범위를 추정할 수 있을 때에 적용 가능하다.

• AI 분석 방법론(알고리즘) 구축 절차

- N-HiTS 알고리즘은 Python과 PyTorch로 구축하는데 다음 성분들로 구성된다.

- * 다중-비율 시계열 데이터 샘플링 (Multi-Rate Signal Sampling)
- * 다층 신경망을 이용한 비선형 회귀분석 (Nonlinear Regression)
- * 계층적 내삽 (Hierarchical Interpolation)

- 모형 학습은 MAE (Maximum Accuracy Error) 손실함수에 대하여 최적화 함수로 Adam Optimizer를 이용한다.

2.3 분석 체험

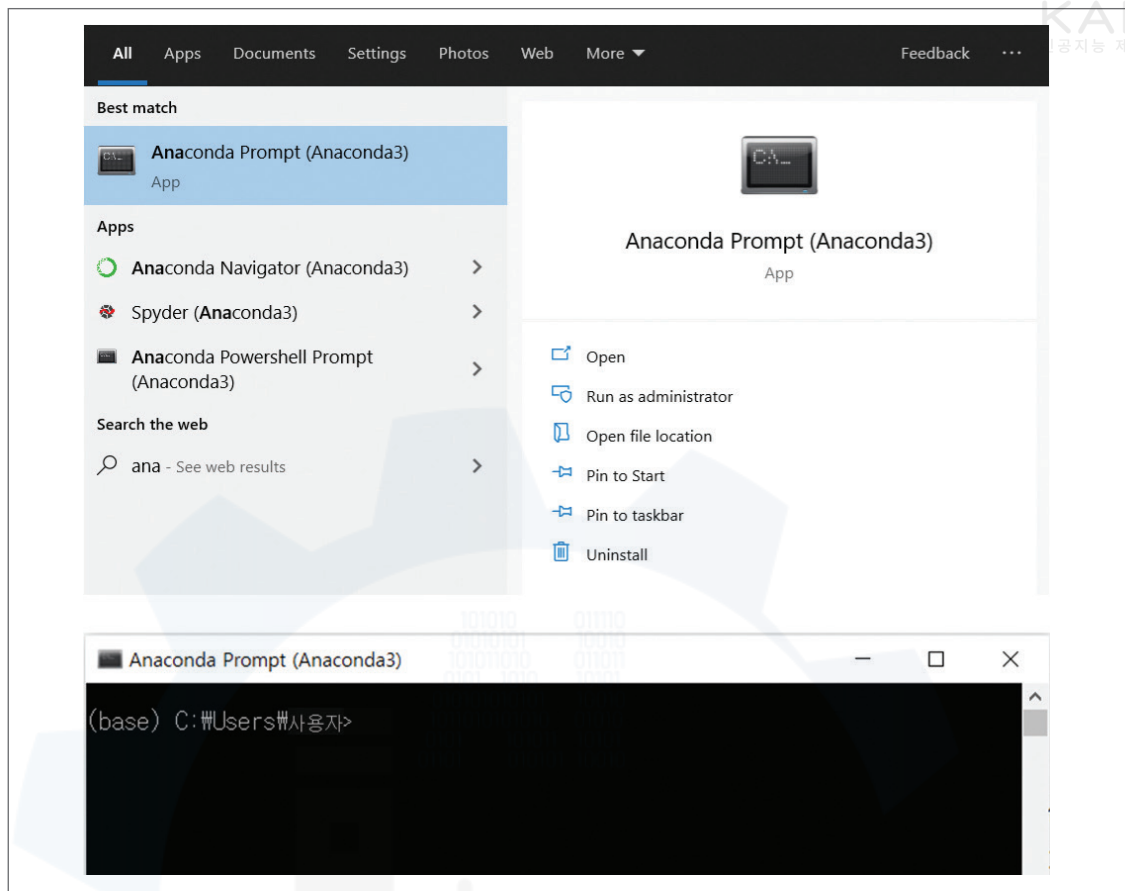
1) 필요 SW, 패키지 설치 방법 및 절차 가이드

▶ SW 설치는 '부록 분석환경 구축을 위한 설치 가이드' 참고

▶ 가상환경 구축 가이드

- 부록 3. [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주 피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

① 아나콘다 프롬프트 실행



② 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

```
conda create -n KAMP python=3.8.8 jupyter
```

```
(base) C:\Users\User>conda create -n KAMP python=3.8.8 jupyter
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.12.0
  latest version: 22.11.0
```

...

```
Proceed ([y]/n)? y
```

해당 문구가 뜨면 'y' 를 입력한다.

...

```
done
#
# To activate this environment, use
#
#     $ conda activate KAMP
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\사용자>
```

③ 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시된다.

```
# conda activate 가상환경이름
conda activate KAMP
```

```
(base) C:\Users\사용자>conda activate KAMP
(KAMP) C:\Users\사용자>
```

④ 분석이 끝난 후에는 가상 환경을 종료한다.

```
conda deactivate
```

```
(KAMP) C:\Users\사용자>conda deactivate
(base) C:\Users\사용자>
```

⑤ 주피터 노트북에서 가상 환경 실행

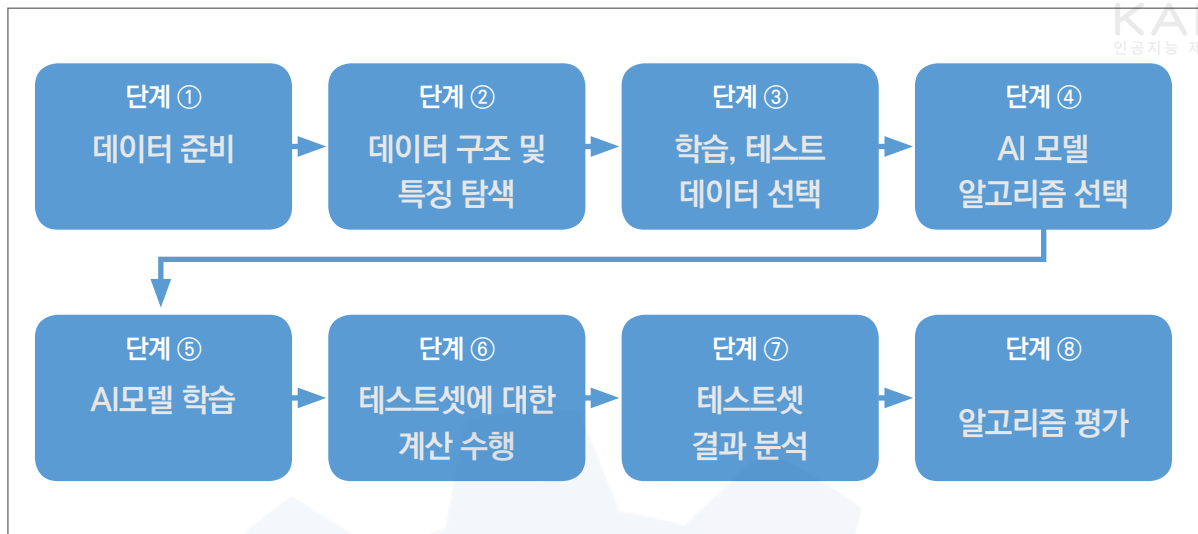
주피터 노트북에서 가상 환경을 실행하기 위해 가상 환경 활성화 상태에서 아래의 코드를 작성한다. 주피터 노트북 실행창에서 빨간 박스처럼 생성한 가상 환경이 있는 것을 확인할 수 있다. 해당 가상 환경을 선택 후 실습을 진행한다.

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
python -m ipykernel install --user --name KAMP --display-name "KAMP"

# jupyter notebook 실행
jupyter notebook
```



2) 분석 단계별 프로세스 - Flow Chart



[그림 19] 분석 단계별 프로세스

[단계 ①] 데이터 준비

①-1. 패키지 설치

- 현재 설치된 패키지 목록을 보기 위해서는 “!pip list”를 실행한다.

```
!pip list
```

Package	Version
absl-py	1.0.0
alabaster	0.7.12
anaconda-client	1.7.2
anaconda-navigator	2.0.3
anaconda-project	0.9.1
anyio	2.2.0
appdirs	1.4.4
argh	0.26.2
argon2-cffi	20.1.0
asn1crypto	1.4.0
astroid	2.5
astropy	4.2.1

[코드 1] 패키지 목록 확인

- 주피터 노트북 내에서 추가적으로 필요한 패키지 및 모듈은 “!pip install” 명령어를 사용하여 설치한다. 예를 들어 Python에서 시각화 도구로 많이 사용되는 matplotlib, 데이터 작업을 쉽고 직관적으로 할 수 있도록 해주는 pandas, 숫자 연산을 위한 Python 라이브러리인 numpy, 시각화를 위한 plotly, 데이터를 모델이 학습할 수 있는 형태로 변형하기 위한 pytorch-forecasting을 설치하기 위해서는 다음 명령문을 실행한다.



```
!pip install pytorch_forecasting==0.10.3
!pip install pytorch-forecasting[mqf2]==0.10.3
!pip install --force-reinstall torch==1.12.1+cu116 torchvision==0.13.1+cu116 --extra-index-url https://download.pytorch.org/whl/cu116
!pip install pytorch_lightning==1.7.7
!pip install matplotlib==3.6.2
!pip install pandas==1.5.2
!pip install numpy==1.23.5
!pip install plotly==5.10.0
```

[코드 2] 패키지 설치

①-2. 라이브러리/패키지 불러오기

- 작업에 필요한 패키지 및 모듈을 import 하여 사용한다. 이때 별명(alias)을 주어 import 할 수 있으며, 친숙한 별명을 사용하면 편리하게 관련 패키지 및 모듈을 사용할 수 있다. numpy를 np, pandas를 pd 라는 별명으로 사용하고 싶은 경우, 다음 명령어를 사용한다. 또한, 라이브러리로부터 특정한 모듈만 import할 경우에는 다음과 같이 from~import 구문을 사용한다.

```
# "as name" indicates the name we will use to refer to this library

import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# we will only import certain module from those libraries

from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.covariance import EllipticEnvelope
from sklearn.ensemble import IsolationForest
from random import randrange
from datetime import datetime
import math
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots
from pytorch_forecasting.data.examples import generate_ar_data
warnings.simplefilter(action='ignore', category=FutureWarning)
pd.set_option('mode.chained_assignment', None)
```

[코드 3] 라이브러리 및 패키지 호출

- file_path에 분석 파일인 “Training_Data.csv”를 지정하고, pandas 라이브러리를 사용하여 csv 파일을 불러오며, Pandas DataFrame 형식으로 데이터를 사용한다. Pandas DataFrame은 2차원 테이블 형식으로 되어 있어 데이터를 직관적으로 쉽게 처리할 수 있다.

```
file_path = './data/raw_data/train/Training_Data.csv'# the file path is the archive location of the file in our computer
data = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
# After reading the file, it will be used as a Pandas DataFrame.
# Pandas DataFrame is a special data structure from Pandas Library that is two-dimensional,
# size-mutable, potentially heterogeneous tabular data.
```

[코드 4] 파일 호출

①-3. 데이터 종류 및 개수 확인

- 데이터의 속성 (column) 이름과 각 속성의 처음 5개 행을 표시하기 위해서 head() 메소드를 사용하며, 이를 실행하면 다음의 내용이 표시된다.

```
# Showing the first five rows of our data
```

```
data.head()
```

	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1688	1000	100	82	1154	2022-01-08 10:11:45.181
1	2	250	241.2	1713	1000	100	83	1670	2022-01-08 10:11:48.487
2	3	250	241.1	1695	1000	100	82	1153	2022-01-08 10:11:51.245
3	4	250	241.2	1717	1000	100	83	1670	2022-01-08 10:11:54.493
4	5	250	241.1	1698	1000	100	82	1154	2022-01-08 10:11:57.269

[코드 5] 데이터의 처음 5개의 행 확인

- 데이터의 속성을 확인하기 위하여 다음 구문을 사용하며, 전체 9개의 속성이 있음을 확인할 수 있다.

```
total_cols = len(data.columns)
print("There are "+str(total_cols) + " columns.")
print("The names are shown below.")
data.columns
```

```
There are 9 columns.
The names are shown below.

Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetFrequency',
       'SetDuty', 'SetPower', 'GateOnTime', 'WorkingTime'],
      dtype='object')
```

[코드 6] 데이터 속성 확인

- 컬럼명에 공백이 포함되어 있으면 때때로 그 컬럼의 사용에 불편을 초래한다. 따라서 다음 구문을 사용하여 컬럼명의 공백을 처리한다.



```
data = data.rename(columns=lambda col_name: col_name.strip())
data.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetFrequency', 'SetDuty',
       'SetPower', 'GateOnTime', 'WorkingTime'],
      dtype='object')
```

[코드 7] 데이터 속성 확인

- info() 메소드는 Data frame의 요약 정보를 보여준다. 표시되는 정보는 데이터 수가 135,759개이며, 9개의 속성이 있음을 나타낸다. 또한, 데이터의 유형, non-null value 및 메모리 사용 등의 정보가 표시된다.

```
# Concise summary of the dataframe. This method prints information about a DataFrame
# including the index dtype and columns, non-null values and memory usage.
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 135759 entries, 0 to 135758
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PageNo      135759 non-null  int64
1   Speed       135759 non-null  int64
2   Length      135759 non-null  float64
3   RealPower   135759 non-null  int64
4   SetFrequency 135759 non-null  int64
5   SetDuty     135759 non-null  int64
6   SetPower    135759 non-null  int64
7   GateOnTime  135759 non-null  int64
8   WorkingTime 135759 non-null  object
dtypes: float64(1), int64(7), object(1)
memory usage: 9.3+ MB
```

[코드 8] 데이터 프레임 요약 확인

- 데이터의 형태를 (rows, columns) 형식으로 보여준다. 데이터 수가 135,759개이며, 9개의 속성이 있음을 알 수 있다.

```
data.shape
```

```
(135759, 9)
```

[코드 9] 데이터 형태 확인

①-4. 데이터 특성 파악 및 시각화

- 각 속성의 데이터 유형을 표시하기 위해서는 다음 구문을 사용한다.



```
data.dtypes
```

```
PageNo      int64
Speed       int64
Length      float64
RealPower   int64
SetFrequency int64
SetDuty     int64
SetPower    int64
GateOnTime  int64
WorkingTime object
dtype: object
```

[코드 10] 속성 별 데이터 유형 확인

- 각 속성의 고유한 값 (unique value)의 수를 표시하기 위해서는 다음 구문을 사용한다.

```
data.nunique(axis=0)
```

```
PageNo      39
Speed        6
Length       4
RealPower    71
SetFrequency  1
SetDuty       1
SetPower     3
GateOnTime   9
WorkingTime  135759
dtype: int64
```

[코드 11] 속성 별 고유한 값의 수 확인

- describe() 메소드는 데이터의 기초 통계를 요약하여 보여준다. 데이터의 각 속성에 대하여 데이터수(count), 평균 (mean), 표준편차 (std), 최소값 (min), 25/50/75 분 위수, 그리고 최대값(max) 등의 값을 표시한다.

```
data.describe()
```

	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime
count	135759.00000	135759.000000	135759.000000	135759.000000	135759.0	135759.0	135759.000000	135759.000000
mean	20.00000	165.013811	155.534759	1310.510780	1000.0	100.0	65.384615	1117.989651
std	11.25467	106.977945	107.806431	493.224711	0.0	0.0	21.653072	421.956610
min	1.00000	30.000000	19.400000	0.000000	1000.0	100.0	38.000000	650.000000
25%	10.00000	30.000000	19.400000	688.000000	1000.0	100.0	38.000000	650.000000
50%	20.00000	250.000000	241.100000	1688.000000	1000.0	100.0	82.000000	1154.000000
75%	30.00000	250.000000	241.200000	1704.000000	1000.0	100.0	83.000000	1670.000000
max	39.00000	250.000000	241.200000	1733.000000	1000.0	100.0	83.000000	1670.000000

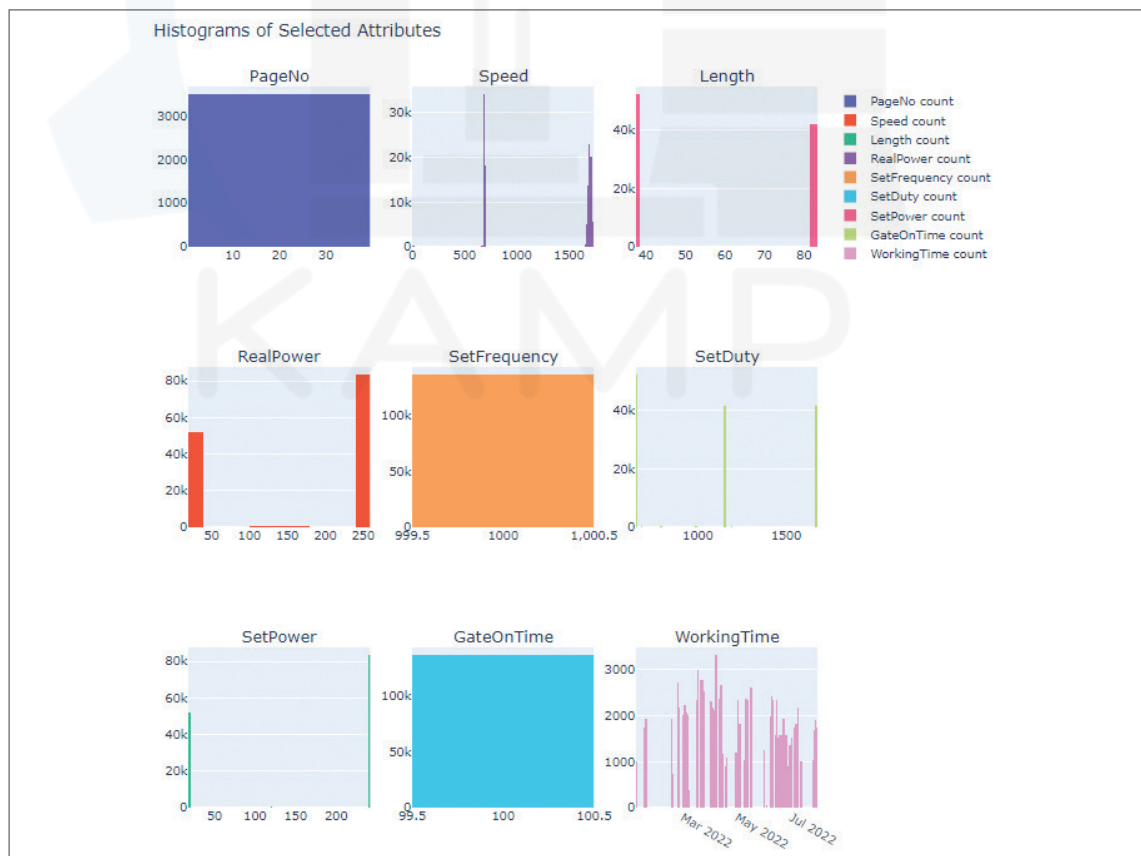
[코드 12] 데이터 기초 통계 요약 확인

- 히스토그램 (Histogram)은 측정값이 존재하는 범위를 복수 개의 구간으로 나눈 후, 각 구간을 X축으로 하고 각 구간에 속하는 측정값의 출현 빈도수를 Y축으로 하여 배열한 그래프이다. 다음 그림은 속성 'PageNo', 'Speed', 'Length', 'RealPower', 'SetFrequency', 'SetDuty', 'SetPower', 'GateOnTime', 'WorkingTime' 에 대하여 히스토그램을 행렬 형태로 나타낸 것이다.

```
# array_attributes = the names of the columns we would like the histograms for
# cols_number = indicate how many columns for the grid
def showHistograms(data, histogram_attributes, cols_number, height, width):
    rows_number = math.ceil(len(histogram_attributes) / cols_number) # rows number calculated automatically
    fig = make_subplots(rows=rows_number, cols=cols_number, subplot_titles=(histogram_attributes))

    for i in range(len(histogram_attributes)):
        legend = histogram_attributes[i] + " count"
        # cols and rows start in 1, not in 0, thus we add one.
        row_number = i % cols_number + 1
        col_number = math.ceil((i+1) / cols_number)
        fig.append_trace(go.Histogram(name=legend, x=data[histogram_attributes[i]]), row_number, col_number)
    fig.update_layout(height=height, width=width,
                      title_text="Histograms of Selected Attributes")
    fig.show()
```

```
# In the histogram_attributes, add the names of the columns we want on the histogram grid
# We can get the available names by using the below line
# print(list(data.columns))
histogram_attributes = ['PageNo', 'Speed', 'Length', 'RealPower', 'SetFrequency', 'SetDuty', 'SetPower',
                        'GateOnTime', 'WorkingTime']
showHistograms(data, histogram_attributes, 3, 900, 900)
```



[코드 13] 히스토그램

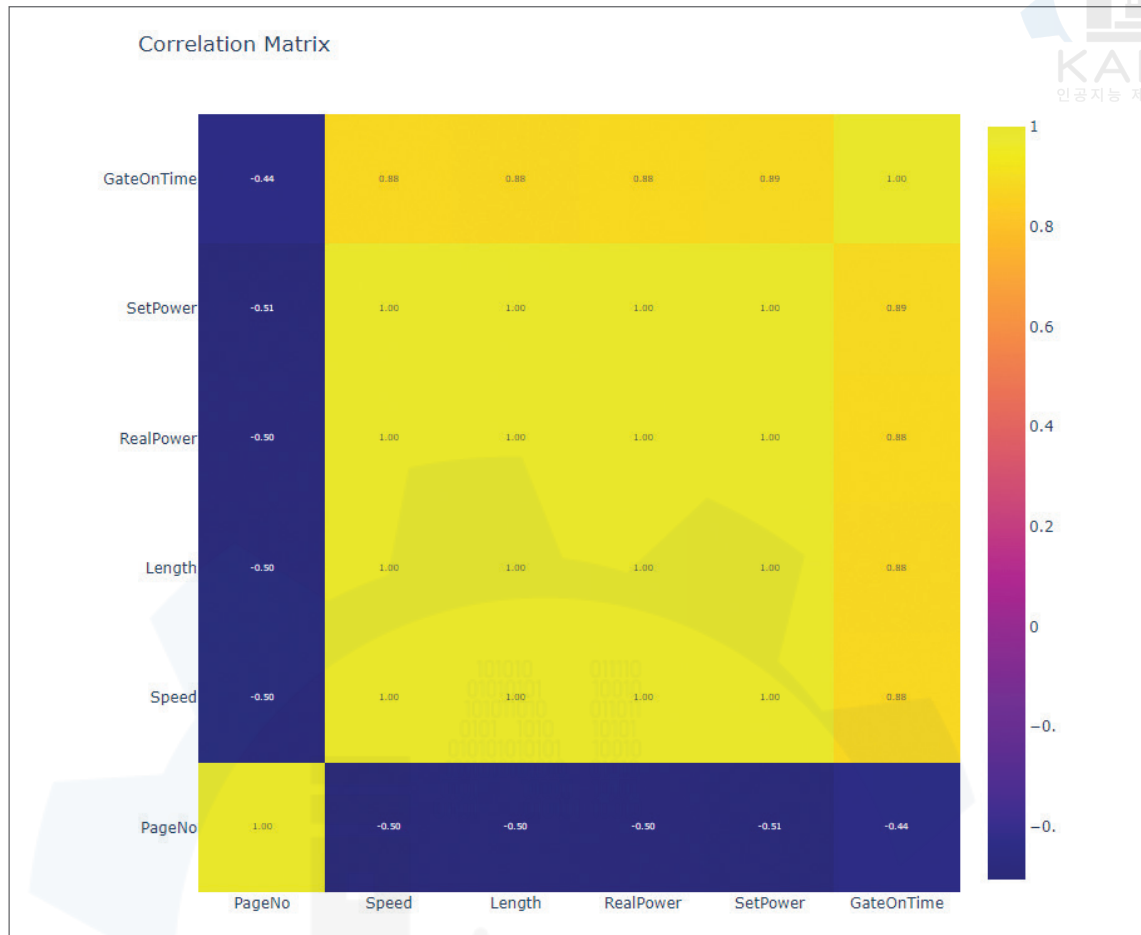
- 상관관계 히트 맵 (Correlation Heat Map)에서 상관관계는 두 변수 간의 연관성 정도를 나타내는 통계적 척도이며, 여기에서는 -1에서 1 사이의 값을 생성하는 Pearson 상관관계를 사용한다. 값이 1이면 두 변수가 강한 양의 상관관계를 갖는 것이고, -1이면 변수는 음의 상관관계를 가진다. 그리고 값이 0이면 두 변수 사이에 상관관계가 없음을 나타낸다.
- 히트맵은 데이터의 패턴과 변화를 시각적으로 보여주는 데 사용된다. 시간에 따른 변화를 보여줄 수 있고, 색상 코딩 시스템을 사용하여 데이터의 다양한 값을 그래픽으로 표현할 수 있다. 즉, 색상으로 표현할 수 있는 다양한 정보를 일정한 이미지 위에 열분포 형태의 비주얼한 그래픽으로 출력하는 시각화 도구이다. 히트맵은 대량의 다차원 데이터를 시각화하는 데 적합하고 유사한 값이 있는 행의 클러스터가 유사한 색으로 표시되므로 이러한 행 클러스터를 식별하는 데 유용하다.
- 모든 변수 쌍 간의 상관관계수 값을 히트맵 그래프를 사용하여 시각화할 수 있다. 히트맵에서 셀의 색상이 노란색이면 양의 상관관계가 있음을 의미하고, 셀의 색상이 파란색이면 음의 상관관계가 있음을 나타낸다. 또한, 셀의 색상이 보라색이면 상관관계가 없거나 충분히 강하지 않음을 나타낸다. 다음 함수 plotCorrelationMatrixText()는 주어진 크기에 따라 상관관계 히트맵을 그려주는 함수이다.

```
# Procedure for Drawing Correlation Heatmap with Coefficient Text Printed
# Input parameter: the dataframe name, the width for heatmap graph, and the number of column
# to visualize.
# Output: correaltion heatmap of the input dataframe with size (width X width)
def plotCorrelationMatrixText(df, graphSize, n):
    df = df[[col for col in df if df[col].nunique() > 1]] # keep columns where there are
                                                         # more than 1 unique values
    df = df.iloc[:, :n+1] #get the first n columns
    if df.shape[1] < 2: #check if there are more than one column in dataset.
        #If only one column in a dataset, print error message.
        print(f'No correlation plots shown: ')
        print(f'The number of non-NaN or constant columns ({df.shape[1]}) is less than 2')
        return
    corr = df.corr() #calculate pearson correlation between all attributes in dataset

    heatmap = go.Heatmap(
        z=corr,
        x=corr.columns,
        y=corr.columns,
        text=corr,
        texttemplate="%{text:.2f}",
        textfont={"size":7}
    )

    layout = go.Layout(
        title_text="Correlation Matrix",
        width=graphSize,
        height=graphSize,
    )
    fig = go.Figure(data=[heatmap], layout=layout)
    fig.show()
```

```
plotCorrelationMatrixText(data, 800, 35) #create heatmap correlation for dataframe data2
                                         #with 10 pixels width and for 35 first column
```

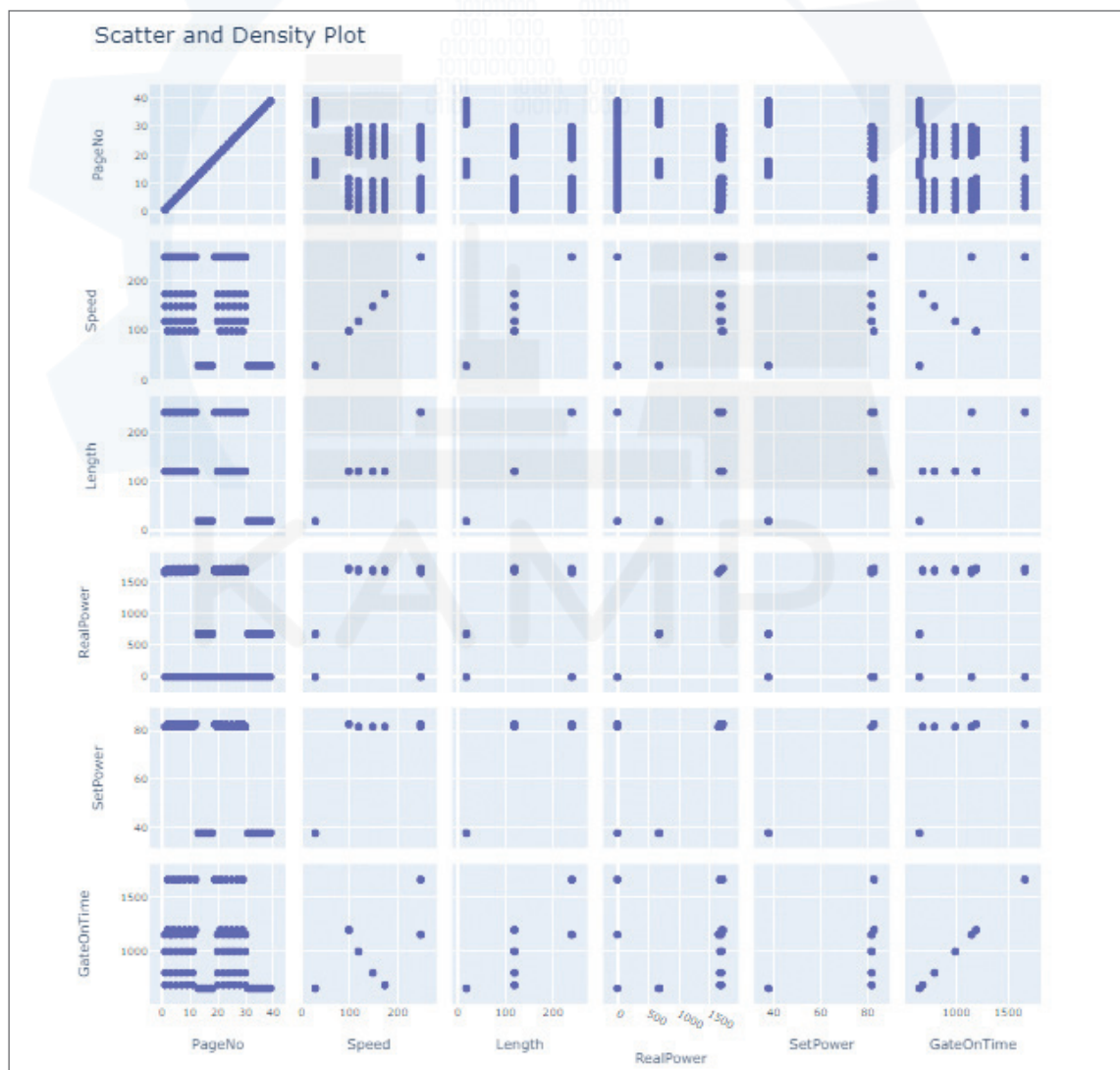


[코드 14] 상관관계 히트맵

- 산점도 행렬(Scatter Matrix)에서 산점도는 두 속성 간의 연관성을 보여주기 위하여 가로 축과 세로 축으로 형성된 2차원 공간에 데이터 포인트를 표시한다. 데이터 테이블의 각 행은 X축 및 Y축에 설정된 값에 따라 위치가 결정되는 마커(marker)로 표시되며, 마커가 산점도에서 직선에 가까운 경우 두 변수의 상관관계가 높고 균등하게 분산되는 경우 상관관계가 낮음을 나타낸다. 산점도 행렬은 데이터셋의 모든 속성 쌍 사이의 산점도를 시각화하여 보여주는 그래프이다.

```
# Procedure for Drawing Scatter Plot Matrix
# Input parameter: the dataframe name, the number of column to visualize, the size of graph, the text size of graph
# Output: Scatter matrix graph with specified size
def plotScatterMatrix(df, n, plotSize):
    df = df.select_dtypes(include=[np.number]) # keep only numerical columns
    df = df.dropna('columns') #drop column with missing data
    df = df[[col for col in df if df[col].nunique() > 1]]
        # keep columns where there are more than 1 unique values
    df = df.iloc[:, :n] #get the first n columns
    columnNames = list(df) #list of all attributes
    df = df[columnNames]
    layout = go.Layout(
        title_text="Scatter matrix",
        width=plotSize,
        height=plotSize,)
    fig=px.scatter_matrix(df)
    fig.update_layout(title='<span style="font-size: 16px;">Scatter and Density Plot</span>', width=plotSize,
        height=plotSize,
        font=dict(size=8))
    fig.show()
```

```
plotScatterMatrix(data, 10, 800) #draw scatter matrix for dataframe dataset2 with size 800px
```



[코드 15] 산점도 행렬



①-5. 데이터 정제 (전처리)

- 결측치 처리(Missing Value Handling) : 데이터에 결측치로 인하여 중요한 의사결정이 달라질 수 있으며, 수집한 데이터에 결측치가 있으면 이를 해결하는 것이 중요하다. 결측치가 전체 데이터에 비해 아주 적다면 가장 좋은 방법은 결측치를 제거하는 것이다. 결측치 제거는 결측치가 존재하는 전체 행을 삭제하는 목록 삭제 방식과 결측치 자체만 삭제하고 다른 변수가 존재하는 경우에는 그대로 유지한 채 분석을 수행하는 페어와이즈 (pairwise) 삭제 방법이 있다. 목록 삭제는 페어와이즈 삭제에 비해 정보 손실이 크다. 그러나, 페어와이즈 삭제의 문제점은 항상 사용할 수 있는 것이 아니고, 샘플과 분석 방법에 따라 다르다.
- file_path에 학습 파일인 “Training_Data.csv”를 지정하고, pandas 라이브러리를 사용하여 csv 파일을 불러온다.

```
file_path = './data/raw_data/train/Training_Data.csv'# the file path is the archive location of the file in our computer
data = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
# After reading the file, it will be used as a Pandas DataFrame.
# Pandas DataFrame is a special data structure from Pandas Library that is two-dimensional,
# size-mutable, potentially heterogeneous tabular data.
```

[코드 16] 데이터 불러오기

- 속성명에 공백이 있으면 때때로 그 속성의 사용에 불편을 초래한다. 따라서 다음 구문을 사용하여 속성명의 공백을 처리한다.

```
data1 = data.rename(columns=lambda x: x.strip())
```

[코드 17] 속성명의 공백 처리

- 개략적인 데이터의 형태를 확인하기 위하여 sample 메소드를 사용하여 5개의 데이터 항목을 표시한다.

```
data1.sample(5)
```

	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime
99219	4	250	241.2	1704	1000	100	83	1670	2022-06-14 11:15:46.787
9542	27	250	241.1	1726	1000	100	83	1670	2022-02-24 14:18:39.484
129739	26	250	241.2	1683	1000	100	82	1154	2022-07-11 10:33:36.445
78926	30	250	241.2	1695	1000	100	82	1154	2022-05-12 09:45:10.712
61774	38	30	19.4	688	1000	100	38	650	2022-04-13 13:52:11.798

[코드 18] 개략적인 데이터 형태 확인

- 데이터의 형태를 (rows, columns) 형식으로 보여주며, 데이터 항목 수가 135,759개이며, 9개의 속성이 있음을 알 수 있다.



```
data1.shape
```

```
(135759, 9)
```

[코드 19] 데이터 형태 확인

- 속성 별 null 개수를 확인하기 위하여 `isna().sum()` 메소드를 사용한다. 표시되는 정보는 다음과 같이 (column name, null 갯수) 형태이다.

```
# Displays the total number of null data of each column
data1.isna().sum()
```

```
PageNo      0
Speed        0
Length       0
RealPower    0
SetFrequency 0
SetDuty       0
SetPower     0
GateOnTime   0
WorkingTime  0
dtype: int64
```

[코드 20] 속성 별 null 개수 확인

- null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다. null 값을 포함하고 있는 행이 없음을 알 수 있다.

```
# Null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다.
null_data = data1[data1.isnull().any(axis=1)]
null_data
```

PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime

[코드 21] null 값을 포함하고 있는 행 확인

- 속성의 값이 오직 하나인 속성을 파악하여 제거한다. 하나의 값만을 가지는 속성은 전체 분석에 의미가 없으므로 이러한 속성은 제거하고 유효한 복수 개의 값을 가지는 속성만을 대상으로 분석을 진행한다.

```
# Function for removing the attribute with unique values below certain threshold
# Input parameter: the dataframe name (df) and threshold value for unique number (n)
# Output: dataframe where all of attributes have more than n unique values
def removeConstant(df, n):
    df = df[[col for col in df if df[col].nunique() > n]]
    return df
data1 = removeConstant(data1, 1)
data1.shape
```

```
135759, 7)
```

[코드 22] 속성의 값이 오직 하나인 속성 제거

- 속성의 값이 오직 하나인 속성을 제거한 후 남은 속성의 수는 7개로서, 2개의 속성이 제거되었음을 알 수 있다. 제거된 속성을 확인하기 위하여 sample() 메소드를 사용하여 데이터 항목을 표시해보면 SetFrequency 와 SetDuty 속성이 제거되었음을 알 수 있다.

```
data1.sample(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
1179	10	250	241.2	1723	83	1670	2022-01-18 11:48:11.513
50449	23	250	241.1	1717	83	1670	2022-04-07 11:20:32.713
9856	29	250	241.1	1727	83	1670	2022-02-24 14:52:27.814
26503	23	250	241.1	1718	83	1670	2022-03-17 16:25:19.595
90039	28	250	241.2	1693	82	1154	2022-06-08 09:13:59.799

[코드 23] 제거된 속성 확인

- 본 분석 파일의 경우에는 모든 속성에서 null 값을 포함하고 있는 행이 없으므로 결측치가 존재하지 않음을 알 수 있다. 결측치가 존재하는 경우에는 결측치 처리를 해야 하며, 결측치 처리를 위한 handleMissingValue() 함수를 제공한다. 이 함수에서는 결측이 발생한 속성이 범주형인 경우는 그 속성의 대표값으로 결측치를 채우고, 수치형인 경우는 결측치가 발생한 시점의 이전 값과 이후 값을 고려하여 보간법을 적용하여 결측치를 채운다. 다음은 handleMissingValue() 함수의 정의이다.

```

# Function for handling missing value
# Input parameter: the dataframe
# Output: dataframe with all missing value already fill in
def handleMissingValue(data):
    df = data.copy()
    numeric_df = df.columns[df.dtypes!='object']
    categorical_df = df.columns[df.dtypes=='object']
    #handling missing value for categorical
    for i in categorical_df:
        df[i].fillna(df[i].mode()[0], inplace=True)

    df_flag_null = df.isnull()
    i, c = np.where(df_flag_null)

    #handling missing value for numerical
    for j in range(len(i)):
        #if missing value in the beginning
        if i[j] == 0:
            s = df.iloc[:, c[j]]
            id_s = s.notna().idxmax()
            val = df.iat[id_s, c[j]]
            df.iat[i[j], c[j]] = val
        #if missing value in the end
        elif i[j] == (len(df)-1):
            s = df.iloc[:, c[j]]
            id_s = s.notna()[::-1].idxmax()
            val = df.iat[id_s, c[j]]
            df.iat[i[j], c[j]] = val
        #if missing value in the middle
        else:
            low = df.iat[i[j]-1, c[j]]
            high = df.iat[i[j]+1, c[j]]
            print(high)
            if(math.isnan(high)):
                val = low
            else:
                val = (low+high) / 2
            df.iat[i[j], c[j]] = val
    return df
  
```

[코드 24] 결측치 처리 함수 정의

- 위의 결측치 처리 함수 handleMissingValue()를 사용하여 결측치를 처리하고 처음 5행을 표시하는 코드는 다음과 같다.



```
data2 = handleMissingValue(data1)
data2.head(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1688	82	1154	2022-01-08 10:11:45.181
1	2	250	241.2	1713	83	1670	2022-01-08 10:11:48.487
2	3	250	241.1	1695	82	1153	2022-01-08 10:11:51.245
3	4	250	241.2	1717	83	1670	2022-01-08 10:11:54.493
4	5	250	241.1	1698	82	1154	2022-01-08 10:11:57.269

[코드 25] 결측치 처리 후 데이터의 처음 5개의 행 확인

- 이상치 처리(Outlier Handling): 이상치는 속성의 값이 일반적인 값과 편차가 큰 값을 말하며, 데이터셋이 이상치를 포함하면 분석의 정확도가 떨어진다. 따라서, 이상치를 검출하여 제거한 후 분석을 수행해야 한다. 이상치 검출 방법으로는 분산 (variance)을 사용하여 정규분포에서 97.5% 이상 또는 2.5%의 이하에 포함되는 값을 이상치로 판별하는 방법, 군집화 (clustering)를 시행하여 데이터를 여러 군집으로 구분한 후 어느 군집에도 속하지 않는 데이터를 이상치로 판별하는 방법, 그리고 사분 범위 (inter-quartile range, IQR)을 이용한 방법 등 다양한 방법이 있다.
- 본 배터리팩 생산 공정 예지보전 분석 실습에서는 데이터의 특성을 고려하여 IQR을 이용한 방법을 사용하여 이상치를 판별하고 제거한다. IQR은 $Q3 - Q1$ 으로 정의되며, 다음 identify_outliers() 함수는 IQR을 사용하여 이상치를 검출한다. 두 번째 인수인 상수 c는 IQR의 c 배수를 의미하며 $c \times IQR$ 범위를 벗어난 값을 이상치로 간주한다.

```
# Function for identify outlier
# Input parameter: the dataframe and constant for IQR boundary
# (make sure all of the column in dataframe is numerical data types)
# Output: all of rows identified as outliers
def identify_outliers(df, c):
    #set the constant for IQR boundary
    constant = float(c)

    # calculate Q1 and Q3
    Q1 = df.quantile(0.25)
    Q3 = df.quantile(0.75)

    # calculate the IQR
    IQR = Q3 - Q1

    # filter the dataset with the IQR
    IQR_outliers = df[((df.lt(Q1-constant*IQR)) | (df.gt(Q3+constant*IQR))).any(axis=1)]
    IQR_outliers = pd.DataFrame(IQR_outliers)
    return IQR_outliers
```

[코드 26] 이상치 판별 함수의 정의

- 이상치가 검출되면 이상치를 포함하고 있는 데이터를 다음의 함수를 사용하여 제거한다.

```
# Function for removing outlier
# Input parameter: the dataframe and constant for IQR boundary
# (make sure all of the column in dataframe is numerical data types)
# Output: dataframe without outliers
def remove_outliers(df, c):
    #find outliers
    df = pd.DataFrame(df)
    outliers = identify_outliers(df, c)
    #remove outliers
    df_out = pd.DataFrame(outliers)
    df.drop(df_out.index, inplace=True)
    return df
```

[코드 27] 이상치 제거 함수 정의

- boolean 속성의 값은 숫자형 (0, 1)로 변환하고, identify_outliers() 함수의 두 번째 인수(c)를 '4'로 주어, 속성의 값이 4 x IQR의 범위를 벗어나면 이상치로 간주하여 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
# 다음으로 boolean 속성의 값을 숫자형 (0, 1)로 변환하고, identify_outliers() 함수의 두 번째 인수를 '4'로 주어, 이 범위를 벗어나면 이상치로 간주하여 이상치를 검출한다.
# 이상치 검출 결과, 여기에서는 다음과 같이 이상치가 검출되지 않았음을 확인할 수 있다.
data_numeric = data2.copy()
for col in data_numeric.columns:
    if (data_numeric[col].dtypes == 'bool'):
        data_numeric[col] = data_numeric[col].astype(int)

print(len(identify_outliers(data_numeric, 4.0)))
```

0

[코드 28] 이상치 검출

- 이상치가 발생한 행을 확인하는 코드는 다음과 같다. 이러한 이상치에 대하여서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 알 수 있다.

```
a = identify_outliers(data_numeric[:, 4.0]) #get the list of index of outliers data
a.index
```

Int64Index([], dtype='int64')

[코드 29] 이상치가 검출된 행 확인



- 학습에 필요한 컬럼의 위치를 확인하기 위해 컬럼의 이름을 출력한다.

```
data2.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetPower', 'GateOnTime',  
      'WorkingTime'],  
      dtype='object')
```

[코드 30] 데이터 속성 확인

- 학습에서는 'RealPower' 속성의 값을 사용한다. 따라서 'RealPower' 속성의 값을 추출하여 데이터프레임 df_test로 저장한다. 학습 및 결과 표시의 편의를 위하여 낮은 값(1300 미만)일 경우에는 일정한 값(=930.5)을 더하여 사용한다. 그 이유는 정상적인 학습 데이터의 경우 'RealPower' 속성의 값이 1670~1740 근처의 값을 가지는 구간과 650~700 근처의 값을 가지는 구간으로 두 그룹으로 나누어져 나타나는데 이때 두 그룹의 중간값 간의 대략적인 차이값 approx 1000이 각 그룹 내에서의 값의 변화의 크기보다 훨씬 커서 이를 그대로 쓰면 학습 과정에서 그룹 내에서의 작은 변화를 효율적으로 학습하는 것이 용이하지 않으리라고 예상되기 때문이다. 즉, 작은 값의 그룹에 일정한 값을 더함으로써 그룹 간의 차이값과 그룹 내의 값의 변화의 크기 사이에 차이가 줄어들도록 하여 그룹 내에서의 작은 변화도 효율적으로 학습할 수 있도록 하려는 의도이다.

더하는 값 930.5을 택한 것은 그룹 간의 차이값이 그룹 내의 값의 변화의 크기의 서너 배 정도가 되도록 정한 것이다. 이 변환은 본래 데이터의 정보를 전혀 훼손하지 않는다는 것을 알 수 있는데 이는 계산 결과에서 더한 값을 빼주면 원래의 데이터를 얻을 수 있기 때문이다. 이는 또한 데이터의 변화를 그래프로 보여주기에 편리하게 하는 면이 있다. 학습데이터와 마찬가지로 테스트 데이터에서도 N-HITS 알고리즘을 쓰는 경우에는 동일하게 작은 그룹에 값을 더하여 테스트한다.



```
# Data adjusting for convenience of training and Display
cols = [data2.columns[3]]
df_test = data2[cols]
for i in range(len(data2)):
    if data2[data2.columns[3]][i] < 1300.0:
        df_test[data2.columns[3]][i] = df_test[data2.columns[3]][i] + 930.5
df_test
```

RealPower	
0	1688.0
1	1713.0
2	1695.0
3	1717.0
4	1698.0
...	...
135754	1613.5
135755	1613.5
135756	1613.5
135757	1613.5
135758	1612.5

135759 rows × 1 columns

[코드 31] 학습 데이터 중 시계열 추출과 조정

- 새로운 데이터프레임 data3를 생성하고 정수값의 시간 인덱스(time_idx)를 추가하며, 또한 “value” 속성을 새로 정의하여 이에 위에서 얻은 “RealPower” 값을 입력한다. 여기서는 하나의 시계열로 학습하므로 “series” 값은 상수값 0이 된다.

```
timesteps_0 = len(data2)
data3 = generate_ar_data(seasonality=10.0, timesteps=timesteps_0, n_series=1, seed=213)
data3["value"] = df_test[data2.columns[3]]
data3["value"] = data3["value"].astype(float)
data3.head()
```

series	time_idx	value
0	0	1688.0
1	0	1713.0
2	0	1695.0
3	0	1717.0
4	0	1698.0

[코드 32] 학습 데이터의 N-HiTS 네트워크 입력 데이터로 변환



- 전처리된 학습 데이터 “Training_Data.csv”를 저장한다.

```
data3.to_csv('./data/preprocessed/train/Training_Data.csv', index=False)
```

[코드 33 전처리된 데이터 저장]

- 학습 결과 테스트에 사용할 데이터들에 대한 전처리 작업을 진행한다.

“WeldingTest_01_OK.csv”를 불러오고 속성명의 빈칸을 제거하고 단일값 제거 전 데이터 형태를 확인한다.

```
file_path = './data/raw_data/test/WeldingTest_01_OK.csv'
test1 = pd.read_csv(file_path)
test1 = test1.rename(columns=lambda col_name: col_name.strip())
test1.shape
```

```
(1950, 9)
```

[코드 34 테스트 데이터 “WeldingTest_01_OK.csv” 불러오기]

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 단일값을 가지는 열을 제거한다.

```
test1 = removeConstant(test1, 1)
test1.shape
```

```
(1950, 7)
```

[코드 35 테스트 데이터 “WeldingTest_01_OK.csv” 단일값 열 제거]

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다. null 값을 포함하고 있는 행이 없음을 알 수 있다.

```
null_data = test1[test1.isnull().any(axis=1)]
null_data
```

PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
--------	-------	--------	-----------	----------	------------	-------------

[코드 36 테스트 데이터 “WeldingTest_01_OK.csv” null 값을 포함하고 있는 행 확인]

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 결측치를 처리한다.

```
test1 = handleMissingValue(test1)
test1.head(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1662	82	1154	2022-02-10 09:02:48.515
1	2	250	241.2	1688	83	1670	2022-02-10 09:02:51.894
2	3	250	241.1	1669	82	1153	2022-02-10 09:02:54.740
3	4	250	241.2	1693	83	1670	2022-02-10 09:02:57.948
4	5	250	241.1	1673	82	1154	2022-02-10 09:03:00.734

[코드 37 테스트 데이터 “WeldingTest_01_OK.csv” 결측치 처리]



- 테스트 데이터 “WeldingTest_01_OK.csv”에서 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
data_numeric1 = test1.copy()
for col in data_numeric1.columns:
    if (data_numeric1[col].dtypes == 'bool'):
        data_numeric1[col] = data_numeric1[col].astype(int)

print(len(identify_outliers(data_numeric1, 4.0)))
```

0

[코드 38] 테스트 데이터 “WeldingTest_01_OK.csv” 이상치 검출

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 이상치가 발견되었다면 발생한 행을 확인하는 코드는 다음과 같다. 이러한 이상치에 대하여서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 알 수 있다.

```
a = identify_outliers(data_numeric1[:, 4.0]) #get the list of index of outliers data
a.index
```

[코드 39] 테스트 데이터 “WeldingTest_01_OK.csv” 이상치가 검출된 행 확인

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 테스트에 필요한 속성의 위치를 확인하기 위해 속성의 이름을 출력한다.

```
test1.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetPower', 'GateOnTime',
       'WorkingTime'],
      dtype='object')
```

[코드 40] 테스트 데이터 “WeldingTest_01_OK.csv” 데이터 속성 확인

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 학습 데이터의 경우와 마찬가지로 테스트 데이터의 필요한 속성 즉 ‘RealPower’ 값을 추출하고 학습과 보여주기에 편리하도록 데이터를 조정한다.

```
# Data adjusting for convenience of training and Display
cols = [test1.columns[3]]
df_test1 = test1[cols]
for i in range(len(test1)):
    if test1[test1.columns[3]][i] < 1300.0:
        df_test1[test1.columns[3]][i] = df_test1[test1.columns[3]][i] + 930.5
df_test1
```

RealPower	
0	1662.0
1	1688.0
2	1669.0
3	1693.0
4	1673.0
...	...
1945	1613.5
1946	1612.5
1947	1612.5
1948	1613.5
1949	1612.5

[코드 41] 테스트 데이터 “WeldingTest_01_OK.csv” 시계열 추출과 조정

- 테스트 데이터 “WeldingTest_01_OK.csv”에서 학습 데이터의 경우와 마찬가지로 새로운 데이터프레임 test1_new를 생성하고 정수값의 시간 인덱스(time_idx)를 추가하며, 또한 “value” 속성을 새로 정의하여 테스트 데이터의 조정된 “RealPower” 값을 입력한다. 여기서도 하나의 시계열로 테스트하므로 “series” 값은 상수값 0이 된다.

```
timesteps_1 = len(test1)
test1_new = generate_ar_data(seasonality=10.0, timesteps=timesteps_1, n_series=1, seed=213)
test1_new["value"] = df_test1[test1.columns[3]]
test1_new["value"] = test1_new["value"].astype(float)
test1_new.head()
```

series	time_idx	value
0	0	1662.0
1	0	1688.0
2	0	1669.0
3	0	1693.0
4	0	1673.0

[코드 42] 테스트 데이터 “WeldingTest_01_OK.csv” N-HiTS 네트워크 입력 데이터

- 전처리된 테스트 데이터 “WeldingTest_01_OK.csv”를 저장한다.



```
test1_new.to_csv('./data/preprocessed/test/WeldingTest_01_OK.csv', index=False)
```

[코드 43] 전처리된 테스트 데이터 “WeldingTest_01_OK.csv” 저장

- “WeldingTest_02_OK.csv”를 불러오고 속성명의 빈칸을 제거하고 단일값 제거 전 데이터 형태를 확인한다.

```
file_path = './data/raw_data/test/WeldingTest_02_OK.csv'
test2 = pd.read_csv(file_path)
test2 = test2.rename(columns=lambda col_name: col_name.strip())
test2.shape
```

```
(1872, 9)
```

[코드 44] 테스트 데이터 “WeldingTest_02_OK.csv” 불러오기

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 단일값을 가지는 열을 제거한다.

```
test2 = removeConstant(test2, 1)
test2.shape
```

```
(1872, 7)
```

[코드 45] 테스트 데이터 “WeldingTest_02_OK.csv” 단일값 열 제거

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다. null 값을 포함하고 있는 행이 없음을 알 수 있다.

```
null_data = test2[test2.isnull().any(axis=1)]
null_data
```

PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
--------	-------	--------	-----------	----------	------------	-------------

[코드 46] 테스트 데이터 “WeldingTest_02_OK.csv” null 값을 포함하고 있는 행 확인

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 결측치를 처리한다.

```
test2 = handleMissingValue(test2)
test2.head(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1660	82	1154	2022-02-11 09:12:28.065
1	2	250	241.2	1685	83	1670	2022-02-11 09:12:31.355
2	3	250	241.1	1666	82	1153	2022-02-11 09:12:34.187
3	4	250	241.2	1690	83	1670	2022-02-11 09:12:37.429
4	5	250	241.1	1670	82	1154	2022-02-11 09:12:40.189

[코드 47] 테스트 데이터 “WeldingTest_02_OK.csv” 결측치 처리



- 테스트 데이터 “WeldingTest_02_OK.csv”에서 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
data_numeric2 = test2.copy()
for col in data_numeric2.columns:
    if (data_numeric2[col].dtypes == 'bool'):
        data_numeric2[col] = data_numeric2[col].astype(int)

print(len(identify_outliers(data_numeric2, 4.0)))
```

```
0
```

[코드 48] 테스트 데이터 “WeldingTest_02_OK.csv” 이상치 검출

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 이상치가 발견되었다면 발생한 행을 확인하는 코드는 다음과 같다. 이러한 이상치에 대하여서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 알 수 있다.

```
a = identify_outliers(data_numeric2[:, 4.0]) #get the list of index of outliers data
a.index
```

```
Int64Index([], dtype='int64')
```

[코드 49] 테스트 데이터 “WeldingTest_02_OK.csv” 이상치가 검출된 행 확인

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 테스트에 필요한 속성의 위치를 확인하기 위해 속성의 이름을 출력한다.

```
test2.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetPower', 'GateOnTime',
       'WorkingTime'],
      dtype='object')
```

[코드 50] 테스트 데이터 “WeldingTest_02_OK.csv” 데이터 속성 확인

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 학습 데이터의 경우와 마찬가지로 테스트 데이터의 필요한 속성 즉 ‘RealPower’ 값을 추출하고 학습과 보여주기에 편리하도록 데이터를 조정한다.

```
# Data adjusting for convenience of training and Display
cols = [test2.columns[3]]
df_test2 = test2[cols]
for i in range(len(test2)):
    if test2[test2.columns[3]][i] < 1300.0:
        df_test2[test2.columns[3]][i] = df_test2[test2.columns[3]][i] + 930.5
df_test2
```

RealPower	
0	1660.0
1	1685.0
2	1666.0
3	1690.0
4	1670.0
...	...
1867	1612.5
1868	1612.5
1869	1612.5
1870	1612.5
1871	1612.5

[코드 51] 테스트 데이터 “WeldingTest_02_OK.csv” 시계열 추출과 조정

- 테스트 데이터 “WeldingTest_02_OK.csv”에서 학습 데이터의 경우와 마찬가지로 새로운 데이터프레임 test2_new를 생성하고 정수값의 시간 인덱스(time_idx)를 추가하며, 또한 “value” 속성을 새로 정의하여 테스트 데이터의 조정된 “RealPower” 값을 입력한다. 여기서도 하나의 시계열로 테스트하므로 “series” 값은 상수값 0이 된다.

```
timesteps_2 = len(test2)
test2_new = generate_ar_data(seasonality=10.0, timesteps=timesteps_2, n_series=1, seed=213)
test2_new["value"] = df_test2[test2.columns[3]]
test2_new["value"] = test2_new["value"].astype(float)
test2_new.head()
```

	series	time_idx	value
0	0	0	1660.0
1	0	1	1685.0
2	0	2	1666.0
3	0	3	1690.0
4	0	4	1670.0

[코드 52] 테스트 데이터 “WeldingTest_02_OK.csv” N-HiTS 네트워크 입력 데이터

- 전처리된 테스트 데이터 “WeldingTest_02_OK.csv”를 저장한다.

```
test2_new.to_csv('./data/preprocessed/test/WeldingTest_02_OK.csv', index=False)
```

[코드 53] 전처리된 테스트 데이터 “WeldingTest_02_OK.csv” 저장

- “WeldingTest_03_NG.csv”을 불러올 순서이며, 해당 테스트 데이터의 경우 이상이 들어 있는(비정상적인) 테스트 데이터 파일이다. 이 데이터에서의 이상값은 여러 개의 특정한 고립된 시점에서 “RealPower” 값이 정상값에서 크게 벗어나는 데이터이다.
- “WeldingTest_03_NG.csv”를 불러오고 속성명의 빈칸을 제거하고 단일값 제거 전 데이터 형태를 확인한다.

```
file_path = './data/raw_data/test/WeldingTest_03_NG.csv'
test3 = pd.read_csv(file_path)
test3 = test3.rename(columns=lambda col_name: col_name.strip())
test3.shape
```

```
(1053, 9)
```

[코드 54] 테스트 데이터 “WeldingTest_03_NG.csv” 불러오기

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 단일값을 가지는 열을 제거한다.

```
test3 = removeConstant(test3, 1)
test3.shape
```

```
(1053, 7)
```

[코드 55] 테스트 데이터 “WeldingTest_03_NG.csv” 단일값 열 제거

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다. null 값을 포함하고 있는 행이 없음을 알 수 있다.

```
null_data = test3[test3.isnull().any(axis=1)]
null_data
```

PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
--------	-------	--------	-----------	----------	------------	-------------

[코드 56] 테스트 데이터 “WeldingTest_03_NG.csv” null 값을 포함하고 있는 행 확인

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 결측치를 처리한다.



```
test3 = handleMissingValue(test3)
test3.head(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1660	82	1154	2022-08-03 09:10:24.996
1	2	250	241.2	1685	83	1670	2022-08-03 09:10:28.312
2	3	250	241.1	1666	82	1153	2022-08-03 09:10:31.076
3	4	250	241.2	1690	83	1670	2022-08-03 09:10:34.284
4	5	250	241.1	1670	82	1154	2022-08-03 09:10:37.046

[코드 57] 테스트 데이터 “WeldingTest_03_NG.csv” 결측치 처리

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
data_numeric3 = test3.copy()
for col in data_numeric3.columns:
    if (data_numeric3[col].dtypes == 'bool'):
        data_numeric3[col] = data_numeric3[col].astype(int)

print(len(identify_outliers(data_numeric3, 4.0)))
```

0

[코드 58] 테스트 데이터 “WeldingTest_03_NG.csv” 이상치 검출

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 이상치가 발견되었다면 발생한 행을 확인하는 코드는 다음과 같다. 이러한 이상치에 대하여서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 알 수 있다.

```
a = identify_outliers(data_numeric3[:, 4.0]) #get the list of index of outliers data
a.index
```

Int64Index([], dtype='int64')

[코드 59] 테스트 데이터 “WeldingTest_03_NG.csv” 이상치가 검출된 행 확인

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 테스트에 필요한 속성의 위치를 확인하기 위해 속성의 이름을 출력한다.

```
test3.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetPower', 'GateOnTime',
      'WorkingTime'],
      dtype='object')
```

[코드 60] 테스트 데이터 “WeldingTest_03_NG.csv” 데이터 속성 확인



- 테스트 데이터 “WeldingTest_03_NG.csv”에서 학습 데이터의 경우와 마찬가지로 테스트 데이터의 필요한 속성 즉 ‘RealPower’ 값을 추출하고 학습과 보여주기에 편리하도록 데이터를 조정한다.

```
# Data adjusting for convenience of training and Display
cols = [test3.columns[3]]
df_test3 = test3[cols]
for i in range(len(test3)):
    if test3[test3.columns[3]][i] < 1300.0:
        df_test3[test3.columns[3]][i] = df_test3[test3.columns[3]][i] + 930.5
df_test3
```

RealPower	
0	1660.0
1	1685.0
2	1666.0
3	1690.0
4	1670.0
...	...
1048	1610.5
1049	1610.5
1050	1610.5
1051	1610.5
1052	1610.5

1053 rows × 1 columns

[코드 61] 테스트 데이터 “WeldingTest_03_NG.csv” 시계열 추출과 조정

- 테스트 데이터 “WeldingTest_03_NG.csv”에서 학습 데이터의 경우와 마찬가지로 새로운 데이터프레임 test3_new를 생성하고 정수값의 시간 인덱스(time_idx)를 추가하며, 또한 “value” 속성을 새로 정의하여 테스트 데이터의 조정된 “RealPower” 값을 입력한다. 여기서도 하나의 시계열로 테스트하므로 “series” 값은 상수값 0이 된다.

```
timesteps_3 = len(test3)
test3_new = generate_ar_data(seasonality=10.0, timesteps=timesteps_3, n_series=1, seed=213)
test3_new["value"] = df_test3[test3.columns[3]]
test3_new["value"] = test3_new["value"].astype(float)
test3_new.head()
```

series	time_idx	value
0	0	0 1660.0
1	0	1 1685.0
2	0	2 1666.0
3	0	3 1690.0
4	0	4 1670.0

[코드 62] 테스트 데이터 “WeldingTest_03_NG.csv” N-HiTS 네트워크 입력 데이터



- 전처리된 테스트 데이터 “WeldingTest_03_NG.csv”를 저장한다.

```
test3_new.to_csv('./data/preprocessed/test/WeldingTest_03_NG.csv', index=False)
```

[코드 63] 전처리된 테스트 데이터 “WeldingTest_03_NG.csv” 저장

- “WeldingTest_04_NG.csv”을 불러올 순서이며, 해당 테스트 데이터의 경우 일정한 시점 이후 모든 값이 연속적으로 정상에서 벗어나는 값을 나타내고 있다. 이를 분석하기 위해서는 N-HiTS 방식보다는 통계적인 분석이 더 적합해 보인다. 그리하여 위의 전처리 작업과는 다르게 진행한다. 즉, 작은 값의 그룹에 일정한 값을 더하거나 하지 않고 데이터를 있는 그대로 데이터의 분포의 특징을 이용한다.
- 정상적인 학습데이터의 경우에 “RealPower” 데이터의 값이 (1650~1750) 구역과 (650~700) 구역의 두 가지 구역으로 갈라져서 분포하고 있으므로 각각의 구역에 대하여 정규분포로 가정하고 그 평균값과 표준편차를 따로 구할 수 있도록 학습 작업을 진행한다.
- “WeldingTest_04_NG.csv”를 불러오고 속성명의 빈칸을 제거하고 단일값 제거 전 데이터 형태를 확인한다.

```
file_path = './data/raw_data/test/WeldingTest_04_NG.csv'
test4 = pd.read_csv(file_path)
test4 = test4.rename(columns=lambda col_name: col_name.strip())
test4.shape
```

```
(351, 9)
```

[코드 64] 테스트 데이터 “WeldingTest_04_NG.csv” 불러오기

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 단일값을 가지는 열을 제거한다.

```
test4 = removeConstant(test4, 1)
test4.shape
```

```
(351, 7)
```

[코드 65] 테스트 데이터 “WeldingTest_04_NG.csv” 단일값 열 제거

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다. null 값을 포함하고 있는 행이 없음을 알 수 있다.

```
null_data = test4[test4.isnull().any(axis=1)]
null_data
```

PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
--------	-------	--------	-----------	----------	------------	-------------

[코드 66] 테스트 데이터 “WeldingTest_04_NG.csv” null 값을 포함하고 있는 행 확인

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 결측치를 처리한다.



```
test4 = handleMissingValue(test4)
test4.head(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1713	82	1154	2022-08-03 15:24:10.491
1	2	250	241.2	1695	83	1670	2022-08-03 15:24:13.743
2	3	250	241.1	1718	82	1153	2022-08-03 15:24:16.537
3	4	250	241.2	1698	83	1670	2022-08-03 15:24:19.741
4	5	250	241.1	1721	82	1154	2022-08-03 15:24:22.509

[코드 67] 테스트 데이터 “WeldingTest_04_NG.csv” 결측치 처리

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
data_numeric4 = test4.copy()
for col in data_numeric4.columns:
    if (data_numeric4[col].dtypes == 'bool'):
        data_numeric4[col] = data_numeric4[col].astype(int)

print(len(identify_outliers(data_numeric4, 4.0)))
```

```
0
```

[코드 68] 테스트 데이터 “WeldingTest_04_NG.csv” 이상치 검출

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 이상치가 발견되었다면 발생한 행을 확인하는 코드는 다음과 같다. 이러한 이상치에 대하여서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 알 수 있다.

```
a = identify_outliers(data_numeric4[:, 4.0]) #get the list of index of outliers data
a.index
```

```
Int64Index([], dtype='int64')
```

[코드 69] 테스트 데이터 “WeldingTest_04_NG.csv” 이상치가 검출된 행 확인

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 테스트에 필요한 속성의 위치를 확인하기 위해 속성의 이름을 출력한다.

```
test4.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetPower', 'GateOnTime',
      'WorkingTime'],
      dtype='object')
```

[코드 70] 테스트 데이터 “WeldingTest_04_NG.csv” 데이터 속성 확인

- 테스트 데이터 “WeldingTest_04_NG.csv”에서 ‘RealPower’ 값을 추출한다.

```
cols = [test4.columns[3]]
df_test4 = test4[cols]
df_test4["RealPower"] = df_test4["RealPower"].astype(float)
df_test4
```

RealPower	
0	1713.0
1	1695.0
2	1718.0
3	1698.0
4	1721.0
...	...
346	127.0
347	127.0
348	127.0
349	126.0
350	127.0

351 rows × 1 columns

[코드 71] 테스트 데이터 “WeldingTest_04_NG.csv” 데이터 추출

- 전처리된 테스트 데이터 “WeldingTest_04_NG.csv”를 저장한다.

```
df_test4.to_csv('./data/preprocessed/test/WeldingTest_04_NG.csv', index=False)
```

[코드 72] 전처리된 테스트 데이터 “WeldingTest_04_NG.csv” 저장

- 통계적 분석을 위해 “Traing_Data.csv”를 다시 불러올 순서이며, “WeldingTest_04_NG.csv”의 이상값 탐지에 이용하기 위해 새로운 전처리를 진행한다. 전처리 방식은 “WeldingTest_04_NG.csv” 전처리와 동일하다.
- “Training_Data.csv”를 불러오고 속성명의 빈칸을 제거하고 단일값 제거 전 데이터 형태를 확인한다.

```
file_path = './data/raw_data/train/Training_Data.csv'
test5 = pd.read_csv(file_path)
test5 = test5.rename(columns=lambda col_name: col_name.strip())
test5.shape
```

```
(135759, 9)
```

[코드 73] 학습 데이터 “Training_Data.csv” 불러오기

- 학습 데이터 “Training_Data.csv”에서 단일값을 가지는 열을 제거한다.



```
test5 = removeConstant(test5, 1)
test5.shape
```

```
(135759, 7)
```

[코드 74] 학습 데이터 “Training_Data.csv” 단일값 열 제거

- 학습 데이터 “Training_Data.csv”에서 null 값을 포함하고 있는 행을 표시하기 위해서는 다음 구문을 사용한다. null 값을 포함하고 있는 행이 없음을 알 수 있다.

```
null_data = test5[test5.isnull().any(axis=1)]
null_data
```

PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
--------	-------	--------	-----------	----------	------------	-------------

[코드 75] 학습 데이터 “Training_Data.csv” null 값을 포함하고 있는 행 확인

- 학습 데이터 “Training_Data.csv”에서 결측치를 처리한다.

```
test5 = handleMissingValue(test5)
test5.head(5)
```

	PageNo	Speed	Length	RealPower	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1688	82	1154	2022-01-08 10:11:45.181
1	2	250	241.2	1713	83	1670	2022-01-08 10:11:48.487
2	3	250	241.1	1695	82	1153	2022-01-08 10:11:51.245
3	4	250	241.2	1717	83	1670	2022-01-08 10:11:54.493
4	5	250	241.1	1698	82	1154	2022-01-08 10:11:57.269

[코드 76] 학습 데이터 “Training_Data.csv” 결측치 처리

- 학습 데이터 “Training_Data.csv”에서 이상치를 검출한다. identify_outliers() 함수의 실행 결과, 여기에서는 이상치가 검출되지 않았음을 확인할 수 있다.

```
data_numeric5 = test5.copy()
for col in data_numeric5.columns:
    if (data_numeric5[col].dtypes == 'bool'):
        data_numeric5[col] = data_numeric5[col].astype(int)

print(len(identify_outliers(data_numeric5, 4.0)))
```

```
0
```

[코드 77] 학습 데이터 “Training_Data.csv” 이상치 검출

- 학습 데이터 “Training_Data.csv”에서 이상치가 발견되었다면 발생한 행을 확인하는 코드는 다음과 같다. 이러한 이상치에 대하여서는 필요에 따라 제거하거나 대체할 수 있다. 여기에서는 이상치가 발생하지 않았음을 알 수 있다.



```
a = identify_outliers(data_numeric5[:, 4.0]) #get the list of index of outliers data
a.index
```

```
Int64Index([], dtype='int64')
```

[코드 78] 학습 데이터 “Training_Data.csv” 이상치가 검출된 행 확인

- 학습 데이터 “Training_Data.csv”에서 테스트에 필요한 속성의 위치를 확인하기 위해 속성의 이름을 출력한다.

```
test5.columns
```

```
Index(['PageNo', 'Speed', 'Length', 'RealPower', 'SetPower', 'GateOnTime',
       'WorkingTime'],
      dtype='object')
```

[코드 79] 학습 데이터 “Training_Data.csv” 데이터 속성 확인

- 학습 데이터 “Training_Data.csv”에서 ‘RealPower’ 값을 추출한다.

```
cols = [test5.columns[3]]
df_test5 = test5[cols]
df_test5["RealPower"] = df_test5["RealPower"].astype(float)
df_test5
```

	RealPower
0	1688.0
1	1713.0
2	1695.0
3	1717.0
4	1698.0
...	...
135754	683.0
135755	683.0
135756	683.0
135757	683.0
135758	682.0

135759 rows × 1 columns

[코드 80] 학습 데이터 “Training_Data.csv” 데이터 추출

- 전처리된 학습 데이터 “Training_Data_Real_Power.csv”를 저장한다.

```
df_test5.to_csv('./data/preprocessed/train/Training_Data_Real_Power.csv', index=False)
```

[코드 81] 전처리된 학습 데이터 “Training_Data_Real_Power.csv” 저장



[단계 ②] 데이터 구조 및 특징 탐색

②-1. 데이터 구조 및 특징 탐색

- 데이터의 구조와 특징에 대해서는 전처리 과정을 참조한다.
- 데이터에서 중요하게 고려할 부분은 “RealPower”로서 “SetPower”의 값에 따라 대략 일정한 주기성을 띠는 기본 거동을 보이고 이에 더하여 상대적으로 작은 잡음 신호가 더해진 성질을 보인다. 즉 “SetPower”의 값은 38%, 82%, 83%로서 이상적인 RealPower의 값은 $2000 \times \text{SetPower}$ 의 값이 되는데 실제로는 이 값에서 어느 정도 차이를 보인다. 공정 설명에 의하면 이상적인 값에서 2000의 3%~4% 즉 60~80 이내를 허용한도로 잡는다. 1% 차이는 20에 해당하는데 이는 예측모형의 정확도의 대략적인 기준으로 쓸 수 있다.

[단계 ③] 학습, 테스트 데이터 선택

③-1. 학습 데이터 선택

- 학습 데이터는 충분히 긴 시간 정상적인 거동을 보이는 데이터를 택한다. 본 가이드북에서는 사용하는 학습 데이터는 “Training_Data.csv”이다.

③-2. 테스트 데이터 선택

- 테스트 데이터는 정상적인 데이터와 비정상적인 데이터를 각각 택하여 테스트를 수행한다. 비정상적인 데이터는 두 가지가 있는데 하나는 고립된 시점의 RealPower 값이 정상에서 크게 벗어나는 경우(“WeldingTest_03_NG.csv”)이고 다른 하나는 연속적인 시간 구간에서 비정상 값을 보이는 경우이다 (“WeldingTest_04_NG.csv”). 두 가지의 경우에 대해 각각 다른 분석 방법을 적용한다.

[단계 ④] AI 모델 알고리즘 선택

④-1. AI 모델 알고리즘

- 데이터분석을 위하여 시계열 예측 알고리즘인 N-HiTS 알고리즘(C. Challu, et al, 2022)을 택한다. N-HiTS 알고리즘은 벤치마크 테스트에서 높은 장시간 예측 정확도를 보이며 메모리 사용이 효율적이며 계산속도도 다른 방법에 비해 빠르다는 점이 이 알고리즘을 택하게 된 이유이다.
- 학습 데이터의 비정상 데이터 값이 연속적으로 긴 시점 구간에서 나타날 경우에는 N-HiTS 알고리즘이 적합하지 않을 수 있다. 이때는 먼저 학습 데이터에 대한 통계분석을 수행하여 학습 데이터의 분포를 몇 개의 가우스 분포의 중첩으로 가정하여 각각



의 가우스 분포 모드에 대한 평균값과 표준편차를 구하고 테스트 데이터에 대하여는 각 시점의 테스트 데이터 값에 대하여 각각의 가우스 모드의 평균값으로부터의 거리를 해당 모드의 표준편차로 나눈 값 즉, Z-score를 사용하여 검출한다.

[단계 ⑤] AI 모델 학습

⑤-1. AI 모델 학습

- 필요한 라이브러리를 설치한다.

```
!pip install cpflows==0.1.2
```

[코드 82] 라이브러리 설치

- 필요한 라이브러리를 불러들인다.

```
import os
import warnings
warnings.filterwarnings("ignore")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch_lightning as pl
from torch_lightning.callbacks import EarlyStopping
import torch
from torch_forecasting import Baseline, NHITS, TimeSeriesDataSet
from torch_forecasting.data import NaNLabelEncoder
from torch_forecasting.data.examples import generate_ar_data
from torch_forecasting.metrics import SMAPE, MQF2DistributionLoss, QuantileLoss
from datetime import datetime
import logging
import matplotlib.dates as mdates
import matplotlib.gridspec as gridspec
from pandas.plotting import register_matplotlib_converters
import math
from sklearn.metrics import precision_recall_fscore_support
from sklearn.metrics import accuracy_score
```

[코드 83] 라이브러리 불러오기

- 학습 데이터를 불러들인다.

```

signal_file = './data/preprocessed/train/Training_Data.csv'
df_train_0 = pd.read_csv(signal_file)
data = df_train_0
data.head()

```

	series	time_idx	value
0	0	0	1688.0
1	0	1	1713.0
2	0	2	1695.0
3	0	3	1717.0
4	0	4	1698.0

[코드 84] 학습 데이터 불러오기

- N-HiTS 알고리즘에서는 임의의 시점 이후의 일정한 구간에 대해 예측할 때에 그 시점까지의 일정한 구간의 데이터 값을 입력으로 이용한다. 아래 코드에서 입력의 길이는 20(“max_encoder_length = 20”)으로 정의되고 예측 구간의 길이는 10(“max_prediction_length = 10”)으로 정의된다. 다음으로 시계열 학습 데이터 클래스(training)와 검증 데이터 클래스(validation)를 정의한다.

```

# create dataset and dataloaders
max_encoder_length = 20
max_prediction_length = 10
training_cutoff = data["time_idx"].max() - 100 * max_prediction_length
context_length = max_encoder_length
prediction_length = max_prediction_length
training = TimeSeriesDataSet(
    data[lambdax: x.time_idx <= training_cutoff],
    time_idx="time_idx",
    target="value",
    #categorical_encoders={"series": NaNLabelEncoder().fit(data.series)},
    group_ids=["series"],
    # only unknown variable is "value" - and N-HiTS can also not take any additional variables
    time_varying_unknown_reals=["value"],
    max_encoder_length=context_length,
    max_prediction_length=prediction_length,
)
validation = TimeSeriesDataSet.from_dataset(training, data, min_prediction_idx=training_cutoff + 1)
batch_size = 128
train_dataloader = training.to_dataloader(train=True, batch_size=batch_size, num_workers=0)
val_dataloader = validation.to_dataloader(train=False, batch_size=batch_size, num_workers=0)

```

[코드 85] N-HiTS 학습 및 검증 데이터 클래스 정의

- N-HiTS 네트워크를 정의한다. 여기에서 learning_rate 변수는 학습률을 의미하고 weight_decay 변수는 로스 함수에서 예측 시점이 멀어짐에 따른 가중치의 감소율을 의미한다. 로스함수는 MQF2DistributionLoss로서 다변수 분포 로스(Multivariate Quantile Loss)이다.



```
pl.seed_everything(42)
trainer = pl.Trainer(gpus=0, gradient_clip_val=1.0)
net = NHiTS.from_dataset(
    training,
    learning_rate=3e-2,
    weight_decay=1e-2,
    loss=MQF2DistributionLoss(prediction_length=max_prediction_length),
    backcast_loss_ratio=0.0,
    hidden_size=64,
)
```

```
Global seed set to 42
C:\Users\SAMSUNG\conda\envs\proc_torch\lib\site-packages\pytorch_lightning\trainer\connectors\accelerator_connector.py:447: LightningDeprecationWarning: Setting `Trainer(gpus=0)` is deprecated in v1.7 and will be removed in v2.0. Please use `Trainer(accelerator='gpu', devices=0)` instead.
rank_zero_deprecation(
GPU available: False, used: False
TPU available: False, using: 0 TPU cores
IPU available: False, using: 0 IPUs
HPU available: False, using: 0 HPUs
```

[코드 86] N-HiTS 학습 네트워크 정의

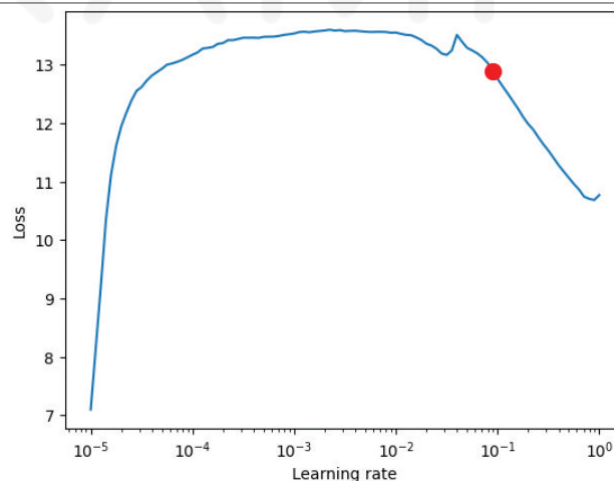
- N-HiTS 알고리즘에서의 최적의 학습률을 찾고 학습을 수행한다. 계산 결과로 제안된 최적 학습률은 0.089로 나오는데 실제 학습에서는 안전하게 이보다 조금 더 낮은 값을 사용하게 된다.

```
# find optimal learning rate
res = trainer.tuner.lr_find(net,
                            train_dataloaders=train_dataloader,
                            val_dataloaders=val_dataloader,
                            min_lr=1e-5)
print(f"suggested learning rate: {res.suggestion()}")
fig = res.plot(show=True, suggest=True)
fig.show()
net.hparams.learning_rate = res.suggestion()
```

```
Finding best initial lr: 0% | 0/100 [00:00<?, ?it/s]
```

```
`Trainer.fit` stopped: `max_steps=100` reached.
Restoring states from the checkpoint path at C:\Users\SAMSUNG\Desktop\Battery_AD\용접 데이터셋. lr_find_e93899f8-5e79-413e-be8c-ed9e3d8c1568.ckpt
```

```
suggested learning rate: 0.08912509381337459
```



[코드 87] N-HiTS 최적 학습률(learning rate) 추정



- 학습 클래스 변수(trainer)와 N-HiTS 네트워크 변수(net)를 정의하고 N-HiTS 학습을 수행한다. 최대 학습 반복 횟수는 30(max_epochs=30)으로 하였고 학습률은 0.04(learning_rate=0.04)로 하였다.

```
early_stop_callback = EarlyStopping(monitor="val_loss", min_delta=1e-4, patience=10, verbose=False,
mode="min")
trainer = pl.Trainer(
    max_epochs=30,
    gpus=0,
    enable_model_summary=True,
    gradient_clip_val=1.0,
    callbacks=[early_stop_callback],
    limit_train_batches=30,
    enable_checkpointing=True,
)
net = NHiTS.from_dataset(
    training,
    learning_rate=0.04,
    log_interval=10,
    log_val_interval=1,
    weight_decay=1e-2,
    backcast_loss_ratio=0.0,
    hidden_size=64,
    loss=MQF2DistributionLoss(prediction_length=max_prediction_length),
)
trainer.fit(
    net,
    train_dataloaders=train_dataloader,
    val_dataloaders=val_dataloader,
)
```

```
GPU available: False, used: False
TPU available: False, using: 0 TPU cores
IPU available: False, using: 0 IPUs
HPU available: False, using: 0 HPUs
```

	Name	Type	Params
0	loss	MQF2DistributionLoss	3.5 K
1	logging_metrics	ModuleList	0
2	embeddings	MultiEmbedding	0
3	model	NHiTS	23.0 K
26.5 K	Trainable params		
0	Non-trainable params		
26.5 K	Total params		
0.106	Total estimated model params size (MB)		

[코드 88] N-HiTS 학습 수행

- 위에서 학습된 N-HiTS 네트워크는 ./lightning_logs/version_0/checkpoints 폴더에 저장되는데 이를 best_model 변수로 불러들여서 검증데이터에 적용하여 절대값 에러의 평균값을 구해본다. 이 구간부터 가이드북과 실제 결과과 다르게 나올 수 있는데, 그 이유는 AI 모델의 학습 차이로 볼 수 있다.

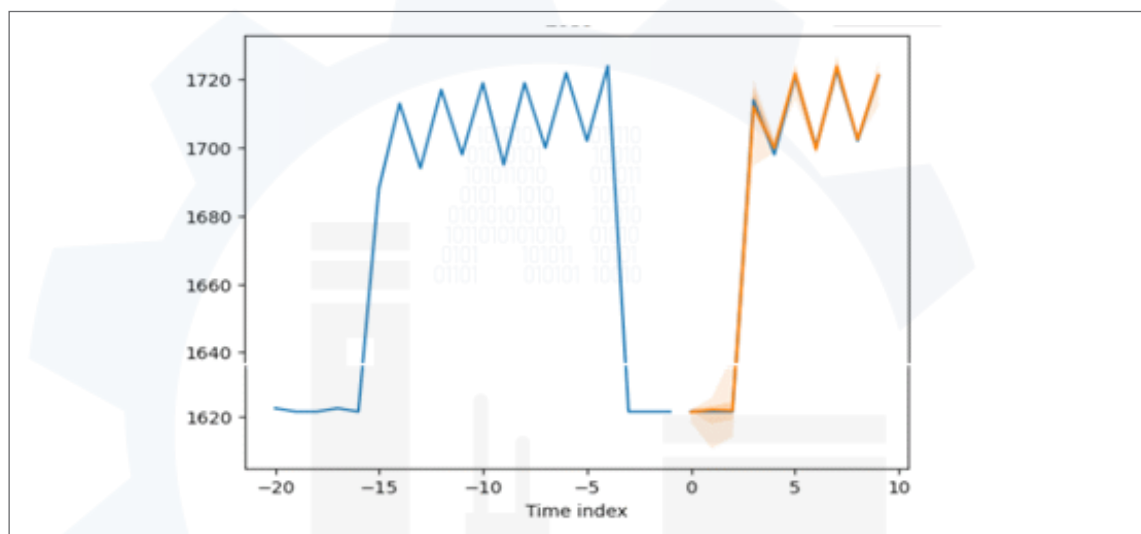
```
best_model_path = trainer.checkpoint_callback.best_model_path
best_model = NHITS.load_from_checkpoint(best_model_path)
actuals = torch.cat([y[0] for x, y in iter(val_dataloader)])
predictions = best_model.predict(val_dataloader)
(actuals-predictions).abs().mean()
```

```
tensor(2.0532)
```

[코드 89] N-HITS 검증 데이터 테스트 수행

- 검증데이터에 대한 예측값과 실제값을 비교하여 그래프로 나타낸다. 아래 그림에서 보듯이 상당히 높은 예측 정확도를 보인다.

```
raw_predictions, x = best_model.predict(val_dataloader, mode="raw", return_x=True)
for idx in range(20):
    best_model.plot_prediction(x, raw_predictions, idx=idx, add_loss_to_title=True)
```



[코드 90] 검증 데이터 예측과 실제 데이터 비교

- 검증데이터에 대한 예측값과 실제값의 차이 즉 에러 값을 출력해본다.

```
errors = (actuals - predictions)
print(errors[:5])
```

```
tensor([[ 0.4613, -0.0155,  0.9480, -1.1418, -5.4849, -0.6193, -2.9961, -3.3326,
          -3.2148,  1.5212],
        [ 1.7002,  0.3093,  1.7145, -0.8975, -2.7294, -2.3091, -3.3804, -3.3794,
          1.4795, -1.8185],
        [ 1.8696,  0.3564, -1.6718, -0.5487, -2.7607, -1.9365, -1.7043, -0.4672,
         -1.3405, -0.9996],
        [ 1.4343, -1.6431, -2.0387, -3.7954, -3.9500, -1.1089, -0.6190, -4.2345,
         -0.6879, -1.1801],
        [ 4.6566, -4.9025, -1.8795, -4.6919, -1.5601, -1.5927, -2.7223, -3.3970,
          2.0697, -2.1172],
```

[코드 91] 검증 데이터 예측과 실제 데이터 값의 차이

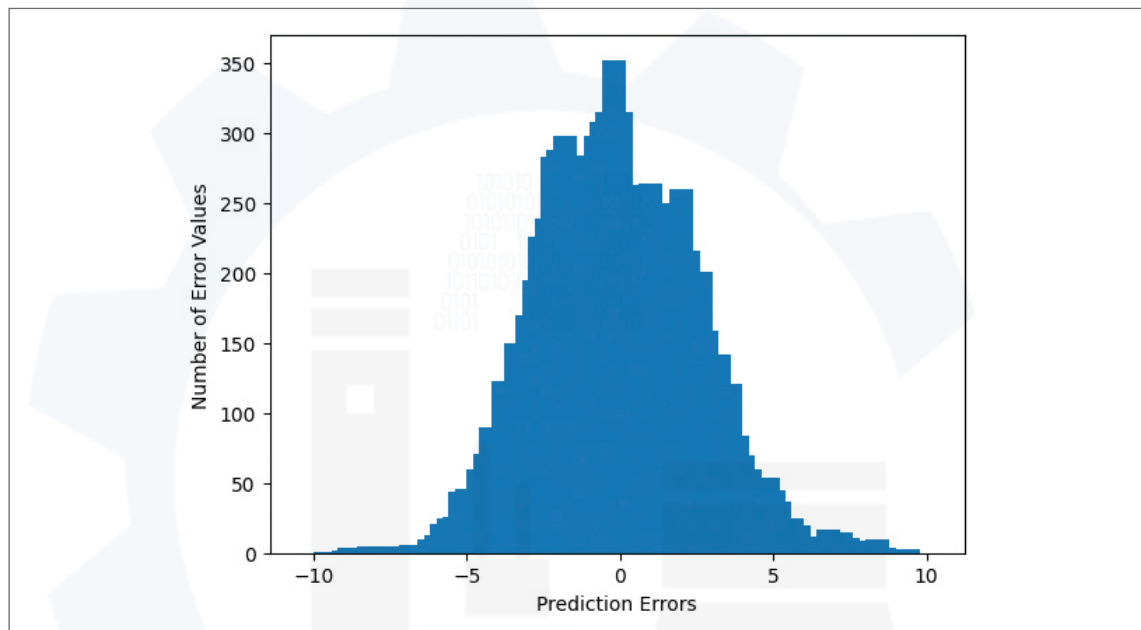
- 검증데이터에 대한 예측과 실제 값의 차이 값의 히스토그램을 그래프로 나타낸다. 대략적으로 에러 값 분포는 정규분포에 가까우며, 대부분의 예측 값은 실제 값과 차이가 5와 +5 사이에 들어감을 알 수 있다.

```

t = errors
print(t[:5])
min_ = -10.0
max_ = 10.0
bins_ = 100
hist = torch.histc(t, bins=bins_, min=min_, max=max_)
factor = (max_ - min_) / bins_
bins = bins_
x = []
for i in range(bins):
    x.append(min_ + factor * float(i))

plt.bar(x, hist, align='center')
plt.xlabel('Prediction Errors')
plt.ylabel('Number of Error Values')

```



[코드 92] 검증 데이터 예측의 에러 분포 히스토그램

[단계 ⑥] 테스트셋에 대한 계산 수행

⑥-1. 정상 테스트 데이터에 대한 테스트

- 정상 테스트 데이터 'WeldingTest_01_OK.csv'를 불러온다.

```

signal_file = './data/preprocessed/test/WeldingTest_01_OK.csv'
df_test_1 = pd.read_csv(signal_file)
data = df_test_1
data.head()

```

	series	time_idx	value
0	0	0	1662.0
1	0	1	1688.0
2	0	2	1669.0
3	0	3	1693.0
4	0	4	1673.0

[코드 93] 정상 테스트 데이터 불러오기

- 학습 데이터의 경우와 마찬가지로 테스트 데이터에 대하여 입력의 길이는 20(“max_encoder_length = 20”)으로 정의되고 예측 구간의 길이는 10(“max_prediction_length = 10”)으로 정의된다. 다음으로 시계열 테스트 데이터 클래스 testing을 정의한다.

```
# create dataset and dataloaders
max_encoder_length = 20
max_prediction_length = 10
testing_cutoff = 100
context_length = max_encoder_length
prediction_length = max_prediction_length
testing = TimeSeriesDataSet(
    data[lambdax: x.time_idx <= testing_cutoff],
    time_idx="time_idx",
    target="value",
    #categorical_encoders={"series": NaLabelEncoder().fit(data.series)},
    group_ids=["series"],
    # only unknown variable is "value" - and N-HiTS can also not take any additional variables
    time_varying_unknown_reals=["value"],
    max_encoder_length=context_length,
    max_prediction_length=prediction_length,
)
test_data = TimeSeriesDataSet.from_dataset(testing, data, min_prediction_idx=testing_cutoff + 1)
batch_size = 128
train_dataloader = testing.to_dataloader(train=True, batch_size=batch_size, num_workers=0)
test_dataloader = test_data.to_dataloader(train=False, batch_size=batch_size, num_workers=0)
```

[코드 94] (N-HiTS 입력) 테스트 시계열 데이터 준비

- 이미 학습된 N-HiTS 모델을 불러와서 변수 best_model에 저장한다.

```
best_model_path = trainer.checkpoint_callback.best_model_path
print("best_model_path: ", best_model_path)
best_model = NHiTS.load_from_checkpoint(best_model_path)
```

```
best_model_path: C:\Users\SAMSUNG\Desktop\Battery_AD\04_전자부품(배터리팩 제조공정) 예지보전_221107\전자부품(배터리팩 제조공정) 예지보전_221031\03_소스\lightning_logs\version_0\checkpoints\epoch=29-step=900.ckpt
```

[코드 95] N-HiTS 네트워크 모델 불러오기

- N-HiTS 모델 best_model을 이용하여 테스트 데이터에 대한 예측을 수행하며 실제 값과의 절대값 차이의 평균값을 구한다. 그 값은 약 2.268로 나온다. 이웃하는 시점 사이의 RealPower 세팅 비율이 변할 때에는 82%→ 83% 등과 같이 1%의 변화를 가지므로 그 차이값은 최대 Power 2000의 1% 즉 2000*1% = 20 가 되는데 2.268은 20 보다 훨씬 작으므로 예측의 정확도는 만족스럽다고 할 수 있다.

```
actuals = torch.cat([y[0] for x, y in iter(test_dataloader)])
predictions = best_model.predict(test_dataloader)
(actuals-predictions).abs().mean()
```

```
tensor(2.2668)
```

[코드 96] N-HiTS 테스트셋에 대한 예측

- 테스트 데이터에 대한 예측값과 실제값을 비교한다.

```
print(len(actuals))
for i in range(len(actuals)):
    print(i= ' ', i, 'actuals=', actuals[i][0], 'pred= ', predictions[i][0])
```

```
1840
i= 0 actuals= tensor(1682.) pred= tensor(1684.1910)
i= 1 actuals= tensor(1699.) pred= tensor(1699.3380)
i= 2 actuals= tensor(1680.) pred= tensor(1683.2281)
i= 3 actuals= tensor(1703.) pred= tensor(1702.3834)
i= 4 actuals= tensor(1683.) pred= tensor(1686.2163)
i= 5 actuals= tensor(1704.) pred= tensor(1703.4186)
i= 6 actuals= tensor(1684.) pred= tensor(1686.8202)
i= 7 actuals= tensor(1613.5000) pred= tensor(1617.7594)
i= 8 actuals= tensor(1613.5000) pred= tensor(1617.5394)
i= 9 actuals= tensor(1613.5000) pred= tensor(1615.1919)
i= 10 actuals= tensor(1613.5000) pred= tensor(1614.6580)
i= 11 actuals= tensor(1613.5000) pred= tensor(1612.2158)
i= 12 actuals= tensor(1613.5000) pred= tensor(1613.3572)
i= 13 actuals= tensor(1613.5000) pred= tensor(1612.4365)
i= 14 actuals= tensor(1613.5000) pred= tensor(1613.0697)
i= 15 actuals= tensor(1613.5000) pred= tensor(1614.8181)
i= 16 actuals= tensor(1667.) pred= tensor(1668.4346)
i= 17 actuals= tensor(1693.) pred= tensor(1691.3855)
```

[코드 97] 테스트 데이터에 대한 예측과 실제값 비교

- N-HiTS에 의한 예측값과 실제 데이터를 비교하여 그래픽으로 나타낸다. 여기에서 파랑색은 실제값 그리고 주황색은 예측값을 나타낸다. 두 가지 그래프가 큰 차이가 없음을 확인할 수 있다.

```

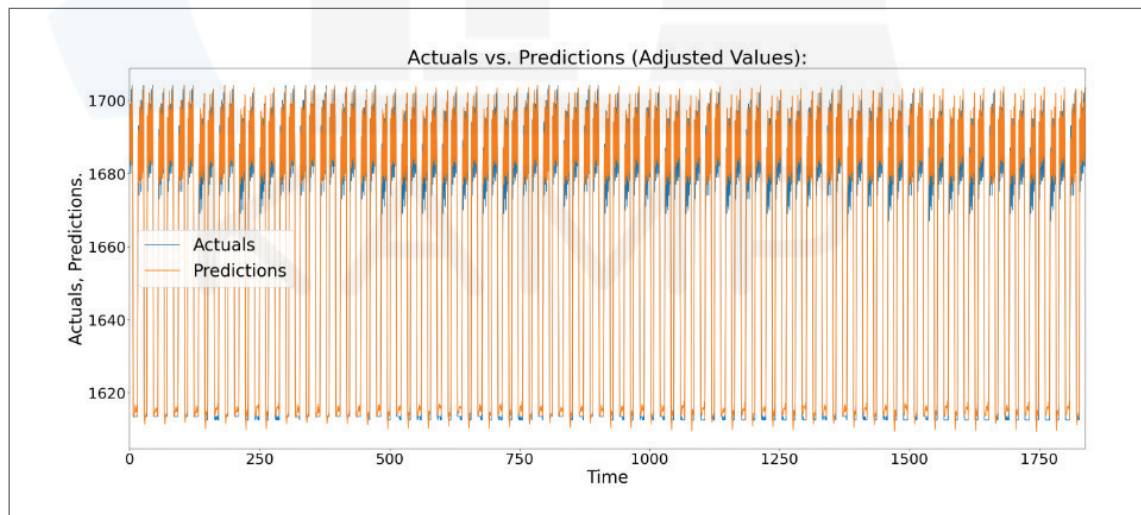
register_matplotlib_converters()
np.random.seed(0)
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = len(actuals)
time = range(max_len)
print("max_len:", max_len)
X_signal = []
X_pred = []
for kk in range(max_len):
    X_signal.append(actuals[kk][0])
    X_pred.append(predictions[kk][0])
for kk in range(10):
    print(actuals[kk][0])
X_signal = np.array(X_signal)
X_pred = np.array(X_pred)
plt.plot(time, X_signal[:,], label='Actuals')
plt.plot(time, X_pred[:,], label='Predictions')
plt.legend(loc=0, fontsize=30)
plt.title('Actuals vs. Predictions (Adjusted Values): ', size=34)
plt.ylabel('Actuals, Predictions.', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.show()

```

```

max_len: 1840
tensor(1682.)
tensor(1699.)
tensor(1680.)
tensor(1703.)
tensor(1683.)
tensor(1704.)

```



[코드 98] 테스트셋에 대한 예측과 실제값 비교

- 예측값과 실제 데이터 사이의 차이 값 즉 에러를 그래프로 나타낸다. 대부분의 에러 값은 +5와 5 사이에 들어 있어서 위에서 계산한 1% 변화값인 20 보다 충분히 작음을 알 수 있다.

```

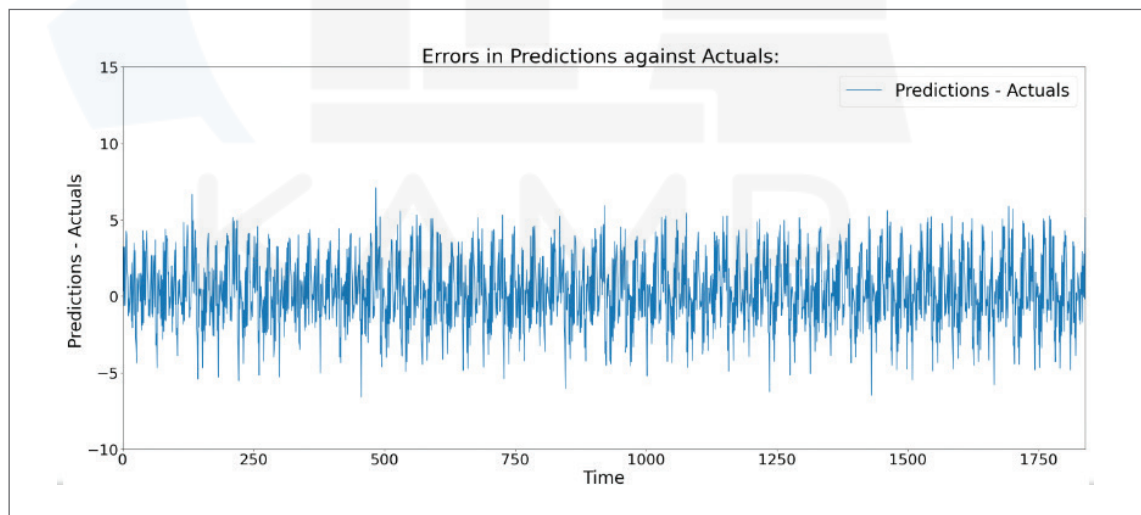
register_matplotlib_converters()
np.random.seed(0)
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = len(actuals)
time = range(max_len)
print("max_len:", max_len)
X_signal = []
X_pred = []
for kk in range(max_len):
    X_signal.append(actuals[kk][0])
    X_pred.append(predictions[kk][0])
for kk in range(10):
    print(actuals[kk][0])
X_signal = np.array(X_signal)
X_pred = np.array(X_pred)
X_errors = X_pred - X_signal
plt.plot(time, X_pred[:] - X_signal[:], label='Predictions - Actuals')
plt.legend(loc=0, fontsize=30)
plt.title('Errors in Predictions against Actuals: ', size=34)
plt.ylabel('Predictions - Actuals ', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.ylim(-10., 15)
plt.show()

```

```

max_len: 1840
tensor(1682.)
tensor(1699.)
tensor(1680.)
tensor(1703.)
tensor(1683.)

```



[코드 99] 테스트셋에 대한 예측과 실제값 사이의 차이값

- 에러값의 평균값과 표준편차로 구한 Z-score를 그래프로 나타내어 threshold=4.0를 써서 이상 여부를 확인한다. 여기에서 모든 시점에 대해 데이터 값이 한도 내에 들어와서 정상임을 알 수 있다.



```

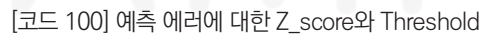
threshold = 4.0
final_scores1 = X_errors
pred_length = len(final_scores1)
pred_score1 = final_scores1
avg = np.average(pred_score1)
sigma = math.sqrt(sum((pred_score1-avg) * (pred_score1-avg)) / len(pred_score1))
Z_score = (pred_score1-avg)/sigma
pred_bin1 = pred_length*[0]
thre = pred_length*threshold
thre_minus = pred_length*[-threshold]
thre = np.array(thre)
thre_minus = np.array(thre_minus)
for i in range(pred_length):

    if (Z_score[i] > threshold or Z_score[i] < -threshold):
        pred_bin1[i] = 1
    else:
        pred_bin1[i] = 0
print("pred_bin1: ")
print(pred_bin1)
pred1 = np.array(pred_bin1)
final_scores1 = np.array(final_scores1)
X_signal0 = np.array(X_errors)

Z_score1 = np.array(Z_score)
register_matplotlib_converters()
np.random.seed(0)
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = len(actuals)
time = range(max_len)
print("max_len:", max_len)
plt.plot(time, thre, label='Z Threshold=4.0')
plt.plot(time, thre_minus, label='Z Threshold=-4.0')
plt.plot(time, Z_score[:,], label='Z_scores')
plt.legend(loc=0, fontsize=30)
plt.title('Z_scores of Errors : ', size=34)
plt.ylabel('Z_score, threshold', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.ylim(-5.0, 5.0)
plt.show()

```

max_len: 1840



- 이상이 들어 있는 (비정상적인) 테스트 데이터 “WeldingTest_03_NG.csv”를 불러온다. 이 데이터에서의 이상값은 여러 개의 특정한 고립된 시점에서 “RealPower” 값이 정상값에서 크게 벗어나는 데이터이다.

```

signal_file = './data/preprocessed/test/WeldingTest_03_NG.csv'
anomaly_file = './data/preprocessed/test/WeldingTest_03_NG_Label.csv'
df_test_0 = pd.read_csv(signal_file)
data = df_test_0
data.head()

```

	series	time_idx	value
0	0	0	1660.0
1	0	1	1685.0
2	0	2	1666.0
3	0	3	1690.0
4	0	4	1670.0

[코드 101] 비정상 테스트 데이터 불러오기

- 테스트 데이터 “WeldingTest_03_NG”에 대하여 테스트 데이터 시계열 클래스를 정의하고 최대 입력값 길이 변수 max_encoder_length=20 그리고 최대 예측값 길이 변수 max_prediction_length =10를 정의한다.

```

# create dataset and dataloaders
max_encoder_length = 20
max_prediction_length = 10
testing_cutoff = 50
context_length = max_encoder_length
prediction_length = max_prediction_length
testing = TimeSeriesDataSet(
    data[lambdax: x.time_idx <= testing_cutoff],
    time_idx="time_idx",
    target="value",
    #categorical_encoders={"series": NaNLabelEncoder().fit(data.series)},
    group_ids=["series"],
    # only unknown variable is "value" - and N-HiTS can also not take any additional variables
    time_varying_unknown_reals=["value"],
    max_encoder_length=context_length,
    max_prediction_length=prediction_length,
)
test_data = TimeSeriesDataSet.from_dataset(testing, data, min_prediction_idx=testing_cutoff + 1)
batch_size = 128
train_dataloader = testing.to_dataloader(train=True, batch_size=batch_size, num_workers=0)
test_dataloader = test_data.to_dataloader(train=False, batch_size=batch_size, num_workers=0)

```

[코드 102] 테스트 데이터 시계열 클래스 정의

- 이미 학습된 네트워크 모델을 불러와서 테스트 데이터 예측계산을 수행하고 실제 값과의 절대값 차이의 평균을 계산한다. 그 값은 여기서는 4.2360으로 나온다. 학습 모델이 달라지면 이 값도 달라질 수 있다.

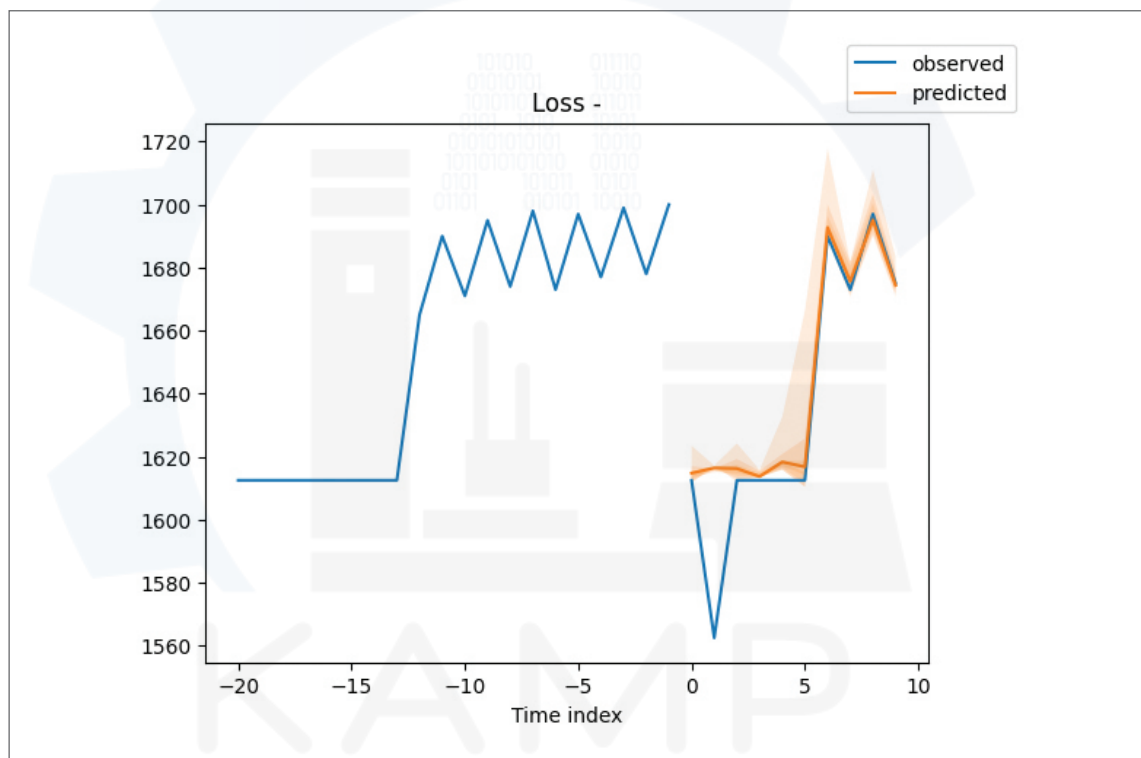
```
best_model_path = trainer.checkpoint_callback.best_model_path
best_model = NHITS.load_from_checkpoint(best_model_path)
actuals = torch.cat([y[0] for x, y in iter(test_dataloader)])
predictions = best_model.predict(test_dataloader)
(actuals-predictions).abs().mean()
```

```
tensor(4.2360)
```

[코드 103] N-HITS 네트워크 모형으로 테스트 데이터 예측하기

- 테스트 데이터의 몇몇 샘플 구간에서의 예측값과 실제값을 비교하고 그래프로 출력한다. 파란선은 테스트 데이터의 샘플 구간에서의 실제값을 표시한 것이고, 주황선은 학습된 네트워크 모형의 테스트 데이터에 대한 예측값을 표시한 것이다.

```
raw_predictions, x = best_model.predict(test_dataloader, mode="raw", return_x=True)
for idx in range(20):
    best_model.plot_prediction(x, raw_predictions, idx=idx, add_loss_to_title=True)
```



[코드 104] 테스트 데이터 예측과 실제값 비교의 예

- 테스트 데이터 예측값과 실제값을 확인한다.

```
print(len(actuals))
for i in range(len(actuals)):
    print('i= ', i, 'actuals=', actuals[i][0], 'pred= ', predictions[i][0])
```

```
993
i= 0 actuals= tensor(1612.5000) pred= tensor(1616.9672)
i= 1 actuals= tensor(1562.5000) pred= tensor(1614.4955)
i= 2 actuals= tensor(1612.5000) pred= tensor(1592.3621)
i= 3 actuals= tensor(1612.5000) pred= tensor(1617.7037)
i= 4 actuals= tensor(1612.5000) pred= tensor(1617.1815)
i= 5 actuals= tensor(1612.5000) pred= tensor(1618.7329)
i= 6 actuals= tensor(1690.) pred= tensor(1690.3540)
i= 7 actuals= tensor(1673.) pred= tensor(1673.3794)
i= 8 actuals= tensor(1697.) pred= tensor(1690.5277)
i= 9 actuals= tensor(1675.) pred= tensor(1681.9978)
i= 10 actuals= tensor(1698.) pred= tensor(1703.2479)
i= 11 actuals= tensor(1678.) pred= tensor(1682.8512)
i= 12 actuals= tensor(1697.) pred= tensor(1691.2490)
i= 13 actuals= tensor(1677.) pred= tensor(1684.6686)
i= 14 actuals= tensor(1699.) pred= tensor(1694.8057)
i= 15 actuals= tensor(1678.) pred= tensor(1685.8778)
i= 16 actuals= tensor(1700.) pred= tensor(1692.1100)
```

[코드 105] 테스트 데이터 예측과 실제값 비교

- 테스트 데이터 예측값과 실제값을 비교하고 그래프로 출력한다. 그래프에서 보듯이 특정한 위치에서 실제값이 스파이크 형태로 예측과 크게 벗어남을 알 수 있다.

```

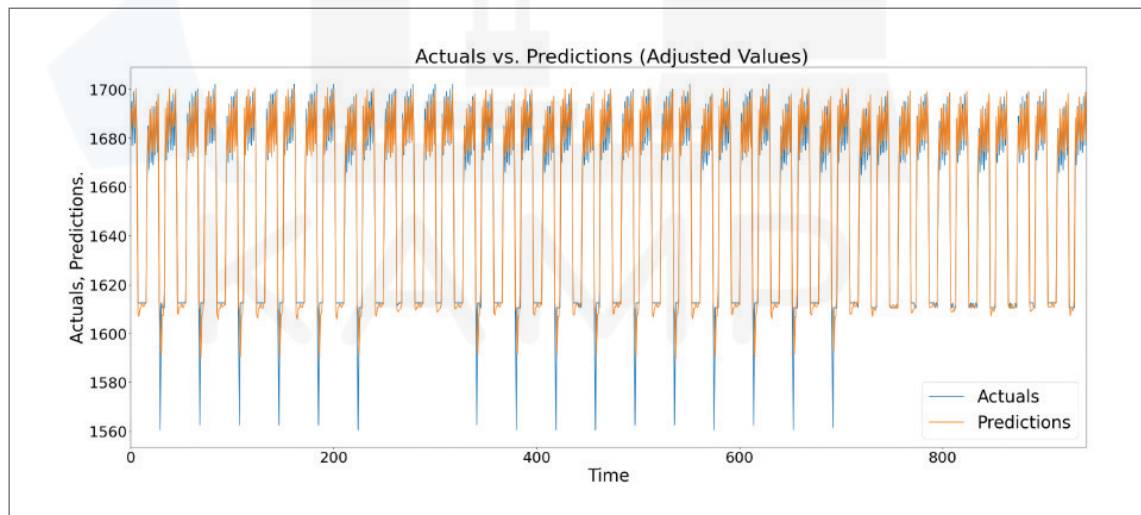
register_matplotlib_converters()
np.random.seed(0)
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = len(actuals)
time = range(max_len)
print("max_len:", max_len)
X_signal = []
X_pred = []
for kk in range(max_len):
    X_signal.append(actuals[kk][0])
    X_pred.append(predictions[kk][0])
for kk in range(10):
    print(actuals[kk][0])
X_signal = np.array(X_signal)
X_pred = np.array(X_pred)
plt.plot(time, X_signal[:,], label='Actuals')
plt.plot(time, X_pred[:,], label='Predictions')
plt.legend(loc=0, fontsize=30)
plt.title('Actuals vs. Predictions (Adjusted Values)', size=34)
plt.ylabel('Actuals, Predictions.', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.show()

```

```

max_len: 993
tensor(1612.5000)
tensor(1562.5000)
tensor(1612.5000)
tensor(1612.5000)
tensor(1612.5000)

```



[코드 106] 테스트셋에 대한 예측값과 실제값 비교

- “WeldingTest_03_NG” 테스트 데이터에 대한 예측값과 실제값 사이의 차이값을 그래프로 출력한다. 예상대로 스파이크 형태의 비정상적인 시점이 나타남을 알 수 있다.

```

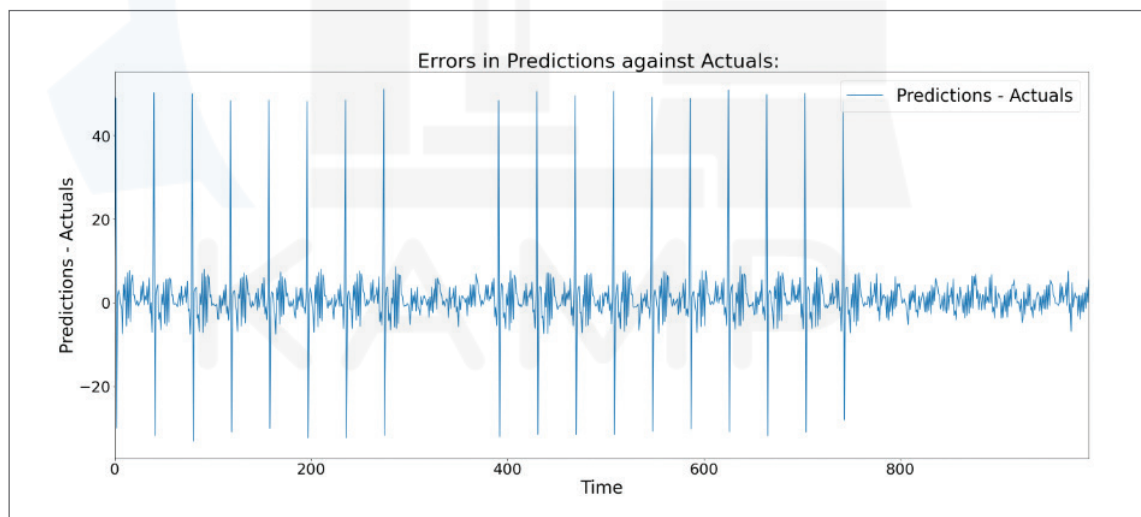
register_matplotlib_converters()
np.random.seed(0)
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = len(actuals)
time = range(max_len)
print("max_len:", max_len)
X_signal = []
X_pred = []
for kk in range(max_len):
    X_signal.append(actuals[kk][0])
    X_pred.append(predictions[kk][0])
for kk in range(10):
    print(actuals[kk][0])
X_signal = np.array(X_signal)
X_pred = np.array(X_pred)
X_errors = X_pred - X_signal
plt.plot(time, X_pred[:] - X_signal[:], label='Predictions - Actuals')
plt.legend(loc=0, fontsize=30)
plt.title('Errors in Predictions against Actuals: ', size=34)
plt.ylabel('Predictions - Actuals', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.show()

```

```

max_len: 993
tensor(1612.5000)
tensor(1562.5000)
tensor(1612.5000)
tensor(1612.5000)

```



[코드 107] 테스트셋에 대한 예측과 실제값 사이의 차이값

- 테스트 데이터에 대한 예측으로부터 얻은 에러 값의 평균과 표준편차로부터 구한 Z-score에 threshold=4.0을 적용하여 이상 구간을 찾아낸다. 그리고 실제 이상 구간과 비교한다.



```

threshold = 4.0
# Iterative
final_scores1 = X_errors
nshift = 12
pred_length = len(final_scores1) + nshift
pred_score1 = final_scores1
avg = np.average(pred_score1)
sigma = math.sqrt(sum((pred_score1-avg) * (pred_score1-avg)) / len(pred_score1))
Z_score = (pred_score1-avg) / sigma
pred_bin1 = pred_length*[0]
thre = pred_length*threshold
thre_minus = pred_length*[-threshold]
thre = np.array(thre)
thre_minus = np.array(thre_minus)
for i in range(len(final_scores1)):
    if (Z_score[i] > threshold or Z_score[i] < -threshold):
        pred_bin1[i+nshift] = 1
print("pred_bin1: ")
print(pred_bin1)
pred1 = np.array(pred_bin1)

known_anomalies = pd.read_csv(anomaly_file)
print("anomaly_file: ", anomaly_file)
print("known_anomalies from 400 to 600 :")
anom_labels = known_anomalies['label']
true = anom_labels
for i in range(400,600):
    print("i =", i, anom_labels[i])
final_scores1 = np.array(final_scores1)
X_signal0 = np.array(X_errors)
gt = np.array(true)
Z_score1 = np.array(Z_score)
pred = pred1
# List of Anomaly sequences
anomalies = []
anomaly_gt = []
anomaly_pred = []
length_anom = len(pred)
print("length_anom: ", length_anom)
anom_pred_init = 0 # inside a sequence or not
anom_gt_init = 0 #

```

```

for k in range(length_anom):
    if gt[k] == 1 :
        if anom_gt_init == 0 : # Now beginning of an anomalous sequence.
            anom_gt_begin = k
            anom_gt_init = 1
        else :
            anom_gt_end = k
            if k == length_anom-1:
                anomaly_gt.append((anom_gt_begin,anom_gt_end))
    if gt[k] == 0 and anom_gt_init == 1: # End of anom. sequence
        anom_gt_end = k - 1
        anomaly_gt.append((anom_gt_begin,anom_gt_end))
        anom_gt_init = 0
    if pred[k] == 1 :
        if anom_pred_init == 0 : # Now beginning of an anomalous sequence.
            anom_pred_begin = k
            anom_pred_init = 1
        else:
            anom_pred_end = k
            if k == length_anom - 1:
                anomaly_pred.append((anom_pred_begin,anom_pred_end))
    if pred[k] == 0 and anom_pred_init == 1: # End of anom. sequence
        anom_pred_end = k - 1
        anomaly_pred.append((anom_pred_begin,anom_pred_end))
        anom_pred_init = 0
anomalies = [anomaly_gt, anomaly_pred]
print("gt:")
print(anomaly_gt)
print("pred:")
print(anomaly_pred)

```

```

gt:
[(13, 13), (52, 52), (91, 91), (130, 130), (169, 169), (208, 208), (247, 247), (286, 286), (325, 325), (4
42, 442), (481, 481), (520, 520), (559, 559), (598, 598), (637, 637), (676, 676), (715, 715), (754, 754),
(793, 793)]
pred:
[(13, 13), (52, 52), (91, 92), (130, 130), (169, 169), (208, 208), (247, 247), (286, 286), (403, 403), (4
42, 442), (481, 481), (520, 520), (559, 559), (598, 598), (637, 637), (676, 676), (715, 715), (754, 754)]

```

[코드 108] 테스트 데이터에 대한 Anomaly 예측

- 테스트 데이터에 대한 예측의 정확도(Accuracy), 정밀도(Precision), 재현율(Recall), 그리고 F-score를 계산한다. 계산 결과 정확도=0.9960, 정밀도=0.8947, 재현율=0.8947, 그리고 F-score=0.8947를 얻는다.

```
# Accuracy, Precision, Recall, and F score
# Iterative
true0 = anom_labels
print("true0 :")
print(true0.tail(40))
final_scores = final_scores1
pred_bin = pred_bin1
true = []
true = true0[: pred_length]
pred = np.array(pred_bin)
gt = np.array(true)
print("pred: ", pred.shape)
print("gt: ", gt.shape)
for j in range(1, len(true)):
    print(" %d %d %d " % (j, pred[j], gt[j]))
accuracy = accuracy_score(gt, pred)
precision, recall, f_score, support = precision_recall_fscore_support(gt, pred,
                                                                    average="binary")
print(
    "Accuracy : {:.4f}, Precision : {:.4f}, Recall : {:.4f}, F-score : {:.4f}".format(
        accuracy, precision,
        recall, f_score))
```

```
999 0 0
1000 0 0
1001 0 0
1002 0 0
1003 0 0
1004 0 0
Accuracy : 0.9960, Precision : 0.8947, Recall : 0.8947, F-score : 0.8947
```

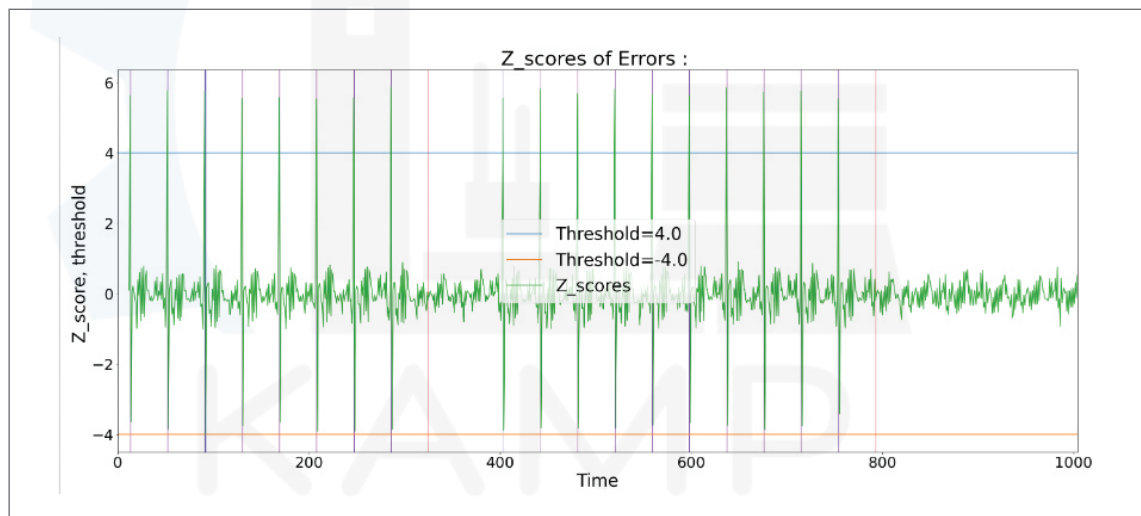
[코드 109] 테스트 데이터에 대한 예측의 F-score 기타

- 예측 에러의 Z-score와 threshold 값, 그리고 예측 시점 및 실제 이상값 시점의 비교를 그래프로 나타낸다.

```

register_matplotlib_converters()
np.random.seed(0)
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = len(pred_bin1)
time = range(max_len)
print("max_len:", max_len)
print("length of thre", len(thre))
plt.plot(time, thre, label='Threshold=4.0')
plt.plot(time, thre_minus, label='Threshold=-4.0')
plt.plot(time[12:], Z_score[:,], label='Z_scores')
plt.legend(loc=0, fontsize=30)
colors = ['red'] + ['blue'] * (len(anomalies)-1)
for i, anomaly in enumerate(anomalies):
    if not isinstance(anomaly, list):
        anomaly = list(anomaly[['start', 'end']].itertuples(index=False))
    for _, anom in enumerate(anomaly):
        t1 = anom[0]
        t2 = anom[1]
        plt.axvspan(t1, t2, color=colors[i], alpha=0.5)
plt.title('Z_scores of Errors : ', size=34)
plt.ylabel('Z_score, threshold', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.show()

```



[코드 110] 예측 에러에 대한 Z_score와 Threshold

⑥-3. 비정상 테스트 데이터 “WeldingTest_04_NG.csv”에 대한 테스트

- “WeldingTest_04_NG.csv” 데이터에서는 일정한 시점 이후 모든 값이 연속적으로 정상에서 벗어나는 값을 나타내고 있다. 이를 분석하기 위해서는 N-HiTS 방식보다는 통계적인 분석이 더 적합해 보인다.
- 따라서 먼저 학습 데이터 “Training_Data.csv”의 “RealPower” 값의 통계 분포 특성을 알아내기 위한 계산을 수행한다. 여기서는 정상적인 학습데이터의 경우에



“RealPower” 데이터의 값이 (1650~1750) 구역과 (650~700) 구역의 두 가지 구역으로 갈라져서 분포하고 있으므로 각각의 구역에 대하여 정규분포로 가정하고 그 평균값과 표준편차를 따로 구하여 저장한다. 이 값들은 다음 단계에서 테스트 데이터 “WeldingTest_04_NG.csv”의 이상값을 탐지할 때에 이용한다.

```
# 학습 데이터의 통계분포 분석을 위해 학습데이터 불러오기
signal_file = './data/preprocessed/train/Training_Data_Real_Power.csv'
df_train = pd.read_csv(signal_file)
df_train.head()
```

RealPower	
0	1688.0
1	1713.0
2	1695.0
3	1717.0
4	1698.0

[코드 111] 학습데이터 불러오기

- 학습 데이터의 통계 분포를 계산한다. 즉 두 구역의 평균값과 표준편차를 각각 구한다.



```

df0=df_train
cols0 = df0.columns
len_grad=20
max_len0 =len(df0)-len_grad
num_cols0 =len(cols0)
total_num = max_len0 * num_cols0
sum_a =0.0
sum_sq_a =0.0
sum_cube_a =0.0
sum_quad_a =0.0
sum_b =0.0
sum_sq_b =0.0
sum_cube_b =0.0
sum_quad_b =0.0
a_list = df0["RealPower"]
list_a = []
list_b = []
boundary =1200.0
for i in range(max_len0):
    power = a_list[i]
    if power < boundary :
        list_a.append(power)
    if power >= boundary :
        list_b.append(power)
list_a = np.array(list_a)
list_b = np.array(list_b)
Pavg_a = sum(list_a)/len(list_a)
Pavg_b = sum(list_b)/len(list_b)
Psigma_a = np.std(list_a)
Psigma_b = np.std(list_b)
print("Pavg_a = ", Pavg_a)
print("Pavg_b = ", Pavg_b)
print("Psigma_a = ", Psigma_a)
print("Psigma_b = ", Psigma_b)

```

```

Pavg_a = 687.1890675856787
Pavg_b = 1700.379827324001
Psigma_a = 19.04951273978001
Psigma_b = 13.730347765487352

```

[코드 112] 학습 데이터 통계 분포

- 테스트 데이터 “WeldingTest_04_NG.csv”를 불러온다.

```

signal_file = './data/preprocessed/test/WeldingTest_04_NG.csv' # test file
anomaly_file = './data/preprocessed/test/WeldingTest_04_NG_Label.csv'
df4 = pd.read_csv(signal_file)
df4.head(5)

```

	RealPower
0	1713.0
1	1695.0
2	1718.0
3	1698.0
4	1721.0

[코드 113] 테스트 데이터 불러오기



- 학습 데이터의 두 가지 성분 분포의 평균과 표준편차를 기준으로 하여 테스트 데이터의 두 가지 Z-score를 구하고 이중 작은 값을 threshold=4 와 비교하여 이상값을 예측한다.

```
df_new = df4
df0 = df_new
max_len0 = len(df0) - 20
num_cols = len(df0.columns[0])
threshold = 4.0
pred_bin = max_len0 * [0]
Z_min_arr = []
Pdiff_arr = []
for i in range(max_len0):
    power = df0.iloc[i, 0]
    Z_score_a = abs(power - Pavg_a) / Psigma_a
    Z_score_b = abs(power - Pavg_b) / Psigma_b

    if Z_score_a > Z_score_b:
        Z_score_min = Z_score_b
        Pdiff_min = (power - Pavg_b) / 100.0
    if Z_score_a <= Z_score_b:
        Z_score_min = Z_score_a
        Pdiff_min = (power - Pavg_a) / 100.0

    Z_min_arr.append(Z_score_min)
    Pdiff_arr.append(Pdiff_min)
    if Z_score_min >= threshold:
        pred_bin[i] = 1
Z_min_arr = np.array(Z_min_arr)
Pdiff_arr = np.array(Pdiff_arr)
pred = np.array(pred_bin)
print("pred: ")
for i in range(max_len0):
    print(i, pred[i], Z_min_arr[i], Pdiff_arr[i])
```

```
pred:
0 0 0.919144430392434 0.126201726759989
1 0 0.3918201793492697 -0.053798273240010984
2 0 1.2833012664317962 0.17620172675998902
3 0 0.1733260777256524 -0.023798273240010985
4 0 1.5017953680554135 0.20620172675998902
5 0 0.3918201793492697 -0.053798273240010984
6 0 1.5017953680554135 0.20620172675998902
7 0 0.10049471051777997 -0.013798273240010985
8 0 1.6474581024711583 0.226201726759989
```

[코드 114] 테스트 데이터 통계분석 및 이상값 예측

- 테스트 이상값 데이터 파일에서 이상값 label을 불러온다.



```
# load known_anomalies
known_anomalies = pd.read_csv(anomaly_file)
print("anomaly_file: ", anomaly_file)
print("known_anomalies from 50 to 200 :")
for i in range(50,200):
    print("i =", i, known_anomalies['label'][i])

anomaly_file: ./data/preprocessed/test/WeldingTest_04_NG_Label.csv
known_anomalies from 50 to 200 :
i = 50 0
i = 51 0
i = 52 0
i = 53 0
i = 54 0
```

[코드 115] 이상값 Label 불러오기

- 실제 이상값 리스트를 변수 anom_labels에 저장한다.

```
print("known_anomalies: ")
print(known_anomalies['label'][:10])
anom_labels = known_anomalies['label']
print(anom_labels.tail(30))
```

```
343    1
344    1
345    1
346    1
347    1
348    1
349    1
350    1
Name: label, dtype: int64
```

[코드 116] 실제 이상값 리스트 anom_labels 정의하기

- 정확도, 정밀도, 재현율, 그리고 F 스코어를 계산한다. 이 경우에는 정확도=정밀도=재현율 = F-score = 1.0로 완벽하게 예측함을 알 수 있다.



```
# Accuracy, Precision, Recall, and F score
# Iterative
true0 = anom_labels
print("true0 :")
print(true0.tail(10))
pred_length = len(Z_min_arr)
true = []
true = true0[: pred_length]
gt = np.array(true)
print("pred: ", pred.shape)
print("gt: ", gt.shape)
thre = []
for j in range(1, len(true)):
    print( "%d %d %d " % (j, pred[j], gt[j]))
accuracy = accuracy_score(gt, pred)
precision, recall, f_score, support = precision_recall_fscore_support(gt, pred, average='binary')
print(
    "Accuracy : {:.4f}, Precision : {:.4f}, Recall : {:.4f}, F-score : {:.4f}".format(
        accuracy, precision,
        recall, f_score))
```

```
323 1 1
324 1 1
325 1 1
326 1 1
327 1 1
328 1 1
329 1 1
330 1 1
Accuracy : 1.0000, Precision : 1.0000, Recall : 1.0000, F-score : 1.0000
```

[코드 117] 정확도, 정밀도, 재현율 및 F 스코어

- 실제 이상값의 구간과 예측값의 구간의 sequence를 구한다. 여기에서 gt는 실제 이상값(ground truth)이고 pred는 예측값이다.



```
# List of Anomaly sequences
anomalies = []
anomaly_gt = []
anomaly_pred = []
length_anom = len(pred)
print("length_anom: ", length_anom)
anom_pred_init = 0 # inside a sequence or not
anom_gt_init = 0 #
for k in range(length_anom):
    if gt[k] == 1 :
        if anom_gt_init == 0 : # Now beginning of an anomalous sequence.
            anom_gt_begin = k
            anom_gt_init = 1
        else :
            anom_gt_end = k
            if k == length_anom-1:
                anomaly_gt.append((anom_gt_begin,anom_gt_end))
    if gt[k] == 0 and anom_gt_init == 1: # End of anom. sequence
        anom_gt_end = k -1
        anomaly_gt.append((anom_gt_begin,anom_gt_end))
        anom_gt_init = 0
    if pred[k] == 1 :
        if anom_pred_init == 0 : # Now beginning of an anomalous sequence.
            anom_pred_begin = k
            anom_pred_init = 1
        else :
            anom_pred_end = k
            if k == length_anom -1:
                anomaly_pred.append((anom_pred_begin,anom_pred_end))
    if pred[k] == 0 and anom_pred_init == 1: # End of anom. sequence
        anom_pred_end = k -1
        anomaly_pred.append((anom_pred_begin,anom_pred_end))
        anom_pred_init = 0
anomalies = [anomaly_gt, anomaly_pred]
print("gt:")
print(anomaly_gt)
print("pred:")
print(anomaly_pred)
```

```
length_anom: 331
gt:
[(78, 330)]
pred:
[(78, 330)]
```

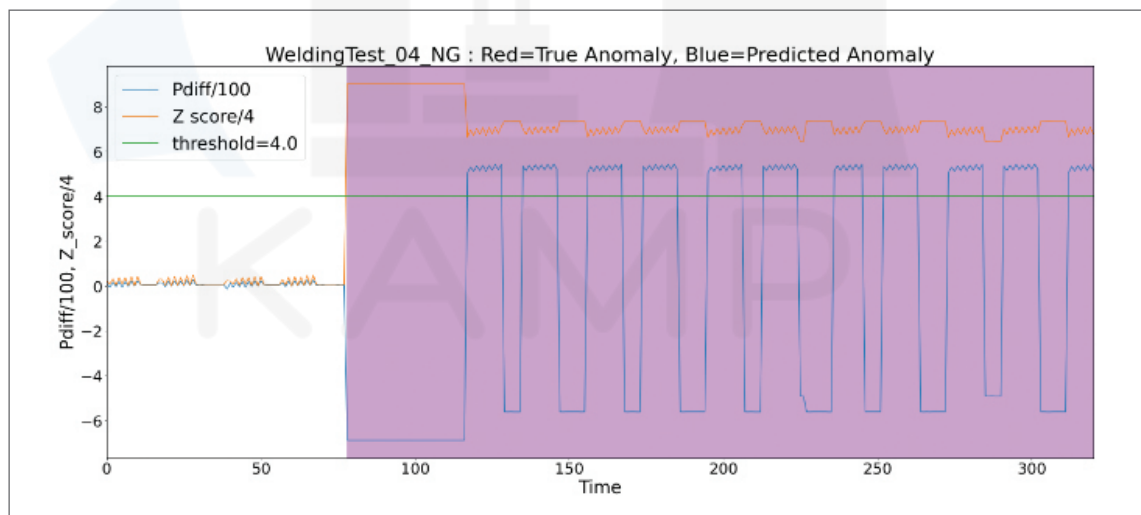
[코드 118] 이상값의 구간 Sequence

- Z score 및 이상값 예측 구간 그리고 실제 이상 구간을 그래프로 나타낸다. 여기에서 실제 이상 구간은 붉은 색으로 예측 구간은 파랑색으로 나타내었는데 두 구간이 완전히 겹쳐져 있음을 알 수 있다. 특히 이상구간의 Z-score가 threshold=4 보다 훨씬 큰 값을 알 수 있다.

```

register_matplotlib_converters()
np.random.seed(0)
if not isinstance(anomalies, list):
    anomalies = [anomalies]
fig = plt.figure(figsize=(30, 12))
ax = fig.add_subplot(111)
max_len = length_anom-10
time = range(max_len)
Z_score2 = 0.25*Z_min_arr[:max_len]
thre = max_len*[threshold]
thre=np.array(thre)
X_signal_2 = np.array(Pdiff_arr[:max_len])
plt.plot(time, X_signal_2[:,], label='Pdiff/100')
plt.plot(time, Z_score2, label='Z score/4')
plt.plot(time, thre, label='threshold=4.0')
plt.legend(loc=0, fontsize=30)
print("length_anom, max_len:", length_anom, max_len)
colors = ['red'] + ['blue'] * (len(anomalies)-1)
for i, anomaly in enumerate(anomalies):
    if not isinstance(anomaly, list):
        anomaly = list(anomaly[['start', 'end']].itertuples(index=False))
    for _, anom in enumerate(anomaly):
        t1 = anom[0]
        t2 = anom[1]
        plt.axvspan(t1, t2, color=colors[i], alpha=0.2)
plt.title('WeldingTest_04_NG : Red=True Anomaly, Blue=Predicted Anomaly', size=34)
plt.ylabel('Pdiff/100, Z_score/4', size=30)
plt.xlabel('Time', size=30)
plt.xticks(size=26)
plt.yticks(size=26)
plt.xlim([time[0], time[-1]])
plt.show()

```



[코드 119] Z score와 이상값 예측의 그래프



[단계 ⑦] 테스트 데이터 결과 분석

⑦-1. 정확도, 정밀도, 재현율, 및 F1-score 계산

- N-HiTS 알고리즘을 이용하는 경우:

단계 ⑥에서 정상적인 학습데이터에 대하여 계산된 (N-HiTS) 예측 값과 실제 값 사이의 차이 값을 계산하여 그 통계 분포로부터 표준편차 σ 를 구할 수 있다. 이를 이용하여 테스트 데이터에 대하여 각 시점 i 에 대한 예측 값과 실제 값의 차이를 학습 데이터의 표준편차로 나눈 값 즉, Z score인 Z_i 를 구할 수 있다. 즉, $Z_i \equiv (x_i^{(p)} - x_i^{(t)})/\sigma$ 이다. 여기서 $x_i^{(p)}$ 는 N-HiTS의 예측값이고 $x_i^{(t)}$ 는 실제값이며 σ 는 학습데이터에 대한 표준 편차이다. 각 시점의 Z_i 가 일정한 threshold 값을 넘어설 때 이를 anomaly로 규정한다.

- 예를 들어 모든 시점에서 정상인 데이터 “WeldingTest_01_OK.csv”에 대하여 N-HiTS 알고리즘을 적용하면 threshold 값을 4로 잡을 때 모든 시점에서 anomaly 없이 정상으로 나타남을 알 수 있다.

- 한편으로 고립된 시점에 비정상이 있는 테스트 데이터인 “WeldingTest03_NG.csv”에 대한 N-HiTS 알고리즘의 정확도(Accuracy), 정밀도(Precision), 재현율(Recall)을 계산하여 F1-score를 다음의 계산 공식으로 구할 수 있다.

$$* \text{정확도}(A) = (TP+TN) / (TN+FP+FN+TP) = 0.9960$$

$$* \text{정밀도}(P) = TP / (TP+FP) = 0.8947$$

$$* \text{재현율}(R)=TP / (TP+FN) = 0.8947 \text{ 인데 여기서}$$

TP = True Positive, TN = True Negative, FP = False Positive,
FN = False Negative를 각각 나타낸다.

- 정밀도와 재현율을 결합하여 다음과 같이 F1-score를 얻는다.

$$F1\text{-score} = 2./(1/P + 1/R) = 0.8947$$

- 연속된 구간에 비정상이 있는 테스트 데이터인 “WeldingTest04_NG.csv”에 대한 통계분석 방법에 의한 이상탐지의 경우에는 위와 같은 공식을 써서

$$* \text{정확도}(A) = (TP+TN) / (TN+FP+FN+TP) = 1.0$$

$$* \text{정밀도}(P) = TP / (TP+FP) = 1.0$$

$$* \text{재현율}(R)=TP / (TP+FN) = 1.0 \text{ 이고}$$

따라서 이를 결합하면 F1-score = 1.0을 얻는다.



[단계 ⑧] 알고리즘 평가

⑧-1. 알고리즘 평가

- N-HiTS 알고리즘 및 통계분석 방법을 배터리 공정에서의 예지보전에 적용하였다. N-HiTS 알고리즘 적용 시 배터리에서의 Real Power의 시간변화를 잘 예측한다는 것을 알 수 있다. 또한, 연속된 구간에서 비정상적인 데이터가 발생한 경우, 통계분석 방법을 적용 시에도 동일하게 잘 예측하는 것을 알 수 있다. 이를 이용하여 데이터의 Anomaly를 높은 정밀도와 재현율로 탐지하는 것을 확인하였다.

3. 유사 타 현장의 「전자부품(배터리팩) 예지보전 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조현장 소개

- 본 분석에서는 배터리 관련 제품 생산에 사용되는 다양한 설비공정 중 용접공정을 대상으로 하며, 추가적인 센서를 설치하지 않고 현장에서 수집 가능한 공정 데이터를 기반으로 설비의 상태를 파악하여 정상과 비정상 레이블을 부여하였다. 실제 제조현장에서 설비 고장으로 인한 생산량 감소는 매우 치명적인 이슈이며, 양산과정에서 설비의 내/외부적인 요인에 의해 비정상적으로 운영되지 않도록 해야 한다.
- 따라서, 배터리 용접 설비의 각종 상태 정보를 모니터링/관리하고자 하는 제조현장에 본 분석에서 제안하는 방법론을 적용하여 기존 운용방식보다 효율적으로 설비를 관리할 수 있다.
- 본 분석에서 제안하는 AI 모델은 설비로부터 수집한 공정 데이터를 분석하기 위해 비교적 최근에 나온 다변량 시계열 예측 알고리즘인 N-HiTS(Neural Hierarchical Interpolation for Time Series Forecasting) 알고리즘을 채택하였다. N-HiTS 알고리즘은 높은 예측 정확도와 빠른 계산 속도로 인해 이상 탐지에 적합하므로 배터리 용접설비 외에도 조립 및 입고검사 등 현장에서 운영되는 다양한 설비들에 적용될 수 있다.
- 특히, 자동화 설비가 갖춰진 설비와 수작업이 함께 운용되어 불량품이 빈번하게 발생하는 제조현장에 본 분석을 적용할 시 설비 이상 상태를 예측할 뿐만 아니라 품질관리 분야까지 확장하여 적용될 수 있다.

3.2 본 「전자부품(배터리팩) 예지보전 AI 데이터셋」 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 대상 설비에 대한 공정 데이터를 실시간 수집하여 활용할 수 있는 인프라가 구축되어 있어야 한다.
- 특히, 본 분석에서는 AAS 표준기반 제조데이터 수집/저장 체계를 통해 수집한 실시간 공

정 데이터를 기반으로 설비 이상 예측 모델을 만드는 과정을 학습할 수 있다는 점에서 의미하는 바가 크다. 단, AAS 표준기반 제조데이터 수집/저장 체계가 준비되어 있어야 하므로 2장 분석실습에서 설명한 바와 같이 깃허브 사이트(<https://github.com/kosmonestfield>)에서 제공하는 가이드에 따라 현장에 적용되어 있어야 한다.

- 대상 설비로부터 수집한 공정 데이터를 기반으로 결론을 도출할 수 있으나 추가적인 센서를 활용(예: 비전, 변위 등)하면 보다 정확한 분석 결과를 얻을 수 있다.
- 대상 설비를 통해 생산되는 제품의 사양이 빈번하게 변경되는 환경에서 본 분석이 제한적으로 적용될 수 있으므로 다른 생산 제품과 명확하게 구분될 수 있도록 데이터를 추가하거나 분류해야 한다.
- 본 분석에서 사용된 N-HiTS 알고리즘은 비정상 데이터가 연속적으로 긴 시점 구간에서 나타나는 경우 적합하지 않을 수 있다. 이 경우 학습 데이터 통계를 분석하여 몇 개의 가우스 분포의 중첩으로 가정하고 각각의 가우스 분포 모드에 대한 평균과 표준편차를 구해야 한다. 그리고, 각 시점의 테스트 데이터에 대해 각 가우스 모드 평균값으로부터의 거리를 해당 모드의 표준편차로 나누어 Z-score를 계산해야 한다. Z-score 중 최소값을 구해 정해진 threshold를 넘으면 anomaly로 예측할 수 있다. 단, 학습 데이터로부터 가우스 모드의 개수와 대략적인 범위를 추정할 수 있어야 한다.
- 대상 설비로부터 불량품이 생산된 경우 단순히 설비 이상으로 인해 불량품이 생산된 것으로 판단해서는 안되며, 각종 공정 데이터와 생산 제품의 입출고에 대한 상관관계를 분석하여 발생 원인을 찾아야 한다.
- 따라서, AI 분석 전문가는 본 분석결과를 현장에 적용하기 위해 설비/공정 전문가와 적극적으로 의사소통을 하면서 이상 데이터 기준을 명확하게 파악하고 있어야 하며, 현장상황을 고려하여 유연하게 진행해야 한다.

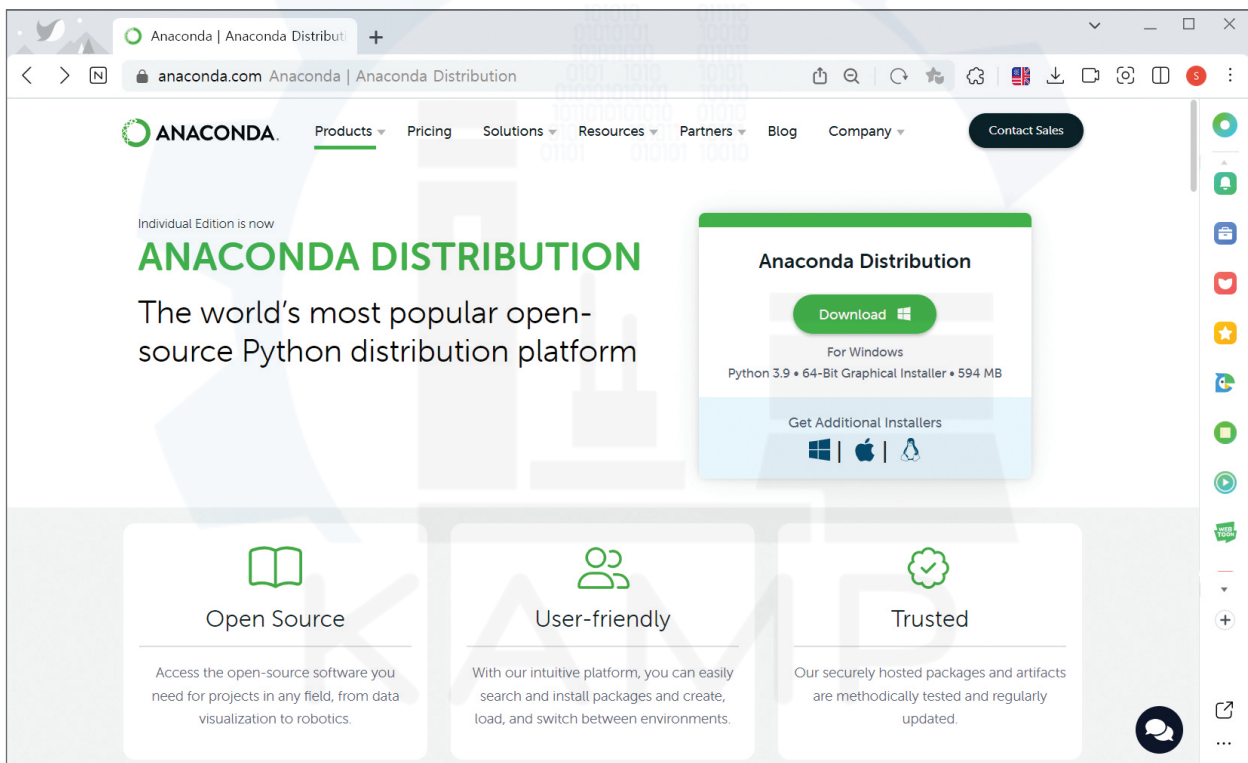
3 부록_분석환경 구축을 위한 설치 가이드

데이터 분석을 위한 Python 환경은 Anaconda를 설치함으로써 간단하게 구축할 수 있다. Anaconda는 데이터 분석 및 예측을 위한 모델 생성 등을 위한 package 들을 모아놓은 Python 배포판으로서, Numpy, SciPy, matplotlib, pandas, Ipython, Jupyter Notebook, scikit-learn 을 모두 포함하고 있다.

① Anaconda 설치 파일 다운로드

Anaconda 다운로드 사이트인

“<https://www.anaconda.com/products/distribution>”에서 설치 파일을 다운로드 받을 수 있다. 이 URL을 주소창에 입력하면 다음 화면이 표시되고 여기에서 Download를 클릭한다.



Anaconda의 설치 파일은 “Anaconda3-2022.05-Windows-x86_64.exe”이며, 현재 사용 중인 컴퓨터에 64비트 윈도우가 설치돼 있으면 64비트용 설치 파일을 다운로드 받고, 32비트 윈도우가 설치돼 있다면 32비트용 설치 파일을 다운로드 받으면 된다. 다운로드 받은 Anaconda는 2022.05 버전이며, Python 3.9를 포함하고 있다.

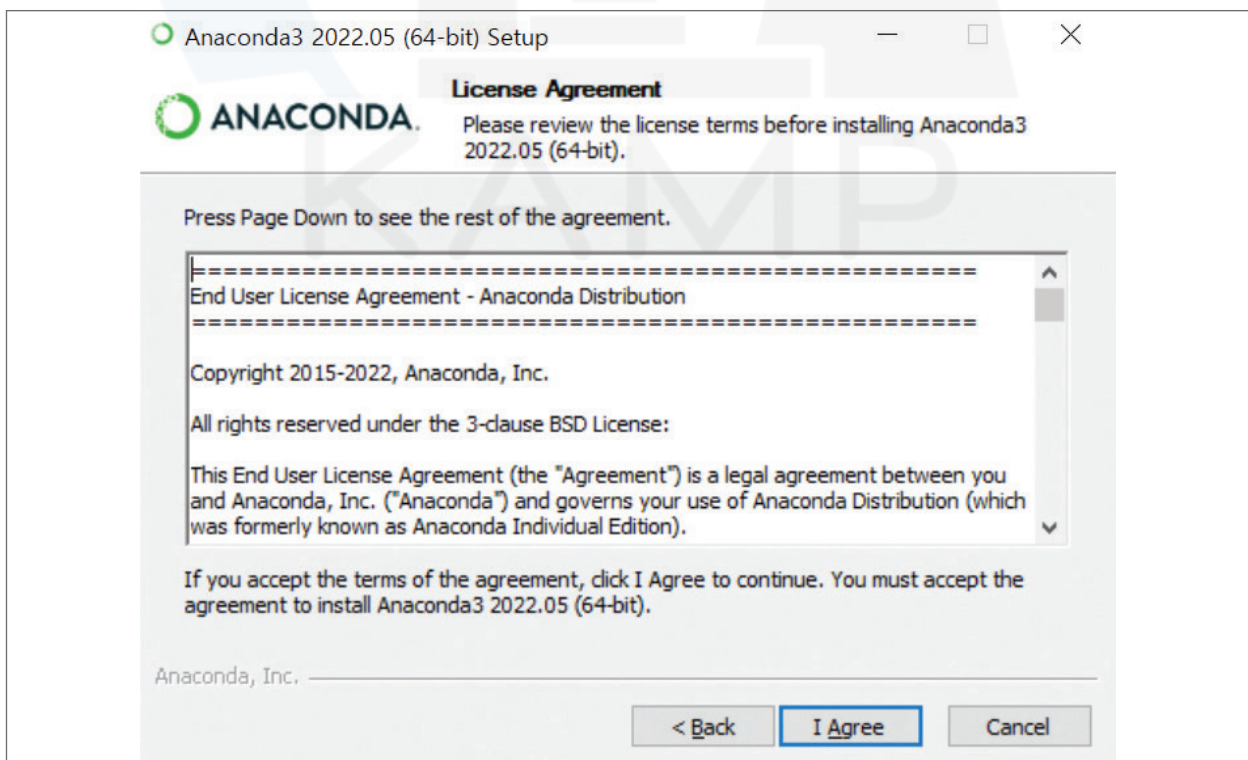


② 설치 파일을 내려받은 후, 해당 파일

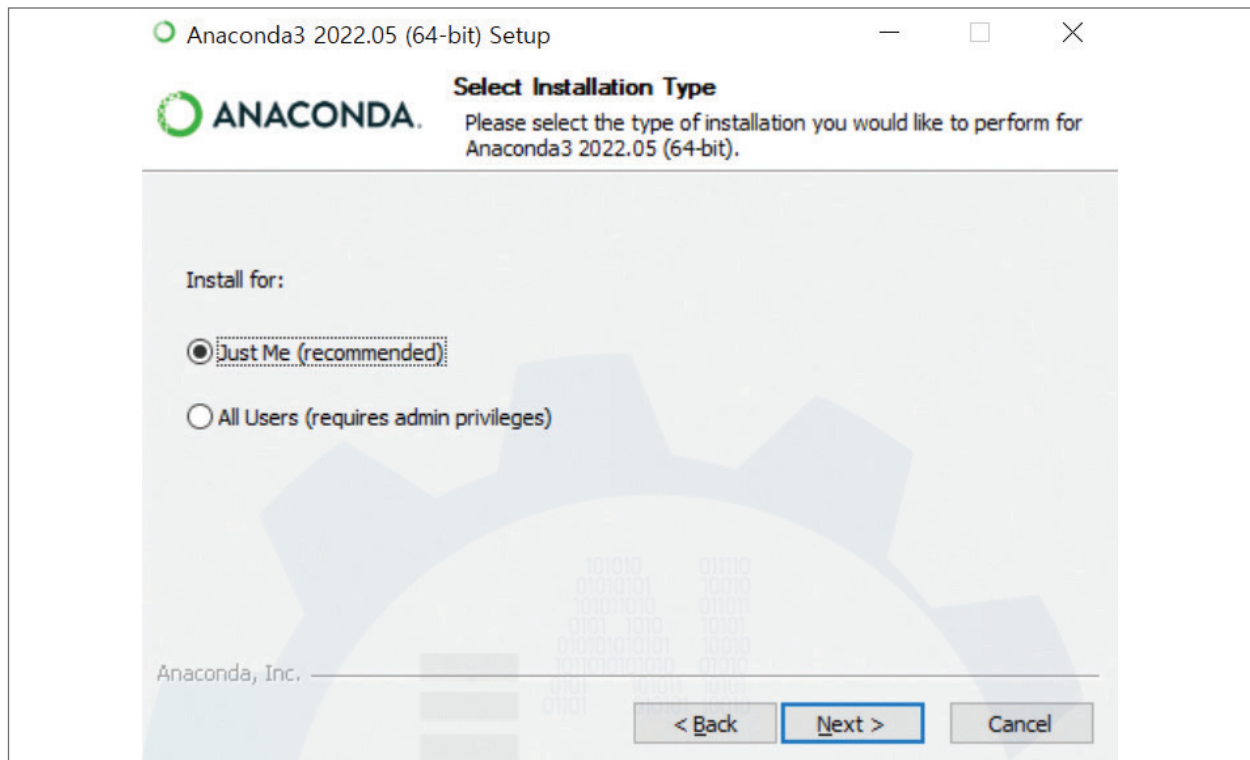
“Anaconda3-2022.05-Windows-x86_64.exe”을 클릭하여 설치를 시작한다. 다음 그림과 같이 설치 화면이 표시되면 [Next] 버튼을 클릭하여 다음 단계로 진행한다.



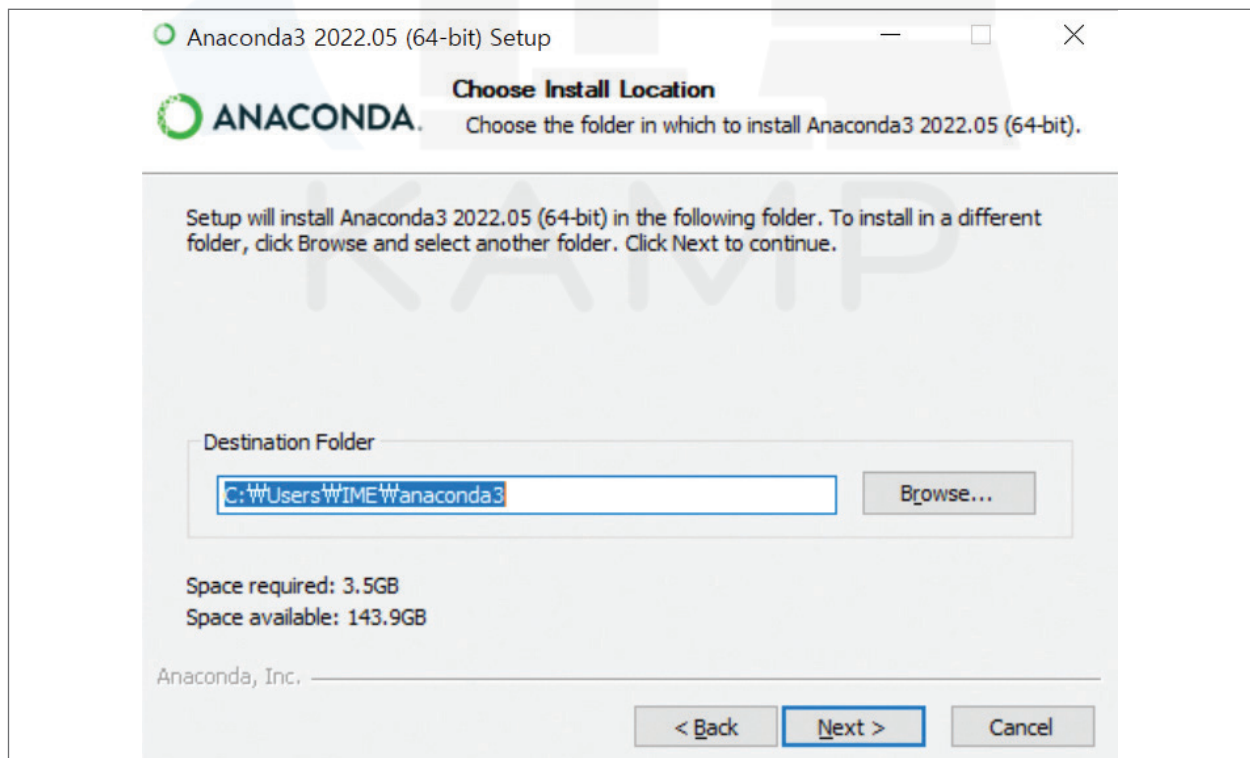
③ 다음에 표시되는 화면은 라이선스 동의에 관한 화면이다. 여기에서 [I Agree] 버튼을 클릭하여 다음 단계로 진행한다.



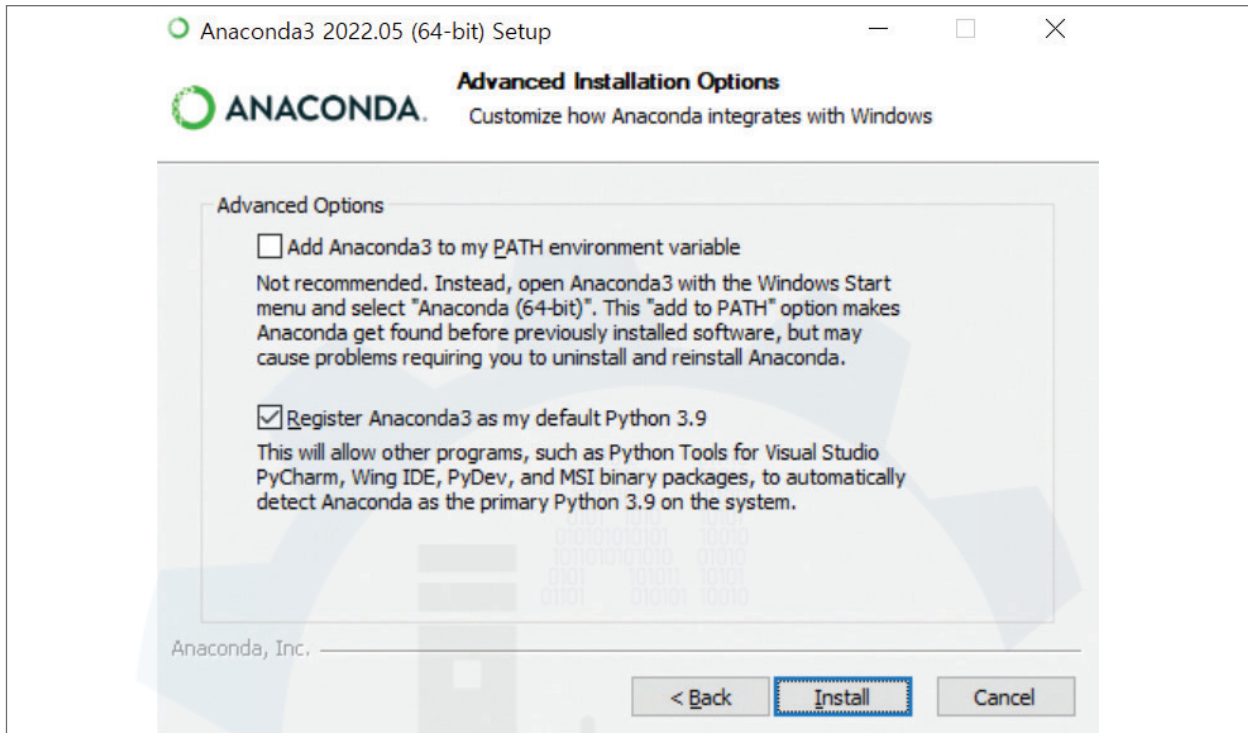
- ④ 다음에 표시되는 화면은 설치 유형을 선택하는 화면이다. [Just Me]를 선택한 후 [Next] 버튼을 클릭한다.



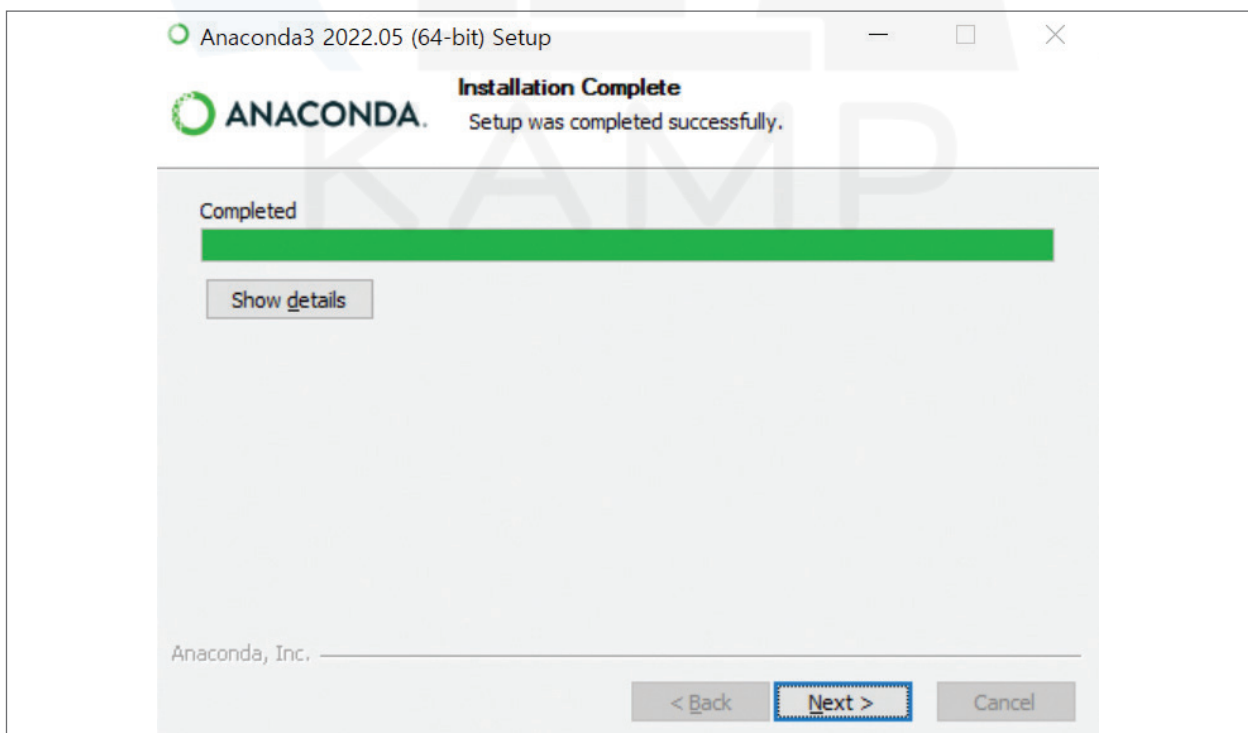
- ⑤ 다음으로 Anaconda를 설치할 폴더를 지정하는 화면이 표시된다. 여기에서 적절한 폴더를 지정하고 [Next] 버튼을 클릭한다.



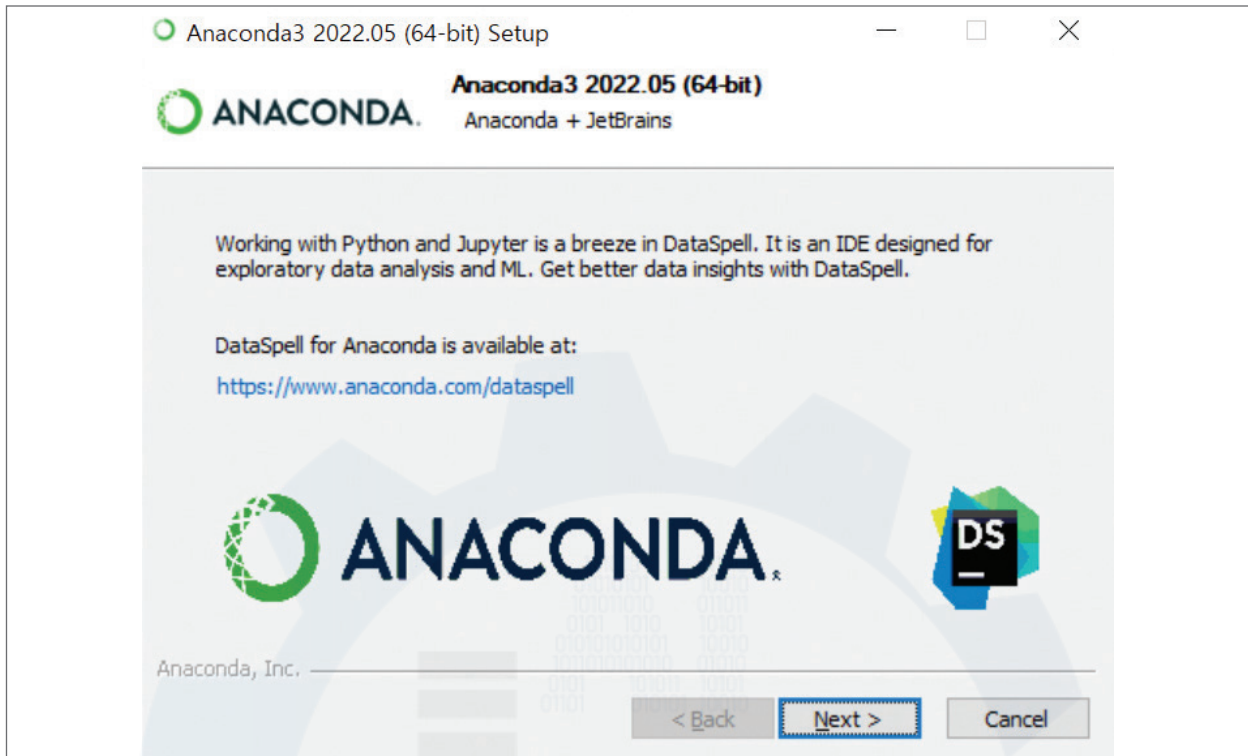
- ⑥ 다음에 표시되는 화면은 Advanced Options를 선택하는 화면이다. 첫째 Option은 Anaconda를 PATH 환경변수에 추가하여 사용하는 것이며, 둘째 Option은 다른 프로그램이 자동으로 Anaconda를 인식하게 하는 것이다. 여기에서는 두 번째 옵션을 선택한 후 [Install] 버튼을 클릭하여 다음 단계로 진행한다.



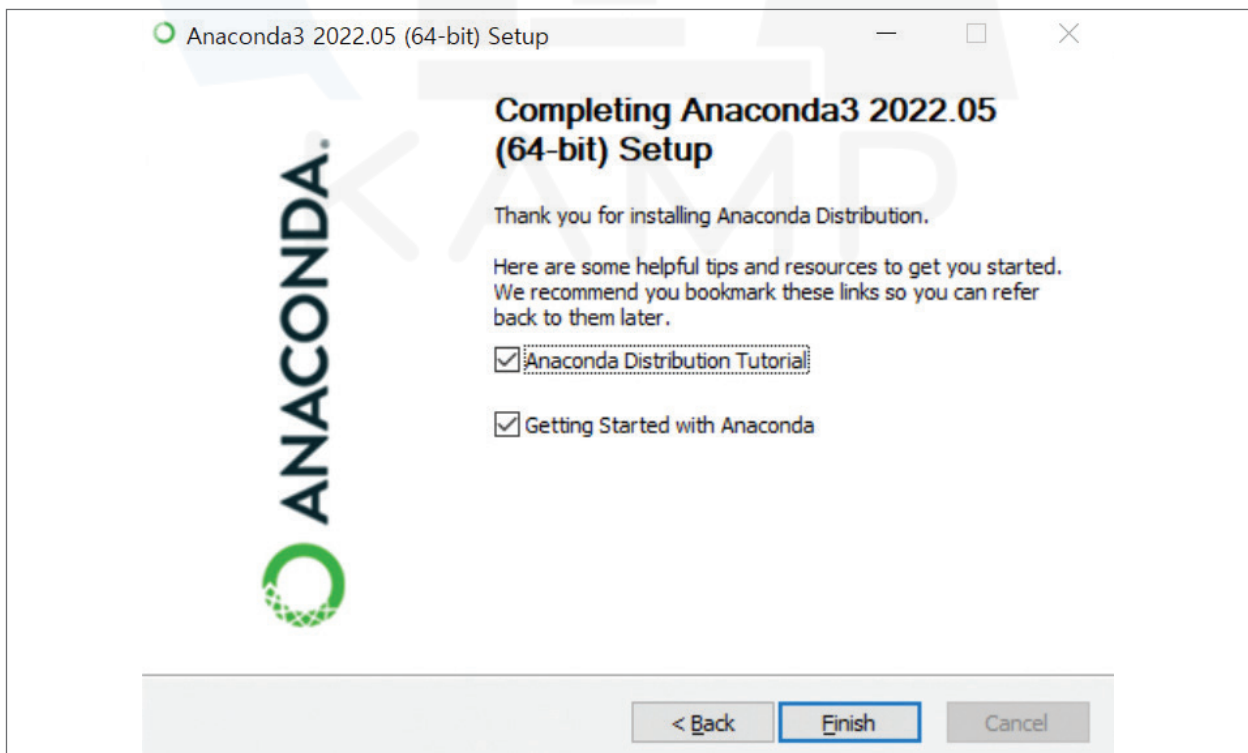
- ⑦ Anaconda 설치가 완료되면 다음 화면에 “Installation Completed”라는 메시지가 표시된다. 이 화면에서 [Next] 버튼을 눌러 다음 단계로 진행한다.



- ⑧ 다음과 같이 데이터 분석가를 위한 도구인 DataSpell에 관한 안내 화면이 표시되며, 이 화면에서 [Next] 버튼을 클릭하여 다음 단계로 진행한다.



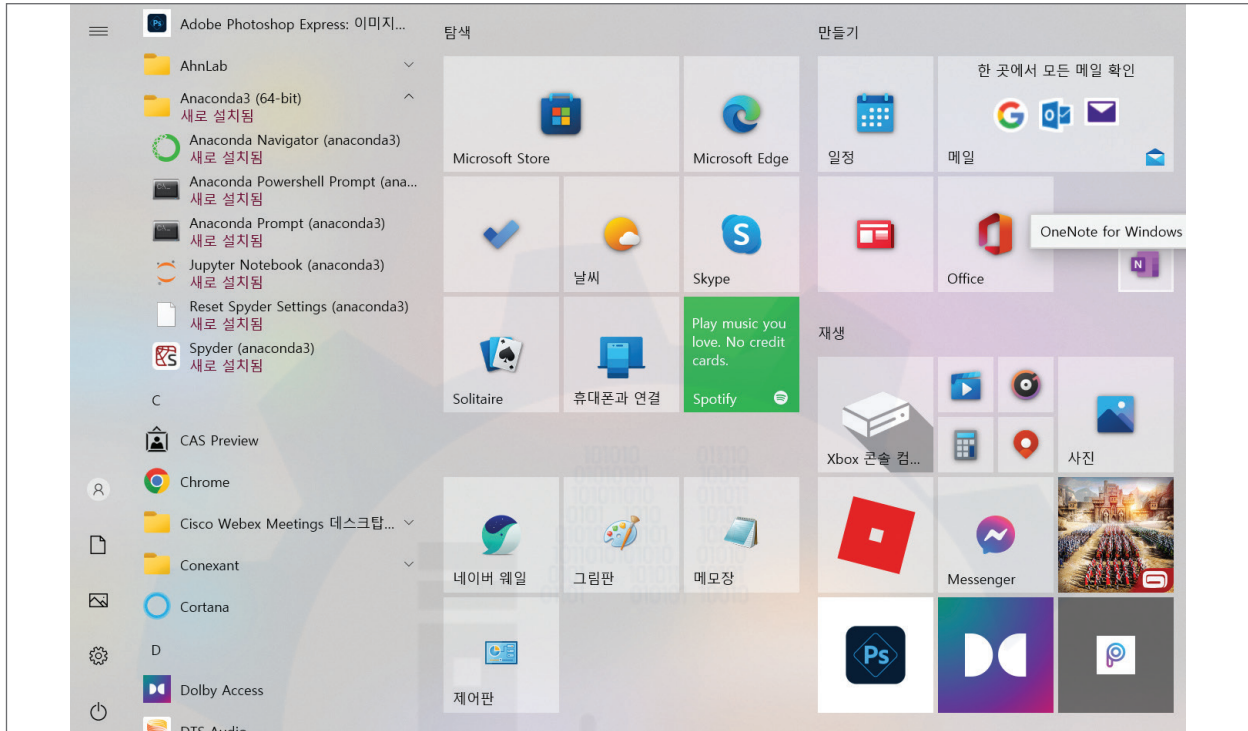
- ⑨ 다음 화면이 표시되면 Anaconda 설치 과정이 모두 완료되었음을 나타내며, Anaconda에 대한 정보를 추가적으로 얻으려면 이 화면에서 두 가지 Option을 선택하여 내용을 살펴볼 수 있다. 모든 설치 과정을 종료하기 위하여 [Finish] 버튼을 클릭한다.



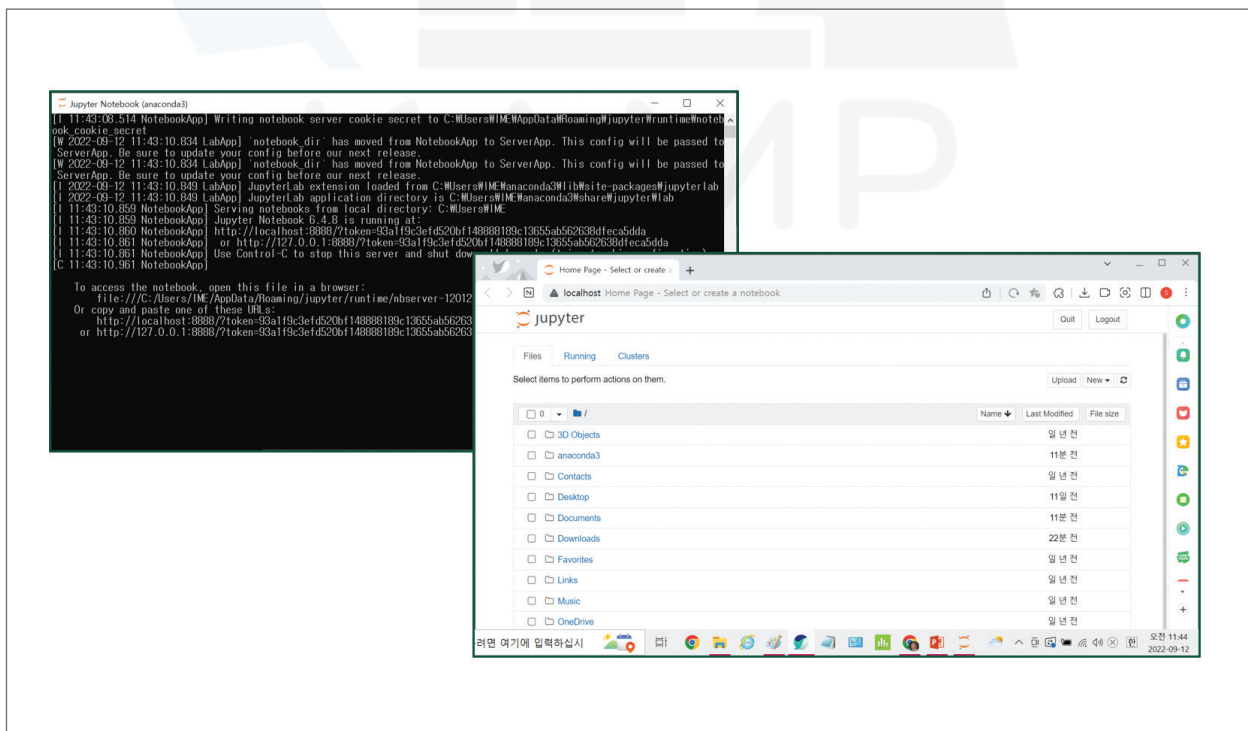


⑩ Jupyter Notebook 시작하기

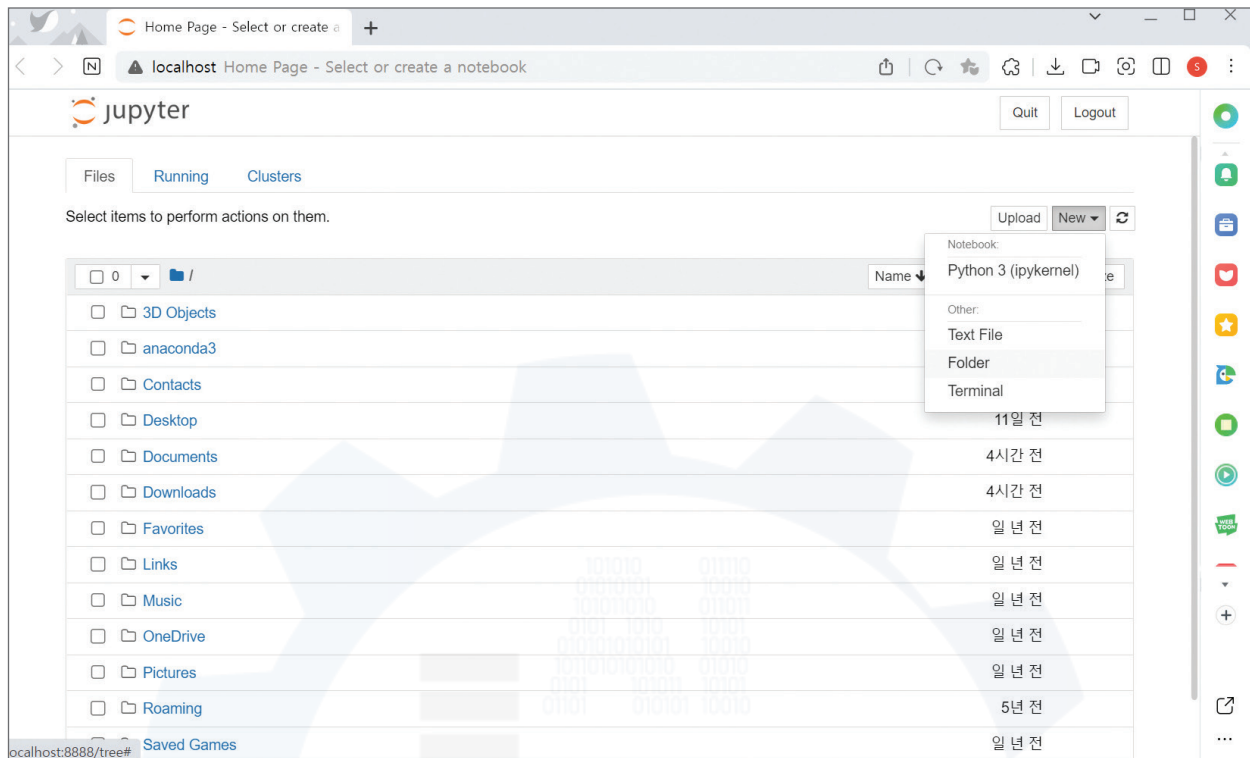
Anaconda를 설치하면 Windows 처음 화면에 다음과 같이 설치된 프로그램의 리스트가 표시된다. 여기에서 “Jupyter Notebook (anaconda3)”를 클릭하여 Jupyter Notebook을 시작한다.



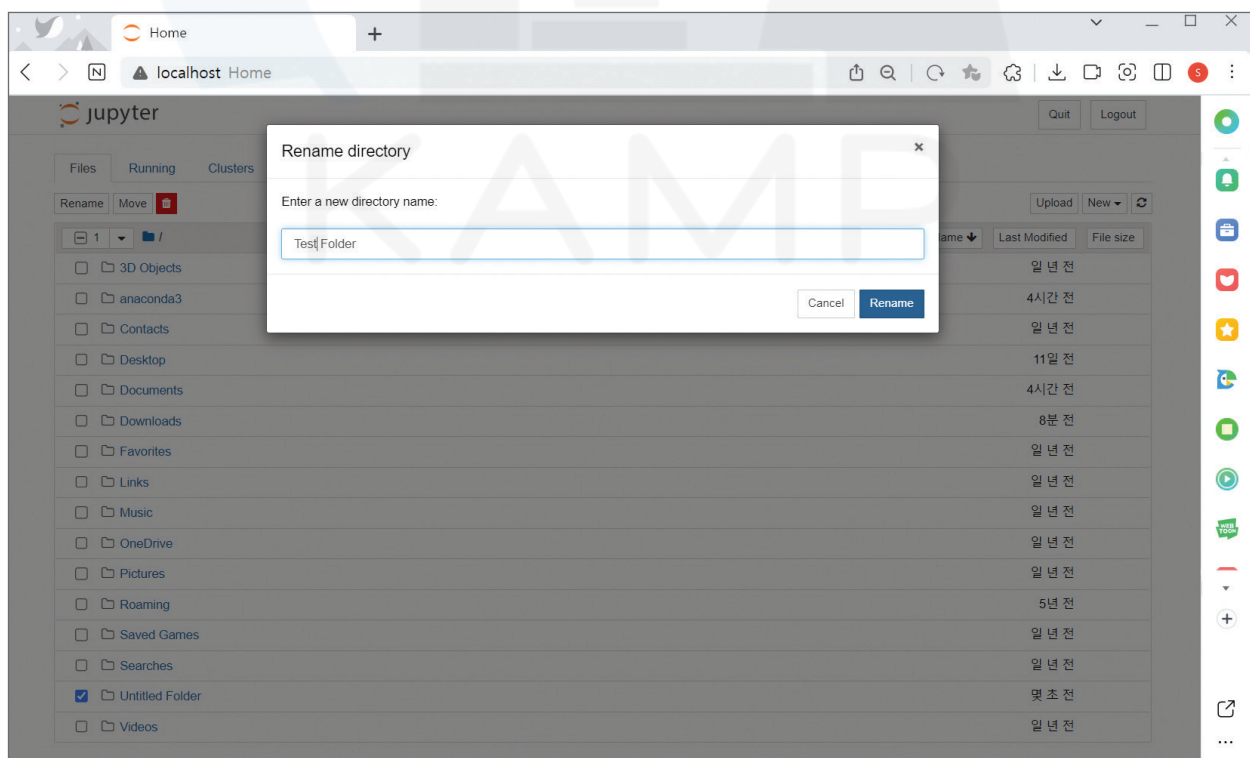
⑪ Jupyter Notebook을 시작하면 다음과 같이 Jupyter Notebook 콘솔 및 프로그래밍 작업을 할 수 있는 메인 화면이 표시된다.



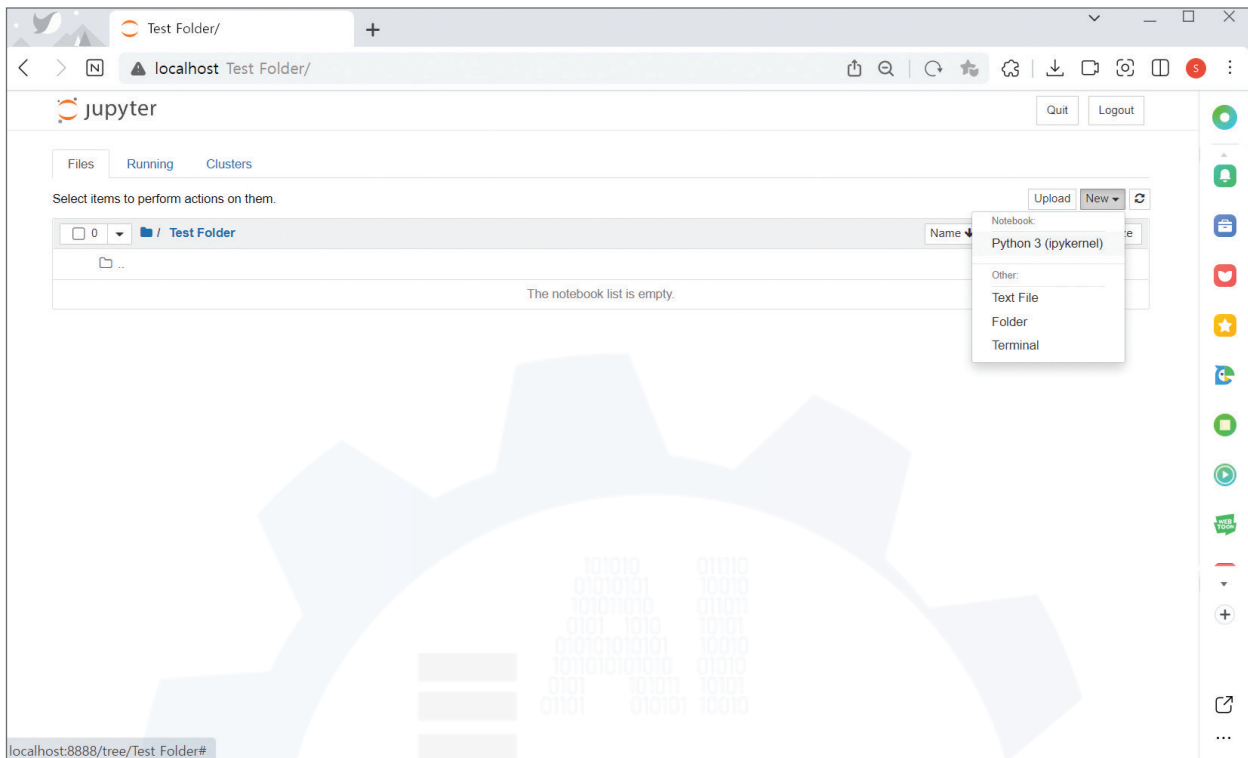
- ⑫ 새로운 폴더 생성: 다음 화면의 오른쪽에서 New를 클릭하여 Folder를 선택하면 새로운 폴더가 “Untitled Folder”라는 이름으로 생성된다.



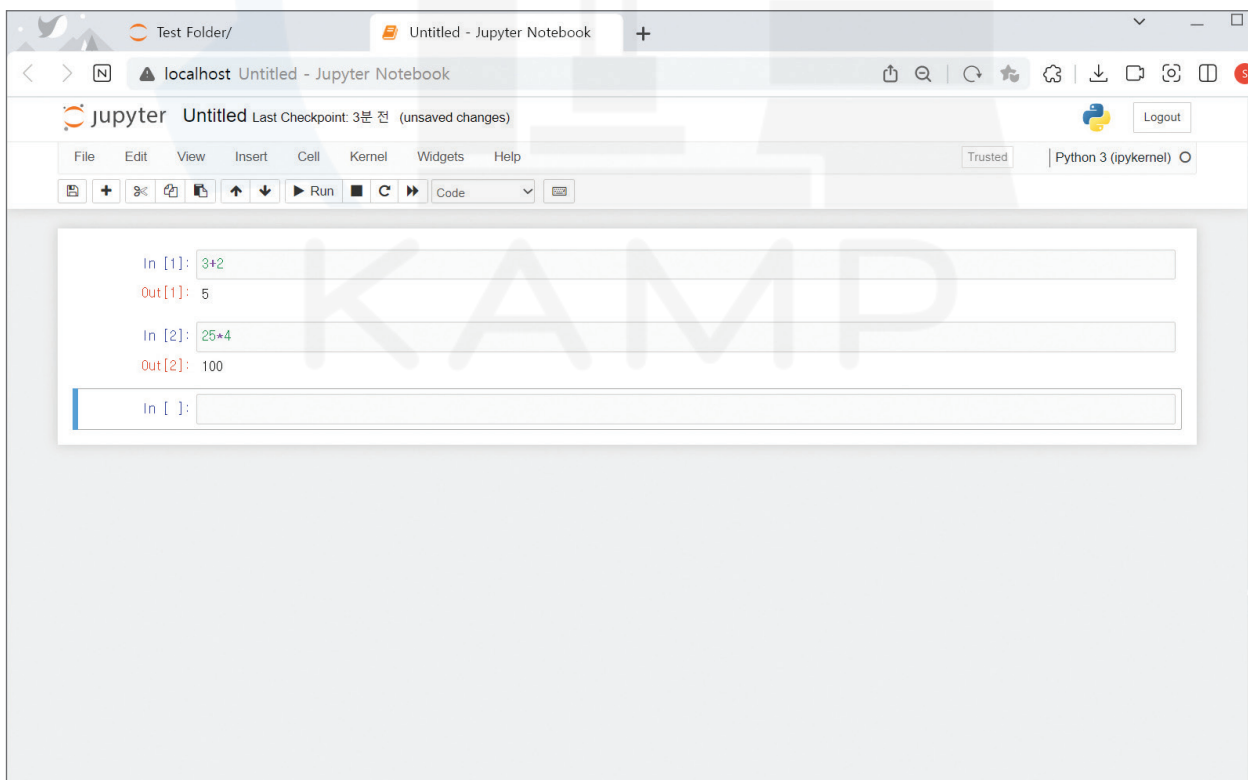
- ⑬ 생성된 “Untitled Folder”를 클릭하여 선택하고 Rename을 클릭하면 폴더의 이름을 지정하는 팝업창이 표시된다. 이 창에서 폴더 이름을 주고 Rename 버튼을 클릭하여 생성된 폴더에 이름을 부여할 수 있다.



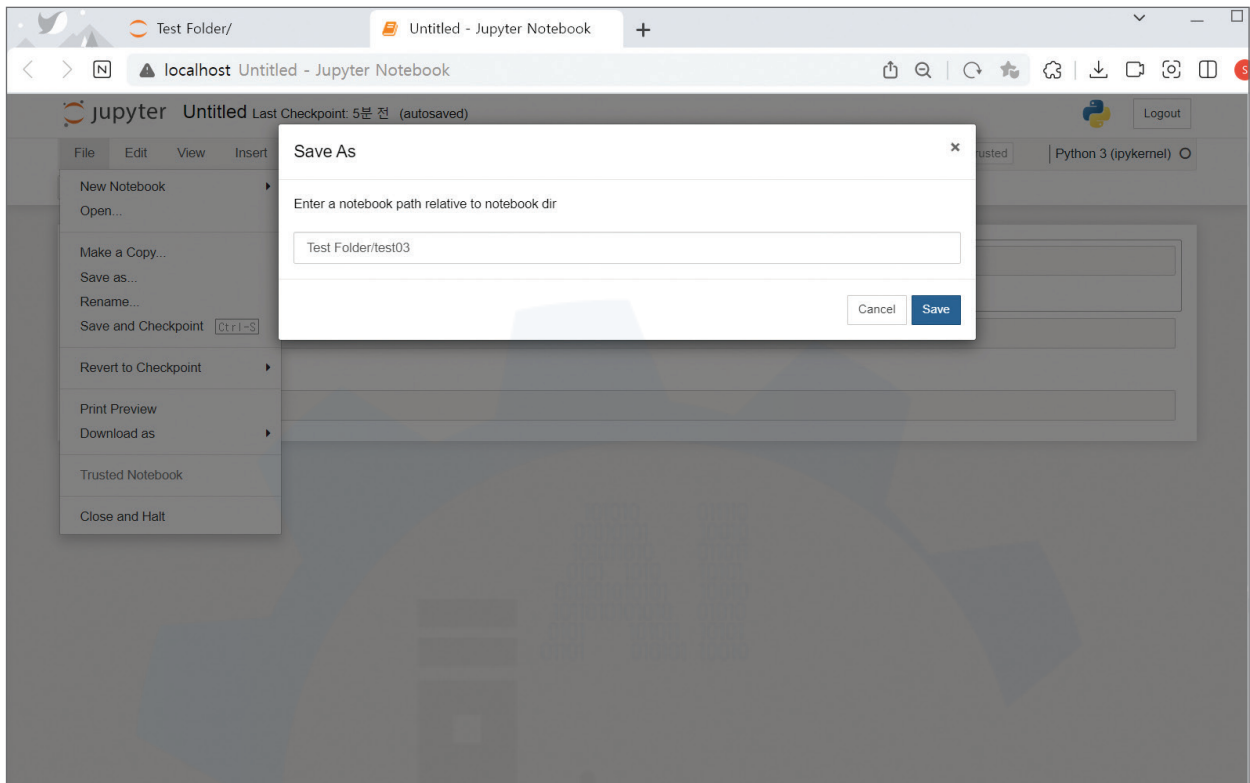
- ⑭ 새로운 파일 생성: 생성된 폴더로 이동하여 화면의 오른쪽에서 New를 클릭하여 Python 3 (ipykernel)을 선택하면 새로운 파일이 “Untitled”라는 이름으로 생성된다.



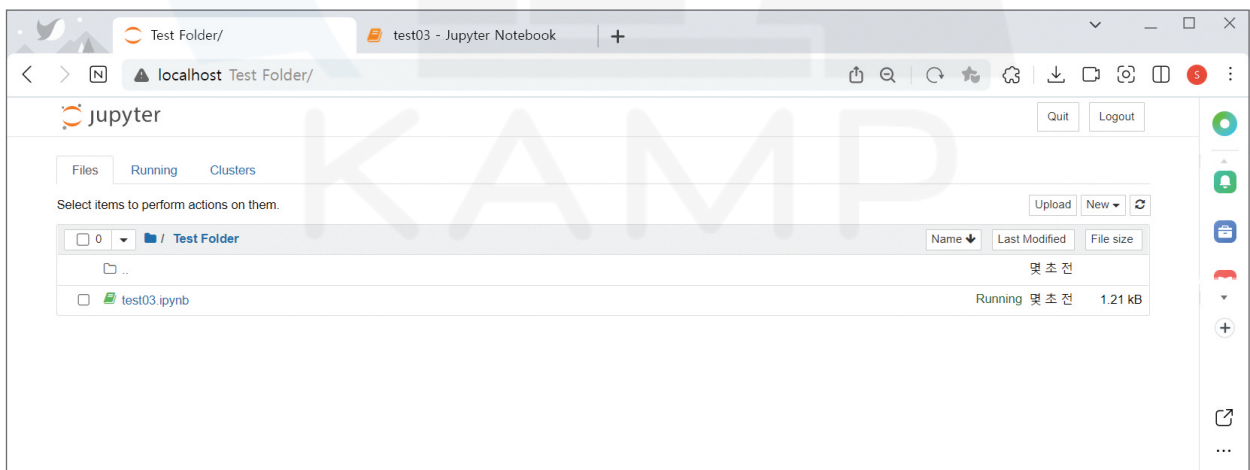
- ⑮ 표시된 화면에서 Python 코드를 작성한다.



- ⑩ 코드를 모두 작성한 후, 파일을 저장하기 위하여 File 메뉴를 클릭하고 “Save As”를 선택하면 파일의 이름을 지정하는 팝업창이 표시된다. 이 창에서 파일 이름을 주고 Save 버튼을 클릭하여 생성된 폴더에 파일을 저장할 수 있다.



- ⑪ 파일을 저장한 후, 다음 화면에서 작업을 마친 파일을 확인할 수 있다. 모든 작업이 완료된 후에는 Jupyter Notebook 화면을 닫고 콘솔에서 Ctrl+c 를 눌러 Anaconda를 종료한다.



4 부록_데이터 품질 전처리 [실습 코드]

데이터 품질 전처리에 필요한 6가지의 데이터 품질 평가지표 (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)의 산출 방법에 대하여 기술한다.

- ▶ 품질 전처리를 위한 라이브러리 import 하고, 필요한 파일들을 불러온 후, 하나의 Data frame으로 병합한다. 병합한 데이터 파일을 표시하면 다음과 같으며, 140,985 행과 9개의 속성을 포함하고 있다.

```
import pandas as pd
import numpy as np
from datetime import date
from datetime import time
from datetime import datetime
from dateutil.parser import parse
file_path1 = 'data/raw_data/train/Training_Data.csv' # the file path is the archive location of the file in our computer
file_path2 = 'data/raw_data/test/WeldingTest_01_OK.csv'
file_path3 = 'data/raw_data/test/WeldingTest_02_OK.csv'
file_path4 = 'data/raw_data/test/WeldingTest_03_NG.csv'
file_path5 = 'data/raw_data/test/WeldingTest_04_NG.csv'
#data = pd.read_csv(file_path) # using pandas library (pd) to read the csv file.
data = pd.concat(
    map(pd.read_csv, [file_path1,file_path2,file_path3,file_path4,file_path5]), ignore_index=True)
data
```

	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1688	1000	100	82	1154	2022-01-08 10:11:45.181
1	2	250	241.2	1713	1000	100	83	1670	2022-01-08 10:11:48.487
2	3	250	241.1	1695	1000	100	82	1153	2022-01-08 10:11:51.245
3	4	250	241.2	1717	1000	100	83	1670	2022-01-08 10:11:54.493
4	5	250	241.1	1698	1000	100	82	1154	2022-01-08 10:11:57.269
...	...	...	...	...	...	...	...	...	...
140980	35	30	19.4	127	1000	100	38	650	2022-08-03 17:46:54.451
140981	36	30	19.4	127	1000	100	38	650	2022-08-03 17:46:56.147
140982	37	30	19.4	127	1000	100	38	650	2022-08-03 17:46:57.901
140983	38	30	19.4	126	1000	100	38	650	2022-08-03 17:47:00.881
140984	39	30	19.4	127	1000	100	38	650	2022-08-03 17:47:02.353

140985 rows x 9 columns

[코드 120] 품질 전처리를 위한 라이브러리 및 파일 불러오기

- ▶ 컬럼명에 공백이 있으면 때때로 그 컬럼의 사용에 불편을 초래하므로, 다음 구문을 사용하여 컬럼명의 공백을 처리한 후, 데이터를 표시한다.

```
# Take out whitespace in column names
data = data.rename(columns=lambda x: x.strip())
data
```

	PageNo	Speed	Length	RealPower	SetFrequency	SetDuty	SetPower	GateOnTime	WorkingTime
0	1	250	241.1	1688	1000	100	82	1154	2022-01-08 10:11:45.181
1	2	250	241.2	1713	1000	100	83	1670	2022-01-08 10:11:48.487
2	3	250	241.1	1695	1000	100	82	1153	2022-01-08 10:11:51.245
3	4	250	241.2	1717	1000	100	83	1670	2022-01-08 10:11:54.493
4	5	250	241.1	1698	1000	100	82	1154	2022-01-08 10:11:57.269
...	...	...	...	...	...	...	...	...	...
140980	35	30	19.4	127	1000	100	38	650	2022-08-03 17:46:54.451
140981	36	30	19.4	127	1000	100	38	650	2022-08-03 17:46:56.147
140982	37	30	19.4	127	1000	100	38	650	2022-08-03 17:46:57.901
140983	38	30	19.4	126	1000	100	38	650	2022-08-03 17:47:00.881
140984	39	30	19.4	127	1000	100	38	650	2022-08-03 17:47:02.353

140985 rows x 9 columns

[코드 121] 컬럼명의 공백을 처리한 후의 데이터 표시

- ▶ 품질 전처리를 위하여 주어진 문자열이 date 형식인지 체크하는 함수를 정의한다.

```
# Check if a string is a date
def is_date(string, fuzzy=False):
    try:
        parse(string, fuzzy=fuzzy)
        return True
    except ValueError:
        return False
```

[코드 122] 문자열이 date 형식인지 체크하는 함수 정의

- ▶ 완전성 품질지수

$$\text{완전성 품질지수} = \left(1 - \frac{\text{결측데이터의 개수}}{\text{전체 데이터의 개수}}\right) \times 100$$

- ① 총 결측 데이터의 개수를 확인하기 위해 'isnull()' 함수를 사용하여 각 열에서의 결측 데이터 개수를 구한 뒤 'sum()' 함수를 이용하여 합산한다. 다음 함수 getCompleteness()는 완전성을 구하는 함수이다.



```
# Function to check completeness of a dataframe
# Returns total of null values and percentage of completeness
def getCompleteness(data):
    total_null_values = data.isnull().sum().sum()
    total_notNull_values = data.count().sum()
    total_values = total_null_values + total_notNull_values
    completeness = (total_notNull_values / total_values)*100
    return total_null_values, completeness
total_null_values, completeness = getCompleteness(data)
print("Total of null values = %d values"%total_null_values)
print("완전성 품질지수 : %.2f%%"%completeness)
```

```
Total of null values = 0 values
완전성 품질지수 : 100.00%
```

[코드 123] 분석도구를 활용한 완전성 품질지수 구하기

- ② 위 함수의 출력인 완전성 품질지수는 100%로 결측 데이터가 존재하지 않음을 알 수 있다. 결측치가 존재할 경우에는 결측치 처리 함수인 `handleMissingValue()`를 사용하여 결측 데이터를 처리한 후, 다시 함수 `getCompleteness()`를 호출하여 완전성 품질지수를 구한다. 이 때, 결측치 처리 함수인 `handleMissingValue()`는 본문 “2.3 분석체험”의 [코드 24]에 정의되어 있다.

▶ 유일성 품질지수

$$\text{유일성 품질지수} = \left(\frac{\text{유일한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터에서 유일한 값을 갖는 열을 찾는다. 시계열 데이터는 주로 Timestamp이다.
- ② 이 테이블에서는 ‘WorkingTime’ 칼럼이 고유 식별자로 사용되었으며, ‘WorkingTime’ 칼럼에 중복되는 항목이 없으므로 유일성의 값은 100%로 계산된다.
- ③ 다음 함수 `getUniqueness()`는 유일성을 구하는 함수이다.

```
# Function to get data uniqueness
# takes a series of data as an input
time_series = data["WorkingTime"]
def getUniqueness(data):
    total_data = data.count()
    unique = len(data.unique())
    uniqueness = (unique / total_data) * 100
    return uniqueness
print("유일성 품질지수 : ", getUniqueness(time_series), "%")
```

```
유일성 품질지수 : 100.0 %
```

[코드 124] 분석도구를 활용한 유일성 품질지수 구하기



▶ 유효성 품질지수

$$\text{유효성 품질 지수} = \left(\frac{\text{유효성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터가 주어진 유효범위를 벗어나는지를 확인한다.
- ② 데이터의 유효 범위를 range_min과 range_max 값으로 지정하며, 유효한 범위 내에 속하는 데이터 항목을 count 하여 유효성을 계산한다.
- ③ 다음 함수 getValidity()는 유효성을 구하는 함수이다.

```
# Function to get data validity of a data series
def getSeriesValidity(data, range_min, range_max):
    column_name = data.name
    total_data = data.count()
    if(len(column_name) >= 4 and column_name == 'WorkingTime'):
        time_conv = pd.to_datetime(data).dt.date
        temp = data.loc[time_conv >= range_min]
        temp = data.loc[time_conv <= range_max]
        in_range_data = temp.count()
    else:
        in_range_data = data.loc[(data >= range_min) & (data <= range_max)].count()
    validity = in_range_data / total_data * 100
    return validity

def getValidity(data, ranges):
    validities = []
    for col in ranges:
        current_validity = getSeriesValidity(data[col], ranges[col][0], ranges[col][1])
        #print(col + " validity is: %.2f%%" % current_validity)
        validities.append(current_validity)
    average_validity = sum(validities) / len(validities)
    return average_validity
```

```
# Calculate the validity of all columns.
columns = data.columns
range_dictionary = {}
default_range = (-200, 2000)
# If there is a special range to be checked, please add in this list.
# In format: "Name_of_Column": (range_min, range_max)
# For example,
set_range = {
    "Speed": (30, 250),
    "WorkingTime": (date(1999, 1, 1), date(2023, 12, 12)),
}

# date format: date(year, month, day)
# time format: time(hour, minutes)
# WorkingTime is going to be taken as Date column
for column_name in columns:
    if column_name in set_range:
        range_dictionary[column_name] = set_range[column_name]
    else:
        range_dictionary[column_name] = default_range

print("유효성 품질지수 : %.2f%%" % getValidity(data, range_dictionary))
```

유효성 품질지수 : 100.00%

[코드 125] 분석도구를 활용한 유효성 품질지수 구하기



▶ 일관성 품질지수

$$\text{일관성 품질지수} = \left(\frac{\text{일관성을 만족한 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터의 유형이 정해진 형식에 맞는지 확인한다.
- ② 전체 데이터의 개수와 형식에 맞는 데이터의 개수를 구하여 일관성 품질지수를 계산한다.
- ③ 다음 함수 getConsistency()는 일관성을 구하는 함수이다.

```
def getSeriesConsistency(data, data_type):
    total_data = data.count()
    #print(data_type)
    if(data.name == 'WorkingTime'):
        types = list(map(lambda item: is_date(item), data))
    else:
        types = list(map(lambda item: isinstance(item, data_type), data))
    types = pd.Series(types)
    consistent_data = types.sum()
    consistency = (consistent_data / total_data) * 100
    return consistency

def getConsistency(data, types):
    consistent_data = []
    for col in types:
        current_const = getSeriesConsistency(data[col], types[col])
        #print(col + " consistency is: %.2f%%" % current_const)
        consistent_data.append(current_const)
    average_const = sum(consistent_data) / len(consistent_data)
    return average_const
```

```
# We will calculate the consistency of all columns
columns = data.columns
const_dictionary = {}
default_type = (float, int)
numeric_type = (float, int)
int_type = int
# If there is a special type to be checked, please add in this list.
# In format: "Name_of_Column": (range_min, range_max)
#For example,
set_types = {
    "PageNo": int_type,
}
# Time and Date types will be set automatically
# Columns names with Date and Time must be or start with the words Date and Time, respectively.
for column_name in columns:
    if column_name in set_types:
        const_dictionary[column_name] = set_types[column_name]
    else:
        const_dictionary[column_name] = default_type
print("일관성 품질지수 : %.2f%%" % getConsistency(data, const_dictionary))
```

일관성 품질지수 : 100.00%

[코드 126] 분석도구를 활용한 일관성 품질지수 구하기



▶ 정확성 품질지수

$$\text{정확성 품질지수} = 1 - \left(\frac{\text{정확성에 위배되는 데이터의 개수}}{\text{전체 데이터의 개수}} \right) \times 100$$

- ① 데이터 항목에 대하여 검증된 정확한 정보 (참조 데이터)가 있는지를 확인하고, 참조 데이터가 있는 경우, 데이터 항목 값과 참조 데이터의 값을 비교한다.
- ② 여기에서 다루는 데이터셋은 참조 데이터가 존재하지 않으므로 정확성을 판단하지 않는다.

▶ 무결성 품질지수

$$\text{무결성 품질지수} = \left\{ 1 - \left(\frac{\text{유일성, 유효성, 일관성 중 100%가 아닌 지수}}{3} \right) \right\} \times 100$$

무결성 품질지수는 앞에서 구한 ‘유일성’, ‘유효성’, ‘일관성’ 지수를 기초로 위의 식으로 계산된다. 이 지수들이 모두 100%이므로 무결성 품질지수도 100%가 된다.

▶ 가중치 지수

$$\text{가중치지수} = \text{품질지수} \times \text{가중치}$$

데이터 및 적용 업무의 특성에 따라 품질지수의 중요도가 달라지며, 이에 따라 적절한 가중치 지수를 부여한다.

▶ 데이터 품질지수

$$\text{데이터 품질지수} = \sum \text{가중치지수}$$

앞에서 구한 6가지의 데이터 품질지수에 적절한 기중치를 반영하여 최종적으로 데이터 품질 지수를 구한다. 다음 테이블과 같이 최종 품질지수를 산출하면 100%가 된다.

항목	품질지수	가중치	가중치지수	오류율
완전성	100%	20%	20%	0%
유일성	100%	20%	20%	0%
유효성	100%	20%	20%	0%
일관성	100%	20%	20%	0%
정확성	-	-	-	-
무결성	100%	20%	20%	0%
최종 품질지수	100%	100%	100%	0%

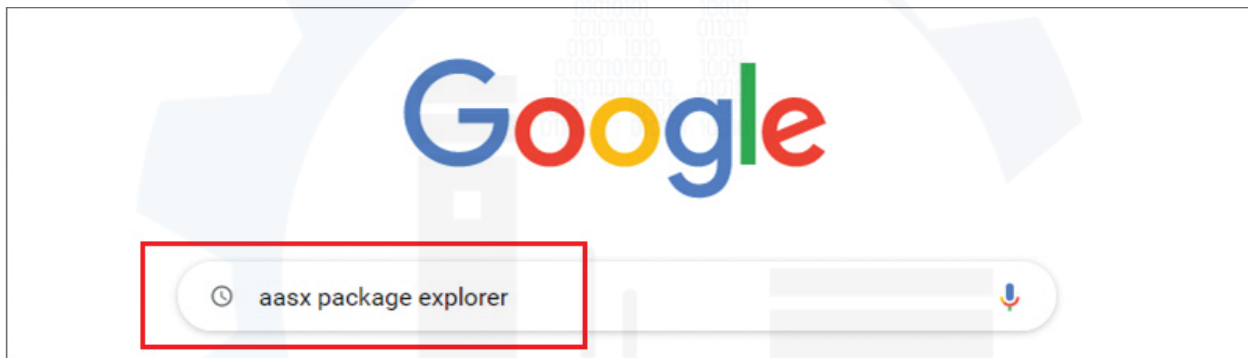
[표 4] 데이터 품질지수의 산출

5 부록_AAS S/W 설치 가이드

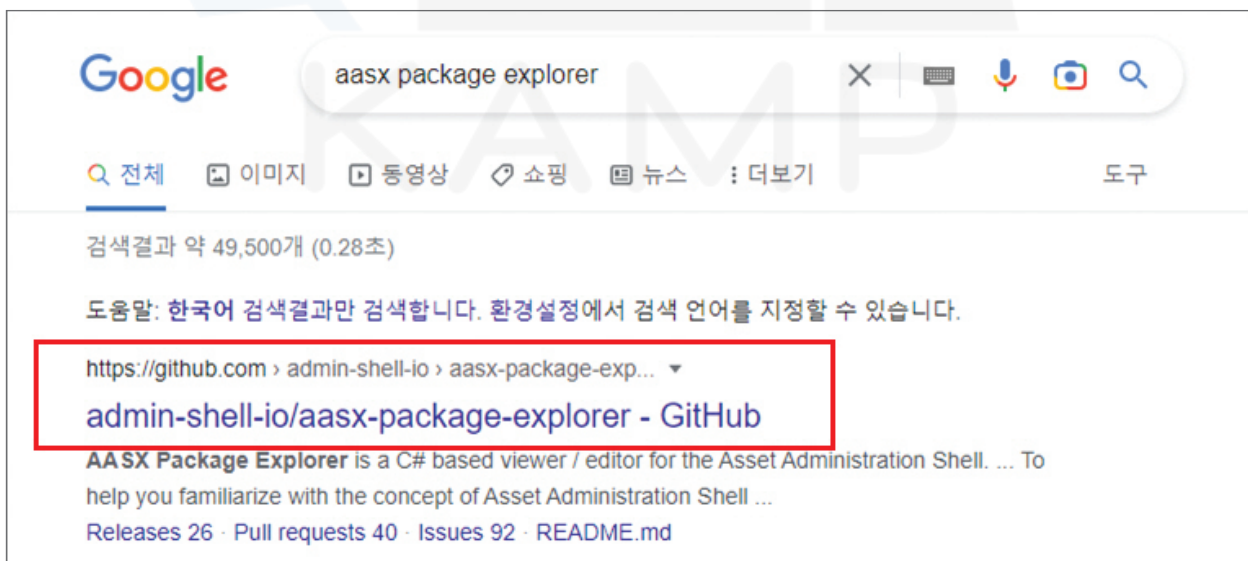
1. AASX Package Explorer 설치

AAS 모델은 독일 IDTA(Industrial Digital Twin Association)에서 제공하는 C# 기반 오픈 소스 소프트웨어 ‘AASX Package Explorer’를 사용하여 개발할 수 있다. 특히, AASX Package Explorer는 윈도우즈 10 및 윈도우즈 11 운영체제에서 실행 가능하고 GUI 또한 제공하기 때문에 처음 접하는 사용자도 어렵지 않게 AAS 모델을 개발 할 수 있다. 본 부록에서는 AAS 모델링에 필요한 소프트웨어 패키지 다운로드 방법과 AAS 파일 오픈 및 편집하기 위해 기본적인 메뉴 설명에 대해 안내한다.

① google 등의 검색 엔진에 ‘aasx package explorer’를 검색한다.



② 아래와 그림과 같이 ‘admin-shell-io/aasx-package-explorer GitHub’ 링크를 선택한다.



- ③ 페이지 하단의 Installation의 'the releases' 링크를 클릭한다.

Installation

We provide the binaries for Windows 10 in [the releases](#).

(Remark: In special cases you may like to use a current build. Please click on a green check mark and select "Check-release" details.)

- ④ aasx-package-explorer.날짜명.zip 이름의 파일을 다운로드 받는다. 단, 본 가이드북에서는 당시 최신 버전인 'aasx-package-explorer.2022-08-06.zip'를 사용

AASX Package Explorer 2022-08-06.alpha Latest

Add first version of signature ([https://github.com/admin-shell-io/aasx-package-explorer/pull/501\[\]](https://github.com/admin-shell-io/aasx-package-explorer/pull/501[]))
<https://github.com/admin-shell-io/aasx-package-explorer/commit/dc5984429699dc50a51f982a7ae273d54df73624>)
 Signing and validating of submodels or collections
 Prototype according actual work of Task Force Secure AAS
 Click on submodel or collection and choose "File / Security / Sign or Validate"
 Multiple signatures with different X509 certificates are possible

[Update test certificate](#)

▼ Assets 5

aasx-package-explorer-blazorui.2022-08-06.zip	3.63 MB	06 Aug 2022
aasx-package-explorer-small.2022-08-06.zip	36.9 MB	06 Aug 2022
aasx-package-explorer.2022-08-06.zip	115 MB	06 Aug 2022
Source code (zip)		05 Aug 2022
Source code (tar.gz)		05 Aug 2022

- ⑤ 다운로드한 'aasx-package-explorer.2022-08-06.zip' 파일의 압축을 해제한다.

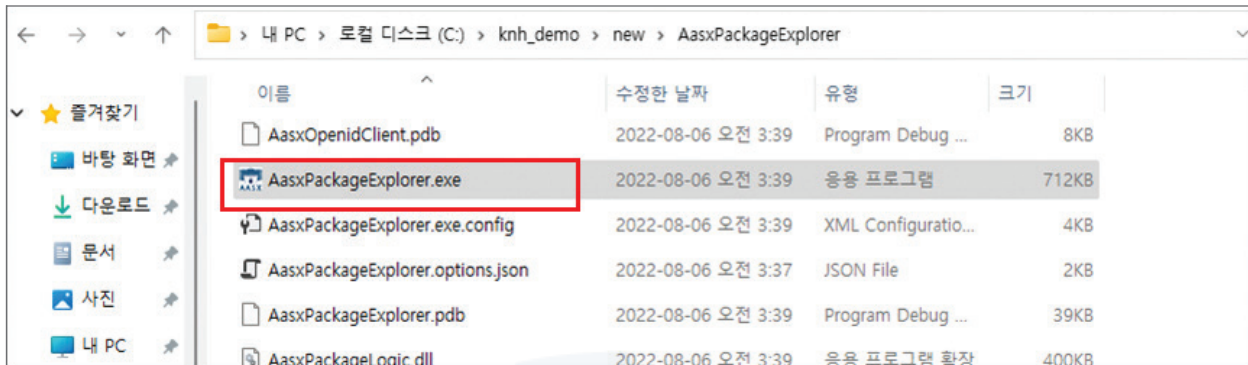
new

새로 만들기 | | | | | | | 정렬 | 보기 | ...

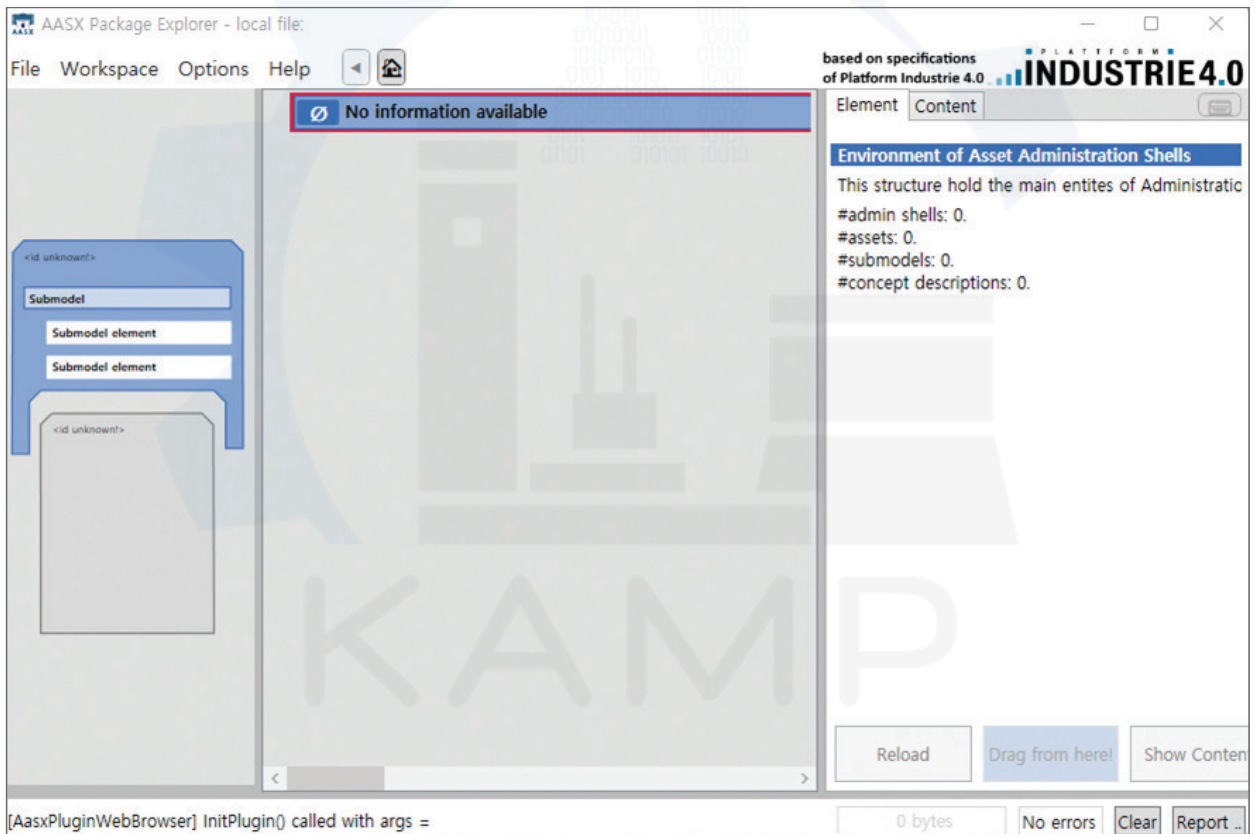
← → ^ ↑ > 내 PC > 로컬 디스크 (C:) > knh_demo > new

이름	수정한 날짜	유형	크기
AasxPackageExplorer	2022-10-27 오후 1:15	파일 폴더	
aasx-package-explorer.2022-08-06.zip	2022-10-27 오후 1:13	압축(ZIP) 파일	117,850KB

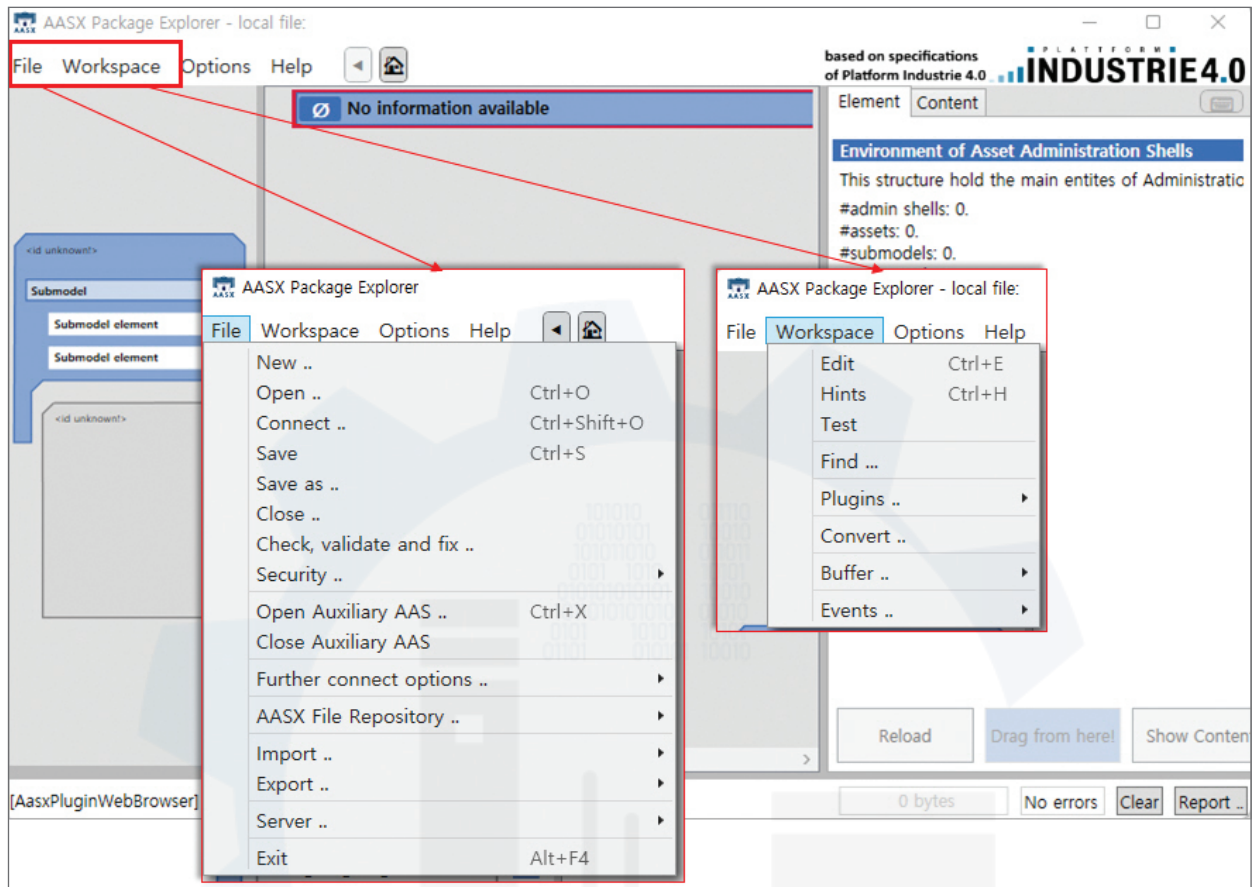
- ⑥ 디렉토리 'AasxPackageExplorer'을 열어서 실행파일인 'AasxPackageExplorer.exe'를 클릭하여 프로그램을 실행한다.



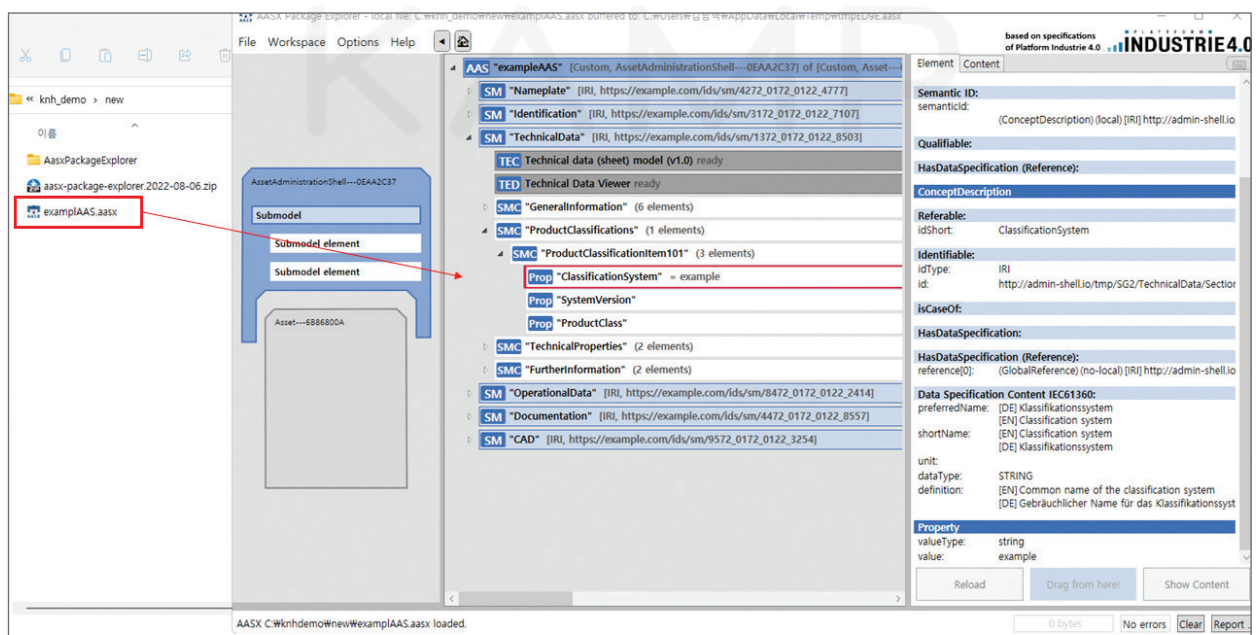
- ⑦ 아래와 같이 실행화면이 나타나면 AASX Package Explorer가 정상적으로 잘 동작하는 것이므로 AAS 개발 환경이 준비되었음을 의미한다.



- ⑧ AASX Package Explorer에서 AAS 생성/편집/뷰 기능은 아래와 같이 'File'과 'Workspace' 메뉴를 통해 사용 가능하다. 기본 확장자는 'aasx'이며, 필요한 경우 'Save as' 또는 'Export' 기능을 통해 JSON이나 XML 확장자로 변환하여 사용 할 수 있다.



- ⑨ aasx 파일은 AASX Package Explorer의 File 메뉴에서 'Open'을 클릭하거나 화면에 aasx 파일을 드래그앤 드롭하여 작성된 AAS 모델을 뷰/편집 할 수 있다.





「전자부품(배터리팩) 예지보전 AI 데이터셋」 분석실습 가이드북